

difes: the technical basis

The algorithm registry: every numerical and metrological method the program uses, with its equation, its choice, its error, its counterparts, its implementation and a check anyone can run

The difes project · Josu Igartua, UPV/EHU

8 October 2026

Table of contents

What this document is	3
The registry in numbers	4
Numerics	4
N01 · Non-linear least squares: trust-region reflective with bounds	4
N02 · Parameter errors and covariance from the Jacobian at the optimum	5
N03 · Jacobians: analytic where the terms allow it, central differences otherwise	6
N04 · Principal-value integrals in closed form on a cubic spline (no quadrature)	7
N05 · Constrained finite-range correction as bounded least squares with a hinge	9
N06 · Multiply subtractive Kramers–Kronig: a Lagrange polynomial in ω^2	10
N07 · The error models of the finite-range correction: log poles, a slope, a piecewise-linear extension	11
N08 · The variational dielectric function: causal triangles with their Kramers–Kronig partner in closed form	12
N09 · The causality check: Maclaurin’s rule on a wide uniform grid	13
N10 · FOCUS’s point-list functions: a monotone cubic with Fritsch–Butland slopes, and its Hilbert integral in closed form	14
N11 · Voigt, causal Voigt and causal Gaussian through the Faddeeva and Dawson functions . . .	15
N12 · Passivity of the four-parameter model as a hinge penalty	16
N13 · Linear interpolation, held at the ends: resampling, tabulated functions, point-list tables . .	17
N14 · Multiple reflections in a plate: closed-form sums or a counted number of rays	18
N15 · Numerical derivative of a spectrum: second-order central differences	19
N16 · Random numbers: a seeded PCG64 generator for the Monte Carlo, a legacy global one in the formula language	20
N17 · χ^2 , weights, σ and degrees of freedom: the conventions behind every reported number . . .	21

Metrology	22
M01 · Law of propagation of uncertainty at the least-squares optimum (GUM §5, Supplement 2)	22
M02 · Monte Carlo propagation of distributions (GUM Supplement 1)	23
M03 · The input budget: Type A and Type B components, their distributions and effect vectors	24
M04 · Coverage factor: $k = 2$ by convention, or from the t-distribution at Welch–Satterthwaite’s effective degrees of freedom	26
M05 · What the budget cannot say: the reduced χ^2 note, the inflated Type A, parameters at a bound	26
M06 · Derived quantities and curve bands: the gradient of $q(p)$ through the parameter covariance	27
M07 · Uncertainty through an inversion: the budget propagated by Monte Carlo alone	28
M08 · Checking the statement: the stated intervals must cover the truth	30
Dependencies	31
D01 · <code>scipy.optimize.least_squares</code>	31
D02 · <code>numpy.random.default_rng</code> (PCG64)	31
D03 · <code>scipy.interpolate.CubicSpline</code> (natural boundary)	32
D04 · <code>numpy.linalg.pinv</code> (SVD pseudo-inverse)	32
D05 · <code>scipy.stats.t.ppf</code> and <code>norm.ppf</code>	32
D06 · <code>scipy.special.wofz</code> and <code>dawsn</code>	33
D07 · <code>numpy.interp</code> , <code>numpy.trapezoid</code> , <code>numpy.gradient</code> , <code>numpy.quantile</code>	33
Physics	33
P01 · The dielectric model: ε_∞ times the factorised modes plus the additive susceptibilities, with bounded, freezable parameters	34
P02 · The Drude term: free carriers	35
P03 · The Lorentz oscillator, in plasma-frequency and in strength form	36
P04 · The four-parameter factorised mode (Gervais–Piriou, FOCUS’s phonon-4p)	37
P05 · The causal Gaussian band for disordered solids (FOCUS’s gaussian-kkg)	38
P06 · The Fano-shaped Lorentz oscillator (RefFIT’s codes –65 and –66)	39
P07 · The Tauc–Lorentz oscillator (Jellison and Modine, RefFIT’s codes –55 and –56)	40
P08 · A user formula as a fittable term	40
P09 · Effective media: Bruggeman, Maxwell–Garnett, Looyenga and Lichtenecker, as a fittable model	41
P10 · From ε to the bulk sample: the principal root, the Fresnel amplitude, R and Kirchhoff’s $E = 1 - R$	42
P11 · The free-standing slab: coherent (Airy) and incoherent sums, and $E = 1 - R - T$	43
P12 · A film on an opaque substrate: the coherent amplitude and two incoherent forms, FOCUS’s and the energy-conserving one	44
P13 · Layer stacks on an opaque substrate: FOCUS’s nested incoherent recursion, or the coherent Airy recursion	45
P14 · Fresnel coefficients at oblique incidence, p and s	46
P15 · What is computed from what: the quantity dispatch and the simulated spectrum	47
P16 · The spectrum type and its units: wavenumbers in cm^{-1} , strictly ascending, bounded quantities clamped within a tolerance	48
P17 · RefFIT’s output quantities and its extended Drude analysis	49

Language	50
L01 · The grammar of the math language: a recursive-descent parser over FOCUS's expression syntax	50
L02 · Evaluation without eval: a tree walk over numpy with a closed table of functions	51
L03 · User functions: <code>name(args) = body</code> , with defaults, a free-name check, substitution and unparsing	52
L04 · FOCUS's absorption profiles and peak functions as language functions, with the sign convention made explicit	53
L05 · Libraries at three scopes, with provenance and promotion	54
L06 · The session graph and the views: named nodes, references, lazy cached evaluation, declarative plots	55
Exchange	56
X01 · Reading OSML: FOCUS's symbols and definitions as formulas	56
X02 · Reading a FOCUS dielectric function as a difes model, term by term	57
X03 · Writing OSML: a difes model as a FOCUS model document, with its reflectivity	58
X04 · A FOCUS project as a difes session	59
X05 · FOCUS's binary project (.pjt): the numbers, found by their shape	60
X06 · RefFIT's model table and its codes, mapped to difes terms and back	61
X07 · RefFIT's files: PAR.DAT in two layouts, the ASC listing, the binary RFM in two layouts, datasets and exports	62
X08 · Numeric text tables in and out: delimiters, headers, column selection, exponents, error bars	63
X09 · Receiving a colleague's folder: inventory, classification, matching by name or by numbers, refits, a report	64
X10 · Curves out of published figures: colour masks on a calibrated bitmap, or the vector paths of a PDF page	65
What the registry says about FOCUS	66

What this document is

The test ledger says what difes reproduced and against what. This document says how: it is the registry of the methods the program uses, one card per method, written for two readers. The auditor wants to know what the program computes, why that way, with what error, and how to check it without reading the code; the maintainer wants to know where the method lives, what it depends on, and what would catch a change. Both halves are on every card. The first half is written by hand: the statement, the equation in the notation of the theory manual, the choice and its alternatives, the error and its order, the reference, and what FOCUS and RefFIT do at the same place together with the class of evidence for that (documented in the program's help or paper; inferred from the program's own numbers in a record of the ledger; unknown). The second half is read from the package when this document is built: the symbols with their file and line, their signatures, the SciPy and NumPy routines their source calls, the parameters and bounds the terms declare, and every test function that names the symbols. Every card closes with a check, a few lines of Python whose last printed line must be the one the card states; the build runs them all before writing this document and refuses a card whose check fails, and the package's test suite runs them again at every release. A card that is still true is a card whose check still passes.

Six natures: the numerics and the metrology, where an auditor's doubt lives; the physics, where the parameters and bounds on the cards are read from the terms themselves; the language; the exchange, where the facts learned the hard way about FOCUS's and RefFIT's files are written down; and the borrowed routines with their licences.

The registry in numbers

cards	65: numerics 17, metrology 8, dependencies 7
checks	65 of 65 passed
environment	difes 0.34.0, numpy 2.5.3, scipy 1.18.1, Python 3.12.7, 2026-10-08

Numerics

integration, interpolation, differentiation, optimisation, linear algebra, special functions, random numbers.

N01 · Non-linear least squares: trust-region reflective with bounds

Numerics · optimisation

Statement. Every free parameter reachable from the fit terms is gathered into one vector $\mathbf{x}$ with its bounds; the residual vector is $(\text{model} - \text{data}) \times \sqrt{w}/\sigma$ per point, all terms concatenated, plus a passivity penalty row per model when a four-parameter mode is present (N12); `scipy.optimize.least_squares` minimises half the squared norm by the trust-region-reflective method. The method “lm” (MINPACK’s `lmdr`) is accepted but falls back to TRF as soon as any bound is finite, because MINPACK takes none. The fitter works on copies of the models unless asked to work in place, and clips the start into the bounds.

$$\chi^2 = \sum_t w_t \sum_i \frac{[y_i - f_t(\omega_i; \mathbf{p})]^2}{\sigma_i^2}, \quad \min_{\mathbf{p}} \frac{1}{2} \|\mathbf{r}\|^2, \quad r_i = \frac{\sqrt{w_t}}{\sigma_i} [f_t(\omega_i; \mathbf{p}) - y_i]$$

Why this choice. TRF (Branch, Coleman and Li 1999) rather than Levenberg–Marquardt because the expert’s controls are bounds and frozen parameters, and Levenberg–Marquardt has no bounds. The tolerances are SciPy’s defaults (`ftol`, `xtol` and `gtol` at 10^{-8}) and no parameter scaling is applied, a known weakness for a plasma frequency of 10^4 cm^{-1} next to a damping of 10 cm^{-1} that the analytic Jacobian (N03) hides in practice; `x_scale` is passed through to SciPy for the cases where it matters. A thickness given as a parameter has its lower bound raised to 10^{-9} cm .

Error and its order. Converges to a stationary point of χ^2 , not necessarily the global minimum; the recovery tests of the ledger (T04, T42) bound this on the problems of the group, and the two-fitter comparison with RefFIT (T42, T46, T47) shows one minimum for both.

Reference. Branch, Coleman and Li, *SIAM J. Sci. Comput.* 21 (1999) 1; SciPy’s `least_squares` documentation; Nocedal and Wright, *Numerical Optimization* (2006), ch. 10

Inputs and domains.

- `x`: the free parameters; bounds [lower, upper] from each `Parameter`, infinite allowed; frozen parameters never reach the optimiser
- `σ_i`: the spectrum’s error bars, or a constant sigma per term; 1 when neither is given (then χ^2 is a plain sum of squares)
- `w_t`: the term’s weight (default 1), for multi-spectrum fits
- `method`: `trf` (default), `lm` (unbounded problems only), `dogbox` (passed through, not used by `difes`)

Counterparts. FOCUS (unknown): FOCUS’s help says only that ‘a non-linear least square optimization method’ is used (`fit-panel.html`, `data-analysis.html`); none of its eighteen papers names it. Numerically equivalent to `difes` on T06–T16 and T42–T48. RefFIT (documented): RefFIT documents Levenberg–Marquardt; on RefFIT’s own tutorial data the two fitters stop at the same minimum (T42).

Implementation.

- `fit(terms: list[FitTerm], *, inplace: bool = False, jacobian: str = auto, method: str = trf, max_nfev: int | None = None, passivity_penalty: float = 1000.0, verbose: int = 0, **ls_kwargs: Any) -> FitResult` — `src/difes/fit.py:235`

Library routines called. `numpy.linalg.pinv` (D04); `scipy.optimize.least_squares` (D01)

Tests that name these symbols. `test_bench.py::test_bench_reads_a_graph_built_session`, `test_bench.py::test_fit_asks_then_adopts`, `test_bench.py::test_cli_bench_on_irati_spectrum`, `test_bench.py::test_layers_and_thicknesses_in_the_bench`, `test_bench.py::test_the_shared_chromium_session`, `test_cli.py::test_fit_command_recovers_parameters`, `test_expr.py::test_expression_term_equals_builtin_lo`, `test_fit.py::test_r1_recovery_from_wrong_guess`, `test_fit.py::test_inplace_updates_callers_model`, `test_fit.py::test_frozen_parameter_stays_and_has_no_error` and 33 more.

Ledger records. T04 (The fitter on synthetic truth: recovery, errors, bounds, linking, Jacobians); T42 (RefFIT's own tutorial, Part 1: RefFIT's data and saved models, and difes's fitter against RefFIT's); T46 (RefFIT's own tutorial, Part 2: four datasets fitted together, RefFIT's weighted χ^2 to the printed digit); T47 (RefFIT's own tutorial, Part 4: the CuO phonon against temperature, the macro's loop in difes against Fig. 3-30).

Check it yourself. The last printed line must be `True 0.0`.

```
import numpy as np
from difes.models import example_model
from difes.optics import simulate
from difes.fit import FitTerm, fit
m = example_model(); w = np.linspace(100, 5000, 500)
spec = simulate(m, w, "R") # a noise-free spectrum of the example
    ↪ model
m2 = example_model()
for _, p in m2.parameters(): p.value *= 1.05 # every parameter 5 % off
r = fit([FitTerm(spec, m2)])
print(r.success, round(r.chi2, 8)) # back on the truth: chi-square zero
```

Printed `True 0.0` at this build, in 0.01 s.

Changelog of the method. 0.1.0: the fitter, finite-difference Jacobian; 0.9.0: analytic Jacobian when every term supports it (N03); 0.14.0: the passivity penalty of the four-parameter model (N12).

N02 · Parameter errors and covariance from the Jacobian at the optimum

Numerics · linear-algebra

Statement. After the fit the covariance is the pseudo-inverse of $J^T J$, multiplied by the reduced χ^2 , with J the optimiser's final Jacobian restricted to the data rows (the penalty rows dropped); each parameter's error is the square root of the diagonal. The pseudo-inverse is numpy's SVD-based `pinv`, so a degenerate pair (the two Drude terms of chromium) gives a large finite error instead of a failure; the rank deficiency itself is not reported. A `LinAlgError` gives NaN errors rather than an exception.

$$V_{\mathbf{p}} = (J^T J)^+ \frac{\chi^2}{n - m}, \quad u(p_j) = \sqrt{(V_{\mathbf{p}})_{jj}}$$

Why this choice. Scaling by the reduced χ^2 is the usual practice (`curve_fit` does the same) and assumes the error bars are understated by a common factor. The unscaled covariance, which the GUM wants, is what M01 computes from its own Jacobian; the uncertainty report prints both. When no error bars are given $\sigma = 1$ and the scaling by $\chi^2/(n - m)$ is what makes the errors meaningful at all.

Error and its order. A linearisation at the optimum; nothing beyond J . The Monte Carlo of M02 is the check where the problem is not linear.

Reference. Press et al., Numerical Recipes, 3rd ed. (2007) §15.5 for the practice; JCGM 100:2008 §5 for what the unscaled matrix means

Counterparts. FOCUS (documented): FOCUS prints no uncertainties (help, Automatic Fit panel). RefFIT (documented): RefFIT prints confidence limits its manual calls human-dependent (T34).

Implementation.

- `fit(terms: list[FitTerm], *, inplace: bool = False, jacobian: str = auto, method: str = trf, max_nfev: int | None = None, passivity_penalty: float = 1000.0, verbose: int = 0, **ls_kwargs: Any) -> FitResult` — `src/difes/fit.py:235`
- `FitResult(success: bool, message: str, nfev: int, chi2: float, chi2_reduced: float, n_points: int, n_free: int, names: list[str], values: np.ndarray, errors: np.ndarray, covariance: np.ndarray, models: list[DielectricModel], terms: list[FitTerm], term_rms: list[float], jacobian: str, details: dict[str, Any] = <factory>) -> None` — `src/difes/fit.py:100`

Library routines called. `numpy.linalg.pinv` (D04); `scipy.optimize.least_squares` (D01)

Tests that name these symbols. `test_bench.py::test_bench_reads_a_graph_built_session`, `test_bench.py::test_fit_asks_then_adopts`, `test_bench.py::test_cli_bench_on_irati_spectrum`, `test_bench.py::test_layers_and_thicknesses_in_the_bench`, `test_bench.py::test_the_shared_chromium_session`, `test_cli.py::test_fit_command_recovers_parameters`, `test_expr.py::test_expression_term_equals_builtin_lo`, `test_fit.py::test_r1_recovery_from_wrong_guess`, `test_fit.py::test_inplace_updates_callers_model`, `test_fit.py::test_frozen_parameter_stays_and_has_no_error` and 33 more.

Ledger records. T35 (Uncertainties following the GUM: the stated intervals cover the truth).

Check it yourself. The last printed line must be `True`.

```
import numpy as np
from difes.models import DielectricModel, Lorentz
from difes.optics import simulate
from difes.fit import FitTerm, fit
from difes.uncertainty import evaluate_uncertainty
rng = np.random.default_rng(1); w = np.linspace(100, 2000, 191)
true = DielectricModel(eps_inf=3.0, terms=[Lorentz(w0=450.0, wp=600.0, gamma=20.0)])
spec = simulate(true, w, "R")
spec = spec.copy(y=spec.y + rng.normal(0, 0.003, w.size), yerr=np.full(w.size, 0.003))
m = DielectricModel(eps_inf=2.8, terms=[Lorentz(w0=460.0, wp=580.0, gamma=25.0)])
r = fit([FitTerm(spec, m)])
u = evaluate_uncertainty(r) # an empty budget: Type A only
# the GUM's Type A (its own Jacobian, unscaled) equals the fitter's errors with the
↳ chi-square scaling undone
print(bool(np.allclose(u.contributions[0].u, r.errors / np.sqrt(r.chi2_reduced),
↳ rtol=1e-3)))
```

Printed `True` at this build, in 0.01 s.

Changelog of the method. 0.1.0.

N03 · Jacobians: analytic where the terms allow it, central differences otherwise

Numerics · differentiation

Statement. Two Jacobians exist. The fitter's (N01) is analytic when every term and model declares derivatives (the built-in additive profiles, the variational term, and the bulk quantities R , E , ε' , ε'' , χ' , χ'' with no thickness, substrate, layers or scale) and SciPy's two-point forward difference otherwise; “auto” chooses, “fd” and “analytic” force. The GUM budget's Jacobian (M01) is always numerical: a central difference with step $h = 10^{-6} \times \max(|v|, 1)$, made one-sided when $v \pm h$ would cross a bound. The derived quantities of M06 use the same rule.

$$\frac{\partial f}{\partial p} \approx \frac{f(v+h) - f(v-h)}{2h}, \quad h = 10^{-6} \max(|v|, 1)$$

Why this choice. The relative step 10^{-6} is the usual compromise in double precision (truncation error of order h^2 , rounding of order ε/h , balanced near the cube root of ε , about 6×10^{-6}). The GUM Jacobian is numerical even when an analytic one exists so that it is independent of the fitter's; their agreement is one of the suite's checks. The one-sided step at a bound loses one order (h instead of h^2), which does not matter because a parameter at a bound is flagged in the report as not determined by the data (M05).

Error and its order. Relative error of order h^2 in the interior, of order h at a bound; for a Lorentz term the difference from the analytic derivative is at 10^{-7} .

Reference. Nocedal and Wright, Numerical Optimization (2006) §8.1

Counterparts. FOCUS (unknown): no note RefFIT (unknown): no note

Implementation.

- `_analytic_jacobian`(terms: list[FitTerm], models: list[DielectricModel], free: list[tuple[str, Parameter]], penalty_rows: int) -> Callable[[np.ndarray], np.ndarray] — src/difes/fit.py:191
- `_analytic_possible`(terms: list[FitTerm], models: list[DielectricModel]) -> bool — src/difes/fit.py:177
- `jacobian_at_optimum`(terms: Sequence[FitTerm], free: list[tuple[str, Parameter]], rel: float = 1e-06) -> np.ndarray — src/difes/uncertainty.py:174

Tests that name these symbols. `test_fit.py::test_analytic_jacobian_matches_finite_differences`.

Ledger records. T35 (Uncertainties following the GUM: the stated intervals cover the truth).

Check it yourself. The last printed line must be `analytic finite-difference True`.

```
import numpy as np
from difes.models import DielectricModel, Lorentz
from difes.optics import simulate
from difes.fit import FitTerm, fit
rng = np.random.default_rng(1); w = np.linspace(100, 2000, 191)
true = DielectricModel(eps_inf=3.0, terms=[Lorentz(w0=450.0, wp=600.0, gamma=20.0)])
spec = simulate(true, w, "R")
spec = spec.copy(y=spec.y + rng.normal(0, 0.003, w.size), yerr=np.full(w.size, 0.003))
m = DielectricModel(eps_inf=2.8, terms=[Lorentz(w0=460.0, wp=580.0, gamma=25.0)])
ra = fit([FitTerm(spec, m)], jacobian="analytic")
rf = fit([FitTerm(spec, m)], jacobian="fd")
print(ra.jacobian, rf.jacobian, bool(np.allclose(ra.values, rf.values, rtol=1e-5)))
```

Printed analytic finite-difference True at this build, in 0.01 s.

Changelog of the method. 0.9.0: analytic Jacobian of the built-in terms and the bulk quantities; 0.20.0: the GUM's own central-difference Jacobian, one-sided at bounds.

N04 · Principal-value integrals in closed form on a cubic spline (no quadrature)

Numerics · integration

Statement. Every Kramers–Kronig integral of the form $P \int_{s_i}^{s_{i+1}} f(s)/(s^2 - w^2) ds$ over the finite range $[a, b]$ is evaluated by interpolating f through its grid with a natural cubic spline (piecewise linear below four points or on request), splitting $1/(s^2 - w^2)$ into partial fractions, and integrating each cubic piece against $1/(s - c)$ exactly: $P(c) \ln|s - c|$ plus the polynomial that the exact division of $P(s) - P(c)$ by $s - c$ leaves. The log terms are regrouped per node; at an interior node hit by the output point their coefficient vanishes by continuity, so $0 \times \ln 0$ is dropped; at the two endpoints the divergence is real and is kept, because it cancels against the boundary term of the derivative form of the phase. The transform is vectorised over the output grid. The phase has two forms, the paper's derivative form (Eq. 15) and the direct form; on a finite range they differ by the boundary term, which the card for the error models (N07) corrects.

$$P \int_{s_i}^{s_{i+1}} \frac{P_i(s)}{s - c} ds = P_i(c) [\ln|s - c|]_{s_i}^{s_{i+1}} + \int_{s_i}^{s_{i+1}} \frac{P_i(s) - P_i(c)}{s - c} ds, \quad \frac{1}{s^2 - w^2} = \frac{1}{2w} \left[\frac{1}{s - w} - \frac{1}{s + w} \right]$$

Why this choice. De Sousa Meneses, Rousseau and Echegut 2007, Eq. 3–4 with $n = 3$. Chosen over quadrature (OptiCFRC's `scipy.integrate.quad` with a Cauchy weight, kept as the test oracle in the fixtures) because quadrature near the pole costs one call per output point and is noisy there, while the closed form

is exact for the spline and does 10^4 points in milliseconds. The natural boundary condition is the paper's; a not-a-knot spline would move the behaviour at the ends slightly. The value at $w = 0$ is left to the caller ($\chi'(0)$ by the trapezoid rule).

Error and its order. The spline's interpolation error, and it is how finely the measurement samples a band that sets it. For a Lorentz band the relative error of χ' at the band is 7 % with 2 points per width (the FWHM of χ''), 1 % with 4, 0.4 % with 5, 5×10^{-5} with 10 and 3×10^{-6} with 20 (measured on 2026-10-08). Resampling the data onto a finer grid before transforming changes nothing (checked: the same 1 % at 4 points per width after a spline to a grid five times finer), because the transform already integrates the spline through the data; the information is the measurement's, and the instrument's resolution decides it. Rule for users: a band sampled with fewer than ten points per width transforms with an error above 10^{-4} of its peak, with fewer than five at the per-cent level; a spectrum at 4 cm^{-1} with phonon bands of 20 cm^{-1} or more is at or below 0.4 %. The truncation of the range is not an error of the method but of the data (the tail of a Lorentz beyond $20\,000 \text{ cm}^{-1}$ contributes 10^{-7}) and is what N06 and N05 correct.

Reference. De Sousa Meneses, Rousseau and Echegut, Appl. Spectrosc. 61 (2007) 1390, Eq. 3–4, 7 and 15

Counterparts. FOCUS (inferred from numbers): The paper is FOCUS's own; its figures are reproduced in T20–T24. RefFIT (documented): RefFIT reaches the real part by the variational dielectric function (N08), not by this transform.

Implementation.

- `_piecewise_coefficients(s: np.ndarray, f: np.ndarray, interp: str) -> np.ndarray` — `src/difes/kk.py:43`
- `_pv_over_s_minus_c(s: np.ndarray, coef: np.ndarray, c: np.ndarray) -> np.ndarray` — `src/difes/kk.py:54`
- `pv_over_s2(s: np.ndarray, f: np.ndarray, w: np.ndarray, interp: str = cubic) -> np.ndarray` — `src/difes/kk.py:84`
- `pv_linear_over_s2(s: np.ndarray, f: np.ndarray, w: np.ndarray) -> np.ndarray` — `src/difes/kk.py:96`
- `chi1_kk(s: np.ndarray, chi2: np.ndarray, w: np.ndarray | None = None, interp: str = cubic) -> np.ndarray` — `src/difes/kk.py:102`
- `phase_kk(s: np.ndarray, R: np.ndarray, w: np.ndarray | None = None, form: str = derivative, interp: str = cubic) -> np.ndarray` — `src/difes/kk.py:113`
- `phase_from_reflectivity(spec: Spectrum, form: str = derivative) -> Spectrum` — `src/difes/kk.py:333`
- `chi1_from_chi2(spec: Spectrum) -> Spectrum` — `src/difes/kk.py:340`

Library routines called. `numpy.trapezoid` (D07); `scipy.interpolate.CubicSpline` (D03)

Tests that name these symbols. `test_kk.py::test_pv_closed_form_matches_quadrature_on_the_same_interpolated`, `test_kk.py::test_cubic_kernel_matches_quadrature_on_the_spline`, `test_kk.py::test_pv_is_finite_at_nodes`, `test_kk.py::test_chi1_from_chi2_recovers_lorentz_on_wide_range`, `test_kk.py::test_phase_from_reflectivity`, `test_kk.py::test_truncation_error_is_visible_on_narrow_range`, `test_kk.py::test_against_opticfrc_fixture`, `test_kk.py::test_speed_two_thousand_points`, `test_kk.py::test_mskk_hits_anchors_and_reduces_error`, `test_kk.py::test_cfrc_phase_log_terms_with_anchors_beats_mskk_away_from_anchors` and 3 more.

Ledger records. T20 (Four one-band exports: the Gaussian convention closed, FOCUS's Voigt characterised); T21 (Pyrolytic boron nitride, González de Arrieta et al. (2017): the $691 \text{ }^\circ\text{C}$ fit).

Check it yourself. The last printed line must be 0.0001.

```
import numpy as np
from difes.kk import chi1_kk
s = np.linspace(1, 20000, 20000); w0, g = 500.0, 20.0 # spacing 1 cm-1, one twentieth
    ↪ of the damping
chi = w0**2 / (w0**2 - s**2 - 1j*g*s) # a Lorentz band of strength 1;
    ↪ its real part peaks near 12.6
sel = (s > 100) & (s < 2000)
w = s[sel][:5] # every fifth point of the band
    ↪ region as output
chi1 = chi1_kk(s, chi.imag, w) # the transform of the imaginary
    ↪ part
print(round(float(np.max(np.abs(chi1 - chi.real[sel][:5]))), 4))
```

Printed 0.0001 at this build, in 2.43 s.

Changelog of the method. 0.3.0: closed form replaces quadrature; OptiCFRC's quadrature kept as the oracle.

N05 · Constrained finite-range correction as bounded least squares with a hinge

Numerics · optimisation

Statement. The few parameters $\mathbf{x}$ of an error model (N07) are fitted so that the truncated transform plus the correction passes through the anchors (one residual per anchor) and/or stays non-negative on the whole grid (the phase-positiveness constraint, as a hinge: `positive_weight` times the negative part of the corrected function, point by point). With the constraint alone the problem is one-sided, so when no anchors are given a weak pull of the corrected function toward zero is added (regularisation $\times$ corrected over the scale and the root of the number of points) and the hinge keeps it non-negative: the corrected phase comes down until its minima touch zero. Solved by `least_squares` with the model's bounds, `x_scale` from the Jacobian, at most 2000 evaluations.

$$\min_{\mathbf{x}} \sum_a [g(\omega_a; \mathbf{x}) - (\theta_a - \theta_{kk}(\omega_a))]^2 + \lambda_+^2 \sum_i \min(\theta_{kk} + g, 0)^2 + \lambda_r^2 \sum_i \frac{(\theta_{kk} + g)^2}{S^2 n}$$

Why this choice. The paper states the constraint but not how it is enforced; a hinge inside a least-squares residual keeps one solver for the anchor case and the positivity case. The pull toward zero is difes's addition, named, defaulted to 1 and switched off whenever anchors are given, because the paper's examples (quartz, NdGaO_3) rest on the phase of a semitransparent crystal nearly vanishing between bands and the hinge alone does not bring it there.

Error and its order. Depends on the family of the error model; the paper's figures are met in T22–T24. The regularisation weight is a choice, and the card says so; an anchor is met to the solver's tolerance.

Reference. De Sousa Meneses et al. 2007, Eq. 12–18

Counterparts. FOCUS (unknown): FOCUS implements the CFRC (it is the paper's own tool) but how the constraint is enforced is not published; the paper's figures match (T22–T24). RefFIT (documented): Not in RefFIT.

Implementation.

- `cfrc(w: np.ndarray, F_kk: np.ndarray, model: ErrorModel, anchors: Sequence[tuple[float, float]]) = (), positive: bool = False, positive_weight: float = 10.0, regularisation: float = 1.0, F_kk_at: np.ndarray | None = None, x0: np.ndarray | None = None) -> CfrarResult` — `src/difes/kk.py:286`
- `CfrarResult(corrected: np.ndarray, correction: np.ndarray, params: dict[str, float], anchor_residuals: np.ndarray, success: bool, message: str, nfev: int, details: dict[str, Any] = <factory>) -> None` — `src/difes/kk.py:274`

Library routines called. `numpy.interp` (D07); `scipy.optimize.least_squares` (D01)

Tests that name these symbols. `test_cli.py::test_kk_and_cfrc_commands`, `test_graph.py::test_transforms_ms`, `test_kk.py::test_cfrc_phase_log_terms_with_anchors_beats_mskk_away_from_anchors`, `test_kk.py::test_cfrc`, `test_kk.py::test_cfrc_chi_extension_with_anchors`, `test_kk.py::test_cfrc_needs_anchors_or_constraint`.

Ledger records. T22 (Boron nitride, the printed table and the thesis figure: one number in question); T23 (The literature's printed four-parameter tables: 36 of them, evaluated); T24 (Paper 17, hexagonal ErMnO_3 : exact vector data, a table that does not regenerate its fit).

Check it yourself. The last printed line must be `True True`.

```
import numpy as np
from difes.models import DielectricModel, Lorentz
from difes.optics import refractive_index
from difes.kk import phase_kk, cfrc, PhaseLogTerms
```

```

m = DielectricModel(eps_inf=3.0, terms=[Lorentz(w0=450.0, wp=600.0, gamma=20.0)])
s = np.linspace(100, 2000, 1901)
n = refractive_index(m.epsilon(s)); rt = (n - 1) / (n + 1)
R, theta = np.abs(rt) ** 2, np.angle(rt) # the true phase of the
↳ model
theta_kk = phase_kk(s, R, s) # the truncated
↳ transform
model = PhaseLogTerms(poles=[90.0, 2500.0]) # one pole per truncated
↳ side
res = cfrc(s, theta_kk, model, anchors=[(300.0, float(theta[200])), (1500.0,
↳ float(theta[1400]))])
print(res.success, float(np.max(np.abs(res.anchor_residuals))) < 1e-6)

```

Printed True True at this build, in 0.67 s.

Changelog of the method. 0.4.0: anchors; 0.5.0: the positivity hinge and the pull toward zero.

N06 · Multiply subtractive Kramers–Kronig: a Lagrange polynomial in ω^2

Numerics · interpolation

Statement. With anchors $(\omega_i, F_{\text{true}}(\omega_i))$ the truncated transform is corrected by the Lagrange interpolation polynomial in ω^2 through the anchor differences, so that the corrected function passes through every anchor exactly; the transform at the anchors is interpolated linearly from the grid unless given. Anchor wavenumbers must be distinct.

$$F(\omega) = F_{kk}(\omega) + \sum_i L_i(\omega^2) [F_{\text{true}}(\omega_i) - F_{kk}(\omega_i)], \quad L_i(\omega^2) = \prod_{j \neq i} \frac{\omega^2 - \omega_j^2}{\omega_i^2 - \omega_j^2}$$

Why this choice. The paper’s Eq. 9, as written. A polynomial through many anchors oscillates between them (Runge), which is the reason the CFRC (N05) with a physical error model exists; the MSKK is kept because it is exact at the anchors and is what the literature compares against.

Error and its order. Exact at the anchors; between them the polynomial’s own behaviour, which grows with the number of anchors.

Reference. De Sousa Meneses et al. 2007, Eq. 9; Palmer, Williams and Budde, J. Opt. Soc. Am. A 15 (1998) 2050

Counterparts. FOCUS (unknown): FOCUS has anchors in its CFRC; whether it also offers the plain MSKK is not stated in the help. RefFIT (documented): Not in RefFIT.

Implementation.

- mskk(w: np.ndarray, F_kk: np.ndarray, anchors: Sequence[tuple[float, float]], F_kk_at: np.ndarray | None = None) -> np.ndarray — src/difes/kk.py:137

Library routines called. numpy.interp (D07)

Tests that name these symbols. test_graph.py::test_transforms_mskk_cfrc_and_terms, test_kk.py::test_mskk_hits_anchors_and_reduces_error, test_kk.py::test_cfrc_phase_log_terms_with_anchor

Ledger records. T20 (Four one-band exports: the Gaussian convention closed, FOCUS’s Voigt characterised).

Check it yourself. The last printed line must be 0.5 -0.2.

```

import numpy as np
from difes.kk import mskk
w = np.linspace(100, 2000, 1901); F = np.sin(w / 300.0)
c = mskk(w, F, [(400.0, 0.5), (1200.0, -0.2)]) # two anchors with arbitrary
↳ target values
print(round(float(c[300]), 6), round(float(c[1100]), 6)) # w = 400 and 1200 are grid
↳ points

```

Printed 0.5 -0.2 at this build, in 0.0 s.

Changelog of the method. 0.4.0.

N07 · The error models of the finite-range correction: log poles, a slope, a piecewise-linear extension

Numerics · approximation

Statement. Three families of correction term, each with a few bounded parameters that N05 fits. PhaseLogTerms (Eq. 18): a sum of $\text{strength} \times \ln|(\omega + \text{pole})/(\omega - \text{pole})|$, one pole per truncated side, each the phase of a delta of absorption placed outside the range; the poles start at the given positions and move within $[0.2, 5]$ times them unless bounds are given. PhaseLogLinear (Table II, NdGaO₃): one such term plus a linear slope. ChiExtension (Eq. 14): χ'' extended below a and above b by knots, some fixed and some free, continuous with the measured ends and zero beyond the last knot; the correction is the Kramers–Kronig transform of the extension, done with the piecewise-linear kernel of N04.

$$g(\omega; \mathbf{x}) = \sum_k x_{2k-1} \ln \left| \frac{\omega + x_{2k}}{\omega - x_{2k}} \right| (+ x_3 \omega), \quad \Delta\chi'(\omega) = \frac{2}{\pi} P \int_{\text{ext}} \frac{s \chi'_{\text{ext}}(s)}{s^2 - \omega^2} ds$$

Why this choice. The paper uses cubic splines through the extension knots; difes uses piecewise-linear pieces so that the transform of the extension is exact (N04’s linear kernel) and the knots stay interpretable; smoothness is a lower-priority refinement noted on the site. The log term is clamped at 10^{-12} from its pole so that an output point on a pole does not produce an infinity.

Error and its order. The families are the paper’s; the fit of their parameters is N05’s. The linear extension underestimates a smooth tail’s curvature between knots.

Reference. De Sousa Meneses et al. 2007, Eq. 14 and 18, Table II

Counterparts. FOCUS (inferred from numbers): The paper’s Fig. 3–6 are reproduced with these families (T22–T24). RefFIT (documented): Not in RefFIT.

Implementation.

- `ErrorModel()` — `src/difes/kk.py:164`
- `PhaseLogTerms(poles: Sequence[float], strengths: Sequence[float] | None = None, pole_bounds: Sequence[tuple[float, float]] | None = None, strength_bounds: tuple[float, float] = (-10.0, 10.0))` — `src/difes/kk.py:179`
- `PhaseLogLinear(pole: float, x0: Sequence[float] | None = None, pole_bounds: tuple[float, float] | None = None)` — `src/difes/kk.py:203`
- `ChiExtension(a: float, b: float, chi2_a: float, chi2_b: float, low_knots: Sequence[tuple[float, float] | None] = (), high_knots: Sequence[tuple[float, float] | None] = (), bounds: tuple[float, float] = (0.0, 10000.0))` — `src/difes/kk.py:217`

Library routines called. `numpy.interp` (D07)

Tests that name these symbols. `test_kk.py::test_cfrc_phase_log_terms_with_anchors_beats_mskk_away_from_`
`test_kk.py::test_cfrc_phase_positivity_constraint_alone`, `test_kk.py::test_cfrc_chi_extension_with_anchors`
`test_kk.py::test_cfrc_needs_anchors_or_constraint`.

Ledger records. T22 (Boron nitride, the printed table and the thesis figure: one number in question); T23 (The literature’s printed four-parameter tables: 36 of them, evaluated); T24 (Paper 17, hexagonal ErMnO₃: exact vector data, a table that does not regenerate its fit).

Check it yourself. The last printed line must be `True` `['high_0@2000']` `True`.

```

import numpy as np
from difes.kk import PhaseLogTerms, ChiExtension
w = np.array([100.0, 500.0, 900.0])
g = PhaseLogTerms(poles=[1000.0]).correction(w, np.array([1.0, 1000.0])) # strength 1,
↳ pole at 1000
ext = ChiExtension(a=200.0, b=1000.0, chi2_a=0.1, chi2_b=0.05, high_knots=[(2000.0,
↳ None)])
c = ext.correction(w, np.array([0.0])) # a tail falling linearly from 0.05 at b to
↳ 0 at 2000
print(bool(np.allclose(g, np.log(np.abs((w + 1000) / (w - 1000))))), ext.names,
↳ bool(np.all(c > 0)))

```

Printed True ['high_0@2000'] True at this build, in 0.0 s.

Changelog of the method. 0.4.0: PhaseLogTerms; 0.5.0: PhaseLogLinear and ChiExtension.

N08 · The variational dielectric function: causal triangles with their Kramers–Kronig partner in closed form

Numerics · basis-expansion

Statement. RefFIT’s model-independent dielectric function on an anchor mesh. In the constrained form ε'' is the piecewise-linear curve through amplitudes A_i at the anchors, written as a sum of triangles; the real part of each triangle is its exact Kramers–Kronig transform, integrated piece by piece from the antiderivative of $x/(x^2 + t)/(x^2 - w^2)$, with the singular log terms cancelling between adjacent pieces wherever ε'' is continuous. The end amplitudes are frozen at zero (a step in ε'' would make ε' diverge). The basis is cached per grid, the term carries analytic derivatives (the basis columns), and the ordinary fitter (N01) adjusts the amplitudes with the formula model frozen underneath, RefFIT’s two-step recipe. The unconstrained form has independent ε' and ε'' curves, 2N parameters, for data that fix both.

$$\varepsilon_{\text{var}}(\omega) = \sum_i A_i \varepsilon_i(\omega), \quad \varepsilon_{1,i}(\omega) = \frac{1}{\pi} \left[\frac{g(\omega, \omega_{i-1})}{\omega_i - \omega_{i-1}} - \frac{(\omega_{i+1} - \omega_{i-1}) g(\omega, \omega_i)}{(\omega_i - \omega_{i-1})(\omega_{i+1} - \omega_i)} + \frac{g(\omega, \omega_{i+1})}{\omega_{i+1} - \omega_i} \right], \quad g(x, y) = (x+y) \ln |x+y| + (x-y)$$

Why this choice. The manual’s eqs. 2-20 to 2-22, so that causality holds by construction and no transform is ever applied to the data. Kept as a Term so that bounds, frozen values, errors and the analytic Jacobian come for free; a non-negativity bound on every amplitude is an option for the pure variational fit without a base model. When there is no base model, $\varepsilon = 1$ is a stationary point of R (zero reflectance, zero gradient) and the amplitudes are seeded away from it.

Error and its order. Exact for the piecewise-linear ε'' ; the mesh sets the resolution, and RefFIT’s default of the data frequencies is reproduced.

Reference. Kuzmenko, Rev. Sci. Instrum. 76 (2005) 083108; RefFIT manual §2.2.4–2.2.5 and §4.6.3

Counterparts. FOCUS (documented): Not in FOCUS. RefFIT (inferred from numbers): RefFIT’s own variational output is met on Irati’s protocol run (T34, T43).

Implementation.

- `hat_functions(w: np.ndarray, mesh: np.ndarray) -> np.ndarray` — `src/difes/vdf.py:45`
- `_segment_kk(w: np.ndarray, x0: float, x1: float, s: float, t: float) -> np.ndarray` — `src/difes/vdf.py:62`
- `triangle_basis(w: np.ndarray, mesh: np.ndarray) -> np.ndarray` — `src/difes/vdf.py:76`
- `default_mesh(spectra: Sequence[Spectrum], every: int = 1) -> np.ndarray` — `src/difes/vdf.py:95`
- `VDFTerm(mesh: Sequence[float], kk: bool = True, **params: Any) -> None` — `src/difes/vdf.py:102`
- `VDFResult(model: DielectricModel, term: VDFTerm, fit: Any, omega: np.ndarray, epsilon: np.ndarray, epsilon_base: np.ndarray) -> None` — `src/difes/vdf.py:183`

- `vdf_fit(spectra: Sequence[Spectrum] | Spectrum, base: DielectricModel | None = None, *, mesh: Sequence[float] | None = None, every: int = 1, kk: bool = True, sigma: float | None = None, weights: Sequence[float] | None = None, nonnegative: bool = False, seed: float = 0.5, **fit_kwargs: Any) -> VDFResult` — `src/difes/vdf.py:198`

Tests that name these symbols. `test_vdf.py::test_hat_functions_partition_unity_and_interpolate`, `test_vdf.py::test_triangle_real_part_is_the_kk_transform_of_the_imaginary_part`, `test_vdf.py::test_vdf_t`, `test_vdf.py::test_kk_vdf_recovers_epsilon_from_reflectivity`, `test_vdf.py::test_free_vdf_fits_eps1_and_e`, `test_vdf.py::test_nonnegative_pure_vdf_without_base`.

Ledger records. T32 (The variational dielectric function, RefFIT’s method, on a known case); T34 (RefFIT’s own output: the comparison protocol run by Irati, number against number); T43 (Irati’s own RefFIT fits of polished chromium (200 and 500 °C): her models, RefFIT’s curves, her data).

Check it yourself. The last printed line must be `True`.

```
from difes.vdf import VDFTerm
from difes.kk import causality_error
t = VDFTerm([300.0, 400.0, 500.0], a1=1.0)      # one triangle of eps'' between 300 and
↳ 500 cm-1
# its real part must be the Kramers-Kronig transform of its imaginary part (N09 measures
↳ the mismatch)
print(causality_error(t.epsilon, w_max=20000.0, h=1.0) < 1e-2)
```

Printed `True` at this build, in 0.02 s.

Changelog of the method. 0.17.0: the constrained and unconstrained forms, the two-step recipe.

N09 · The causality check: Maclaurin’s rule on a wide uniform grid

Numerics · integration

Statement. Whether a term is causal is measured, not assumed: the relative difference between its real part and the Kramers–Kronig transform of its imaginary part, at a few points of a wide uniform grid from 0 to `w_max`, with the principal value taken by the Maclaurin (odd/even) rule: for an output point at an even grid index the sum runs over the odd indices with double spacing, so that no term is singular. The same rule is the suite’s oracle for every built-in profile and for user-written formula terms.

$$\chi'(\omega_{2j}) \approx \frac{2}{\pi} 2h \sum_{k \text{ odd}} \frac{\omega_k \chi''(\omega_k)}{\omega_k^2 - \omega_{2j}^2}$$

Why this choice. A second, deliberately different integrator from N04 (a quadrature rule rather than a closed form), so that the two check each other; the grid to 40 000 cm⁻¹ at 1 cm⁻¹ makes the truncation error of a phonon or Drude term below 1 % of the scale. The threshold is the caller’s; the suite uses 1 %, 0.1 % for the causal Gaussian.

Error and its order. The rule’s own, of order h^2 , plus the truncation at `w_max`; a term with weight beyond `w_max` reads as slightly non-causal, which the tests account for by the grid they choose.

Reference. Ohta and Ishida, Appl. Spectrosc. 42 (1988) 952 (the Maclaurin method for Kramers–Kronig)

Counterparts. FOCUS (documented): No such check in FOCUS; a formula typed in FOCUS is used as typed. RefFIT (documented): None.

Implementation.

- `causality_error(chi, w_max: float = 40000.0, h: float = 1.0, w_eval: np.ndarray | None = None) -> float` — `src/difes/kk.py:350`

Tests that name these symbols. `test_expr.py::test_expression_term_causality_check`.

Ledger records. T01 (The profiles against physics: limits, closed forms, passivity, causality); T33 (ReffIT's model table and its line shapes).

Check it yourself. The last printed line must be `True True`.

```
import numpy as np
from difes.models import Lorentz
from difes.kk import causality_error
good = causality_error(Lorentz(w0=450.0, wp=600.0, gamma=20.0).epsilon, 40000.0, 1.0)
bad = causality_error(lambda x: 1j * np.exp(-(x - 450.0) / 20.0) ** 2), 40000.0, 1.0)
↪ # an absorption with no real part
print(good < 1e-2, bad > 0.5)
```

Printed `True True` at this build, in 0.0 s.

Changelog of the method. 0.1.0: in the suite's `confest`; 0.5.0: exposed as `kk.causality_error` for formula terms.

N10 · FOCUS's point-list functions: a monotone cubic with Fritsch–Butland slopes, and its Hilbert integral in closed form

Numerics · interpolation

Statement. `fritsch_carlson(x, point list)` is the monotone piecewise-cubic Hermite interpolant through a point list, with the node slopes of Fritsch and Butland (1984), the harmonic-mean formula weighted by the neighbouring interval lengths and set to zero where the secants change sign; those are the slopes that reproduce FOCUS's exported numbers (Mireia's Bergman curve to 3.9×10^{-6}), not the 1980 Fritsch–Carlson projection. `fritsch_carlson_hilbert(s, point list)` is the Bergman integral of that interpolant, taken in closed form interval by interval with the same device as N04 (the value of the cubic at s times a log, minus the polynomial quotient); `fritsch_carlson_integral` is its plain integral. `table_linear` reads a two-column table with linear interpolation (FOCUS's file-2c); `cubic_spline` is SciPy's natural spline.

$$d_i = \frac{w_1 + w_2}{w_1/\Delta_{i-1} + w_2/\Delta_i} \text{ if } \Delta_{i-1}\Delta_i > 0, \text{ else } 0; \quad w_1 = 2h_i + h_{i-1}, \quad w_2 = h_i + 2h_{i-1}; \quad F(s) = \int \frac{m(z)}{s-z} dz$$

Why this choice. Three slope rules were tried against FOCUS's 512-point curve (log L090); only Butland's met it. The card records an empirical identification of FOCUS's method, which is why the evidence is inferred from numbers and the ledger record is cited. The closed-form Hilbert integral is evaluated for complex s , which is what the Bergman representation needs.

Error and its order. The interpolant is C^1 and monotone where the data are; the integrals are exact for the interpolant.

Reference. Fritsch and Carlson, SIAM J. Numer. Anal. 17 (1980) 238; Fritsch and Butland, SIAM J. Sci. Stat. Comput. 5 (1984) 300; Bergman, Phys. Rep. 43 (1978) 377; FOCUS help, `osml-core.html`

Counterparts. FOCUS (inferred from numbers): T48: FOCUS's exported reflectance met to 3.9×10^{-6} at worst, 7.7×10^{-7} rms. ReffIT (documented): Not in ReffIT.

Implementation.

- `_fc_derivatives(z: np.ndarray, y: np.ndarray) -> np.ndarray` — `src/difes/expr.py:115`
- `_fc_coefficients(pts) -> tuple[np.ndarray, np.ndarray]` — `src/difes/expr.py:146`
- `_fritsch_carlson(x, pts)` — `src/difes/expr.py:160`
- `_fritsch_carlson_integral(lo, hi, pts)` — `src/difes/expr.py:169`
- `_fritsch_carlson_hilbert(s, pts)` — `src/difes/expr.py:183`
- `_table_linear(x, pts)` — `src/difes/expr.py:110`
- `_cubic_spline(x, pts, bc: str = natural)` — `src/difes/expr.py:213`

Library routines called. `numpy.interp` (D07); `scipy.interpolate.CubicSpline` (D03)

Tests that name these symbols. `test_bergman.py::test_fritsch_carlson_interpolates_and_stays_monotone`, `test_bergman.py::test_bergman_integrals_against_quadrature`, `test_focus_exact.py::test_raman_sno2_spline`

Ledger records. T48 (Mireia’s alumina project: her Bergman model read from the OSML export, FOCUS’s own curve met to 4e-6, her two complaints explained).

Check it yourself. The last printed line must be `True True 1.0`.

```
import numpy as np
from difes.expr import evaluate_formula
x = np.linspace(0, 3, 301)
y = np.real(evaluate_formula("fritsch_carlson(x, 0,0, 1,1, 2,1, 3,0.5)", x=x)) # rises,
↳ stays flat, falls
d = np.diff(y)
rising, falling = x[:-1] < 1, x[:-1] >= 2
print(bool(np.all(d[rising] >= -1e-12)), bool(np.all(d[falling] <= 1e-12)),
↳ float(y[100]))
```

Printed `True True 1.0` at this build, in 0.0 s.

Changelog of the method. 0.31.0: added; the slopes chosen against FOCUS’s numbers.

N11 · Voigt, causal Voigt and causal Gaussian through the Faddeeva and Dawson functions

Numerics · special-functions

Statement. The Voigt peak is the real part of the Faddeeva function $w(z) = \exp(-z^2) \operatorname{erfc}(-iz)$ at $z = X + iY$, with X the reduced distance to the centre and Y the ratio of Lorentzian to Gaussian widths, normalised by its value at the centre so that the peak is A ; the limit of zero Lorentzian width is the Gaussian and of zero Gaussian width the Lorentzian. The causal Voigt band (FOCUS’s voigt-kkg) is the mirrored pair of $w(z)$ at $\pm X_c$, its real parts the absorption and its imaginary parts the dispersion, since the imaginary part of w is the Hilbert transform of the real part; normalised so that $\chi''(X_c) = A$ exactly, the factor FOCUS applies. The causal Gaussian (gaussian-kkg and the CausalGaussian term) uses the Dawson function for the dispersive part, the published form without renormalisation, with its analytic derivatives in the term.

$$V(x) = A \frac{\operatorname{Re} w(X + iY)}{\operatorname{Re} w(iY)}, \quad X = 2\sqrt{\ln 2} \frac{x - x_c}{w_G}, \quad Y = \sqrt{\ln 2} \frac{w_L}{w_G}; \quad \chi_{kkg}(x) = A \frac{i[w(z_-) - w(z_+)]^*}{\operatorname{Re} w(iY) - \operatorname{Re} w(2X_c + iY)}$$

Why this choice. The exact Faddeeva function (SciPy’s `wofz`, Algorithm 916 of Zaghoul and Ali) rather than an approximation. FOCUS’s own Voigt was found, from one-band exports, to carry an approximation ripple of 1.5×10^{-3} of the peak in the Gaussian limit that no standard Faddeeva algorithm reproduces (T18); `difes` keeps the exact function and the ledger records the difference as explained.

Error and its order. `wofz` is accurate to about 10^{-13} relative; the difference from FOCUS is FOCUS’s approximation, 1.5×10^{-3} of the peak at worst.

Reference. Zaghoul and Ali, ACM Trans. Math. Softw. 38 (2011) 15; De Sousa Meneses, Malki and Echegut, J. Non-Cryst. Solids 352 (2006) 769 (the causal Gaussian); FOCUS help, `core-functions.htm`

Counterparts. FOCUS (inferred from numbers): The normalisation of voigt-kkg and the ripple of FOCUS’s Voigt were both found against FOCUS’s exports (T18, T20). RefFIT (documented): RefFIT has no Voigt profile; its Gaussian band is a different construction (T33).

Implementation.

- `_voigt(x, A, xc, wg, wl)` — `src/difes/expr.py:231`
- `_voigt_kkg(x, A, xc, wg, wl)` — `src/difes/expr.py:199`
- `_gaussian_kkg(x, A, xc, w)` — `src/difes/expr.py:221`

- CausalGaussian(**params: Any) -> None — src/difes/models.py:220

Library routines called. `scipy.special.dawsn` (D06); `scipy.special.wofz` (D06)

term	parameter	lower	upper
CausalGaussian	w0	0.0	inf
CausalGaussian	A	0.0	inf
CausalGaussian	sigma	1e-09	inf

Tests that name these symbols. `test_fit.py::test_analytic_jacobian_matches_finite_differences`, `test_focus_exact.py::test_single_band_exports_settle_the_conventions`, `test_models.py::test_gaussian_peak`, `test_models.py::test_fpsq_equals_lorentz_when_dampings_equal`, `test_models.py::test_gaussian_causal`, `test_osml.py::test_write_and_read_mixed_model`.

Ledger records. T01 (The profiles against physics: limits, closed forms, passivity, causality); T18 (The Borofloat plate: 32 causal Voigt bands, R and T of a 1 mm slab); T20 (Four one-band exports: the Gaussian convention closed, FOCUS's Voigt characterised).

Check it yourself. The last printed line must be `True 2.0`.

```
import numpy as np
from difes.expr import evaluate_formula
x = np.linspace(300, 700, 401)
v0 = evaluate_formula("voigt(x, 1, 500, 40, 0)", x=x)           # zero Lorentzian width
↪ ...
g = evaluate_formula("gaussian(x, 1, 500, 40)", x=x)           # ... is the Gaussian
vk = evaluate_formula("voigt_kkg(x, 2, 500, 40, 10)", x=np.array([500.0]))
print(bool(np.allclose(v0, g)), round(float(np.imag(vk[0])), 6)) # and the causal band
↪ peaks at A
```

Printed `True 2.0` at this build, in 0.0 s.

Changelog of the method. 0.1.0: CausalGaussian; 0.5.0: `voigt`, `gaussian_kkg` in the language; 0.12.0: `voigt_kkg` with FOCUS's normalisation.

N12 · Passivity of the four-parameter model as a hinge penalty

Numerics · constraint

Statement. A factorised (four-parameter) model is passive at high frequency only if, summed over its modes, the LO dampings exceed the TO dampings. The fitter adds one residual per model that is zero when the sum of $(\gamma_{TO} - \gamma_{LO})$ is at most zero and grows linearly with the violation times a scale (default 10^3), so the optimiser is pushed back as soon as it crosses. The penalty rows carry a zero row in the analytic Jacobian; their gradient is left to the optimiser's own handling.

$$r_{\text{pass}} = \lambda \max\left(0, \sum_{\text{modes}} (\gamma_{TO} - \gamma_{LO})\right)$$

Why this choice. A soft penalty rather than a hard constraint because the solver has no linear constraints and the violation is physically meaningful to see (a fit that wants it is telling the user something). The scale is a parameter of fit() so that it can be raised when the penalty is being traded against the data.

Error and its order. The constraint can be violated by a small amount proportional to one over the scale; the final model is checked for $\varepsilon'' \geq 0$ on the data grid in the suite.

Reference. Gervais and Piriou, J. Phys. C 7 (1974) 2374; De Sousa Meneses, Gruener, Malki and Echegut, J. Non-Cryst. Solids 351 (2005) 124

Counterparts. FOCUS (unknown): Whether FOCUS's autofit constrains the four-parameter model is not stated; its sample projects satisfy the condition (T15, T16). RefFIT (documented): RefFIT's factorised model is documented without a passivity constraint.

Implementation.

- `_passivity_penalty(models: list[DielectricModel], scale: float) -> list[float]` — `src/difes/fit.py:166`
- `FourParameterMode(**params: Any) -> None` — `src/difes/models.py:195`

term	parameter	lower	upper
FourParameterMode	w_to	0.0	inf
FourParameterMode	g_to	0.0	inf
FourParameterMode	w_lo	0.0	inf
FourParameterMode	g_lo	0.0	inf

Tests that name these symbols. `test_fit.py::test_fd_fallback_for_slab_and_fpsq`, `test_models.py::test_fps`, `test_models.py::test_fpsq_passive_when_lo_damping_not_smaller`, `test_models.py::test_fpsq_causal`, `test_osml.py::test_read_quartz_e_model`, `test_reffit.py::test_tolo_code_is_the_four_parameter_mode`.

Ledger records. T01 (The profiles against physics: limits, closed forms, passivity, causality); T06 (FOCUS's OSML files: quartz E, LiNbO₃ E, the four libraries, the round trip); T15 (FOCUS's sample projects: the spectra read from the binary .pjt files).

Check it yourself. The last printed line must be `[0.0] [20000.0]`.

```
from difes.models import DielectricModel, FourParameterMode
from difes.fit import _passivity_penalty
passive = DielectricModel(eps_inf=3.0, terms=[FourParameterMode(w_to=400.0, g_to=10.0,
↪ w_lo=600.0, g_lo=30.0)])
active = DielectricModel(eps_inf=3.0, terms=[FourParameterMode(w_to=400.0, g_to=30.0,
↪ w_lo=600.0, g_lo=10.0)])
print(_passivity_penalty([passive], 1e3), _passivity_penalty([active], 1e3))
```

Printed `[0.0] [20000.0]` at this build, in 0.0 s.

Changelog of the method. 0.14.0.

N13 · Linear interpolation, held at the ends: resampling, tabulated functions, point-list tables

Numerics · interpolation

Statement. Every resampling in difes is numpy's linear interpolation: a spectrum onto a new grid (refused outside the measured range), error bars with it, a tabulated dielectric function (real and imaginary parts separately, held at the table's end values outside it), k onto n's grid when the two tables differ (zero beyond the last k), the point-list tables of FOCUS's file-2c, and the transform at an anchor (N05, N06). A tabulated model is one vectorised interpolation per evaluation; FOCUS searches and interpolates its point lists inside every formula at every point and every fitter step, which is the mechanism of the minutes-long opening of Mireia's project (log L089).

$$y(x) = y_i + \frac{y_{i+1} - y_i}{x_{i+1} - x_i} (x - x_i) \text{ for } x_i \leq x \leq x_{i+1}; \quad y(x) = y_0 \text{ } (x < x_0), \text{ } y_n \text{ } (x > x_n)$$

Why this choice. Linear because the tables are data, and a spline through measured optical constants invents curvature between points; the ends are held rather than extrapolated so that a model never runs away outside its table. A finer grid is the user's decision, and N04's rule for the transform spacing is the one that bites.

Error and its order. Of order h^2 of the table's spacing; none at the table's points.

Reference. `numpy.interp` documentation

Counterparts. FOCUS (documented): FOCUS's point lists are interpolated linearly or by cubic spline (help, `osml-elements.html`); the search per point is the file's structure (T48). RefFIT (documented): RefFIT's data files are read as given; no tabulated model.

Implementation.

- `interpolate(self, x_new: np.ndarray) -> "Spectrum"` — `src/difes/spectra.py:160`
- `TabulatedModel(w: Any, eps: Any = None, n: Any = None, k: Any = None, name: str = tabulated) -> None` — `src/difes/models.py:561`
- `_table_linear(x, pts)` — `src/difes/expr.py:110`

Library routines called. `numpy.interp` (D07)

Tests that name these symbols. `test_kk.py::test_cubic_kernel_matches_quadrature_on_the_spline`, `test_spectra.py::test_crop_and_interpolate`, `test_tabulated.py::test_tabulated_reproduces_its_source_and`, `test_tabulated.py::test_nk_tables_on_different_grids`, `test_tabulated.py::test_effective_medium_with_tab`, `test_vdf.py::test_hat_functions_partition_unity_and_interpolate`.

Ledger records. T48 (Mireia's alumina project: her Bergman model read from the OSML export, FOCUS's own curve met to 4e-6, her two complaints explained).

Check it yourself. The last printed line must be `[(2.25+0j), (3.12+0.2j), (6.21+1j)]`.

```
import numpy as np
from difes.models import TabulatedModel
t = TabulatedModel([100.0, 200.0, 300.0], n=[1.5, 2.0, 2.5], k=[0.0, 0.1, 0.2])
e = t.epsilon(np.array([50.0, 150.0, 400.0])) # below, inside, above the table
print(np.round(e, 4).tolist())
```

Printed `[(2.25+0j), (3.12+0.2j), (6.21+1j)]` at this build, in 0.0 s.

Changelog of the method. 0.1.0: `Spectrum.interpolate`; 0.30.0: `TabulatedModel`; 0.31.0: `table_linear`.

N14 · Multiple reflections in a plate: closed-form sums or a counted number of rays

Numerics · series-summation

Statement. The reflected and transmitted amplitudes of a plate are geometric series in the round-trip factor $r^2\tau^2$; with all rays detected they are summed in closed form (the Airy amplitude for coherent addition, the sum of squared moduli for incoherent addition), and with a limited number of rays (RefFIT's `RayNo`) the series is summed term by term. The two additions are mixed by RefFIT's degree of coherence. The same holds for the cross terms the ellipsometric angles need.

$$r_{\text{coh}} = \frac{r(1-\tau^2)}{1-r^2\tau^2}, \quad R_{\text{inc}} = |r|^2 + \frac{|r\tau^2(1-r^2)|^2}{1-|r^2\tau^2|^2}, \quad \tau = e^{2\pi i \omega d \sqrt{\varepsilon - \sin^2 \theta}}$$

Why this choice. Closed forms wherever the series converges ($|r^2\tau^2| < 1$ always, for a passive medium) because they are exact and vectorise; the counted sum exists only to reproduce RefFIT's `RayNo` option. The round-trip factor is computed once per polarisation.

Error and its order. None for the closed forms; the truncated sum misses the tail of the series, of order $|r^2\tau^2|^N$.

Reference. RefFIT manual §4.7.9 (code -13); Born and Wolf, Principles of Optics, §7.6

Counterparts. FOCUS (documented): FOCUS's Reflectance-E and Transmittance-E are the incoherent closed forms (T02, T17). RefFIT (inferred from numbers): RefFIT's plate optics reproduced to its printed digits, including `RayNo` (T46).

Implementation.

- `plate_partial_coherence(eps: np.ndarray, omega: np.ndarray, thickness: float, theta_deg: float = 0.0, coherence: float = 0.0, rays: int | None = None, polarizer_deg: float = 0.0, scale: float = 1.0) -> dict[str, np.ndarray]` — `src/difes/optics.py:135`
- `_slab_amplitudes(eps: np.ndarray, omega: np.ndarray, thickness: float) -> tuple[np.ndarray, np.ndarray]` — `src/difes/optics.py:78`

- `_film_amplitudes(eps: np.ndarray, eps_sub: np.ndarray, omega: np.ndarray, thickness: float)` — `src/difes/optics.py:197`

Tests that name these symbols. `test_plate_reffit.py::test_normal_incidence_and_full_coherence_is_the_ai`
`test_plate_reffit.py::test_no_coherence_is_the_intensity_sum_and_meets_the_incoherent_slab_where_absor`
`test_plate_reffit.py::test_energy_is_conserved_and_the_mix_is_linear_in_the_coherence`,
`test_plate_reffit.py::test_truncated_rays_converge_to_the_closed_forms`, `test_plate_reffit.py::test_elli`
`test_plate_reffit.py::test_reffit_plate_wrapper_and_the_polarizer`.

Ledger records. T02 (The optics against textbook values: Fresnel, slabs, films, energy balance); T17 (The phosphate glass: twelve causal Gaussians against FOCUS's export); T46 (ReffIT's own tutorial, Part 2: four datasets fitted together, RefFIT's weighted χ^2 to the printed digit).

Check it yourself. The last printed line must be `True True`.

```
import numpy as np
from difes.optics import plate_partial_coherence
eps = np.array([2.0 + 0.05j]); w = np.array([1000.0])
closed = plate_partial_coherence(eps, w, 0.01) # all rays, closed forms
summed = plate_partial_coherence(eps, w, 0.01, rays=60) # sixty rays, term by term
print(bool(np.allclose(closed["R"], summed["R"], atol=1e-10)),
      ↪ bool(np.allclose(closed["T"], summed["T"], atol=1e-10)))
```

Printed `True True` at this build, in 0.0 s.

Changelog of the method. 0.1.0: slab closed forms; 0.24.0: RefFIT's plate with degree of coherence and RayNo.

N15 · Numerical derivative of a spectrum: second-order central differences

Numerics · differentiation

Statement. Where the derivative of a measured curve with respect to its abscissa is needed (the effect of a wavenumber shift on a spectrum in the uncertainty budget, the derivative of a FOCUS export in the receiving report, the derivative node of a session), it is numpy's gradient: central differences of second order in the interior, one-sided first-order differences at the two ends, on a possibly non-uniform grid.

$$y'_i \approx \frac{h_{i-1}^2 y_{i+1} - h_i^2 y_{i-1} + (h_i^2 - h_{i-1}^2) y_i}{h_{i-1} h_i (h_{i-1} + h_i)}, \quad y'_0 \approx \frac{y_1 - y_0}{h_0}$$

Why this choice. The simplest rule that is exact for a parabola in the interior; no smoothing, because the derivative is used as a sensitivity (per cm^{-1} of shift) and the noise it carries is part of what the budget propagates. A user who needs a smooth derivative of noisy data should fit first.

Error and its order. Of order h^2 in the interior, of order h at the two end points; the check shows the end effect on a parabola.

Reference. `numpy.gradient` documentation; Fornberg, Math. Comput. 51 (1988) 699

Counterparts. FOCUS (documented): FOCUS has a derivative in its formula language (`help, core-functions.htm`); its rule is not stated. RefFIT (documented): Not applicable.

Implementation.

- `effect_on(self, term: FitTerm) -> np.ndarray` — `src/difes/uncertainty.py:117`

Library routines called. `numpy.gradient` (D07)

Ledger records. T35 (Uncertainties following the GUM: the stated intervals cover the truth).

Check it yourself. The last printed line must be `True 210.0 200.0`.

```

import numpy as np
from difes.spectra import Spectrum, Kind
from difes.models import example_model
from difes.fit import FitTerm
from difes.uncertainty import Component
x = np.linspace(100, 200, 11); spec = Spectrum(x, x ** 2, kind=Kind.RAW)      # a
↳ parabola, spacing 10
g = Component("shift", "xshift", 1.0).effect_on(FitTerm(spec, example_model())) # dy/dx
print(bool(np.allclose(g[1:-1], 2 * x[1:-1])), float(g[0]), float(2 * x[0])) # exact
↳ inside, 210 vs 200 at the end

```

Printed True 210.0 200.0 at this build, in 0.0 s.

Changelog of the method. 0.20.0.

N16 · Random numbers: a seeded PCG64 generator for the Monte Carlo, a legacy global one in the formula language

Numerics · random-numbers

Statement. The Monte Carlo of M02 and the transform Monte Carlo of M07 draw from numpy’s default `_rng` (PCG64) created from the seed the caller gives, so that a run is reproducible bit for bit at the same numpy version and the seed is written into the report. The formula language’s `rnd` (FOCUS’s random token) draws from numpy’s legacy global generator, unseeded: a formula containing `rnd` is not reproducible between runs. The registry records this as a known gap rather than a design.

$$u \sim \mathcal{N}(0, u_k) \text{ or } \mathcal{U}(-a, a), \quad a = u_k \sqrt{3}; \quad \text{PCG64 with seed } s$$

Why this choice. One generator per Monte Carlo run rather than a module-level one, so that two runs in the same session do not share a stream; the Gaussian draws are numpy’s `ziggurat`. The formula `rnd` keeps FOCUS’s behaviour of a fresh draw per evaluation, which is what a user who types it expects, at the cost of reproducibility.

Error and its order. The Monte Carlo’s sampling error is M02’s; the generator itself is not a source of error at these sizes.

Reference. O’Neill, PCG: a family of simple fast space-efficient statistically good algorithms for random number generation (2014); `numpy.random` documentation

Counterparts. FOCUS (documented): FOCUS’s formula language has a random token (`help`, `core-functions.htm`); its generator is not stated. RefFIT (documented): None.

Implementation.

- `run_monte_carlo(result: FitResult, budget: Budget, n: int, *, seed: int | None = None, derived: dict[str, Callable[[list[DielectricModel]], Any]] | None = None, level: float = 0.95, max_nfev: int | None = 200) -> MonteCarloResult` — `src/difes/uncertainty.py:4`
- `monte_carlo_transform(spec: Spectrum, budget: Budget, f: Callable[[Spectrum], Any], n: int = 200, *, seed: int | None = None, level: float = 0.95) -> TransformUncertainty` — `src/difes/uncertainty.py:527`
- `evaluate(node, env: dict[str, Any], functions: "dict[str, UserFunction] | None" = None)` — `src/difes/expr.py:405`

Library routines called. `Generator.normal / uniform` (D02); `numpy.quantile` (D07); `numpy.random.default_rng` (D02); `numpy.random.random` (legacy global generator) (D02)

Tests that name these symbols. `test_bergman.py::test_mireia_osml_read_as_document_with_its_tables`, `test_expr.py::test_focus_formula_editor_examples`, `test_expr.py::test_errors`, `test_fit.py::test_analytic`, `test_osml.py::test_focus_projects_of_irati_read_and_evaluate`, `test_scale.py::test_scale_factor_is_recov`

Ledger records. T35 (Uncertainties following the GUM: the stated intervals cover the truth); T36 (The budget through the Kramers–Kronig inversion, and in a session).

Check it yourself. The last printed line must be `True True`.

```
import numpy as np
from difes.expr import evaluate_formula
from difes.spectra import Spectrum, Kind
from difes.uncertainty import Budget, Component, monte_carlo_transform
spec = Spectrum(np.linspace(100, 200, 5), np.full(5, 0.5), kind=Kind.R, yerr=np.full(5,
    ↪ 0.01))
b = Budget([Component("scale", "scale", 0.01)])
a1 = monte_carlo_transform(spec, b, lambda s: s.y, 20, seed=3)
a2 = monte_carlo_transform(spec, b, lambda s: s.y, 20, seed=3)          # the same seed:
    ↪ the same draws
r1, r2 = evaluate_formula("rnd", x=0.0), evaluate_formula("rnd", x=0.0)  # the formula
    ↪ token: unseeded
print(bool(np.array_equal(a1.std, a2.std)), r1 != r2)
```

Printed `True True` at this build, in 0.0 s.

Changelog of the method. 0.5.0: `rnd` in the language; 0.20.0: seeded generator for the Monte Carlo.

N17 · χ^2 , weights, σ and degrees of freedom: the conventions behind every reported number

Numerics · conventions

Statement. χ^2 is the sum over the data rows of the squared scaled residuals, with the penalty rows (N12) excluded; the reduced χ^2 divides by the number of points minus the number of free parameters, never less than one. The per-point σ is the spectrum’s error bars (non-positive entries replaced by the smallest positive one), or a constant given on the term, or 1 when neither exists; RefFIT’s files without an error column enter with 0.01 (RefFIT’s convention) through the reader, not here. A term’s weight multiplies its contribution to χ^2 (it enters the residual as its square root). The rms residual per term is reported separately, unweighted. For emissivity terms with RefFIT’s scaling factor the factor multiplies the reflectance behind the emissivity, $1 - s(1 - R)$.

$$\chi^2 = \sum_t w_t \sum_i \frac{[f_t(\omega_i) - y_i]^2}{\sigma_i^2}, \quad \chi_{\text{red}}^2 = \frac{\chi^2}{\max(n - m, 1)}$$

Why this choice. The conventions of a weighted least-squares fit as the GUM and RefFIT use them, so that χ^2 values are comparable between the programs: RefFIT’s weighted χ^2 of four datasets was reproduced to the printed digit once its grid was identified (T46). Degrees of freedom are reduced by every free parameter, including the experimental ones (a thickness, a scale).

Error and its order. None; these are definitions. Their consequence is in N02: with $\sigma = 1$ the parameter errors are meaningful only through the χ^2 scaling.

Reference. RefFIT manual §4.3 (the weighted χ^2); JCGM 100:2008 (the measurement model)

Counterparts. FOCUS (unknown): FOCUS reports no χ^2 ; its Squeeze option compresses the data before fitting (help, fit-panel.html), which difes does not emulate. RefFIT (inferred from numbers): RefFIT’s printed χ^2 met to the digit on its tutorial (T46).

Implementation.

- `FitTerm(spectrum: Spectrum, model: DielectricModel, weight: float = 1.0, thickness: float | Parameter | None = None, coherent: bool = True, sigma: float | None = None, substrate: DielectricModel | None = None, form: str = energy, layers: list[tuple[DielectricModel, float]] | None = None, scale: Parameter | None = None) -> None` — src/difes/fit.py:30
- `sigmas(self) -> np.ndarray` — src/difes/fit.py:70

- `evaluate(self, model: DielectricModel | None = None) -> np.ndarray` — `src/difes/fit.py:81`

Tests that name these symbols. `test_bergman.py::test_mireia_osml_read_as_document_with_its_tables`, `test_expr.py::test_focus_formula_editor_examples`, `test_expr.py::test_errors`, `test_expr.py::test_expressions`, `test_fit.py::test_r1_recovery_from_wrong_guess`, `test_fit.py::test_inplace_updates_callers_model`, `test_fit.py::test_frozen_parameter_stays_and_has_no_error`, `test_fit.py::test_bounds_respected`, `test_fit.py::test_double_drude_with_one_damping_frozen`, `test_fit.py::test_two_spectra_share_a_linked_parameter` and 11 more.

Ledger records. T42 (RefFIT’s own tutorial, Part 1: RefFIT’s data and saved models, and difes’s fitter against RefFIT’s); T46 (RefFIT’s own tutorial, Part 2: four datasets fitted together, RefFIT’s weighted χ^2 to the printed digit).

Check it yourself. The last printed line must be 299 299 analytic.

```
import numpy as np
from difes.models import example_model
from difes.optics import simulate
from difes.fit import FitTerm, fit
m = example_model(); w = np.linspace(100, 3000, 300)
spec = simulate(m, w, "R"); spec = spec.copy(y=spec.y + 0.01) # a constant offset, no
    ↪ error bars: sigma = 1
m2 = example_model()
for _, p in m2.parameters(): p.frozen = True
m2.eps_inf.frozen = False # one free parameter
r = fit([FitTerm(spec, m2, weight=4.0)])
print(r.n_points - r.n_free, round(r.chi2 / r.chi2_reduced), r.jacobian)
```

Printed 299 299 analytic at this build, in 0.0 s.

Changelog of the method. 0.1.0; 0.24.0: RefFIT’s scaling factor; 0.27.0: RefFIT’s 0.01 convention in the reader.

Metrology

the GUM budget: input components, the law of propagation, Monte Carlo, coverage, what none of it can say.

M01 · Law of propagation of uncertainty at the least-squares optimum (GUM §5, Supplement 2)

Metrology · gum-propagation

Statement. At the optimum the parameters respond linearly to the data, $\mathbf{p} = \mathbf{A} \mathbf{y}$ with $\mathbf{A} = (\mathbf{J}^T \mathbf{W} \mathbf{J})^{-1} \mathbf{J}^T \mathbf{W}$, $\mathbf{J}$ the numerical Jacobian of N03 and $\mathbf{W}$ the fit’s weights over σ^2 . The input covariance is the diagonal of σ^2 (plus any extra independent point components) and, for each Type B component k , u_k^2 times the outer product of its effect vector $\mathbf{g}_k$ (the spectrum itself for a scale, ones for an offset, the derivative of N15 for a wavenumber shift, a given vector for a custom effect). The parameter covariance is $\mathbf{A} \mathbf{V}_y \mathbf{A}^T$; the Type A part is the first term, each Type B contribution to parameter j is $u_k^2 |\mathbf{A} \mathbf{g}_k|_j$, and they add in quadrature to the combined standard uncertainty. The correlation matrix is printed, and every correlation above 0.7 is named in the report.

$$\mathbf{V}_p = \mathbf{A} \mathbf{V}_y \mathbf{A}^T, \quad \mathbf{A} = (\mathbf{J}^T \mathbf{W} \mathbf{J})^{-1} \mathbf{J}^T \mathbf{W}, \quad \mathbf{V}_y = \text{diag}(\sigma_i^2) + \sum_k u_k^2 \mathbf{g}_k \mathbf{g}_k^T$$

Why this choice. The GUM’s own matrix form rather than the fitter’s covariance, so that fully correlated Type B components can be carried; the Type A part equals the fitter’s unscaled covariance exactly when the budget is empty (the check of N02). A pseudo-inverse (D04) so that a degenerate problem gives a finite, large answer and a note rather than a failure.

Error and its order. Linear; M02 is the check where the problem is not. The law says nothing about a wrong model (M05).

Reference. JCGM 100:2008 §5 and Annex G; JCGM 102:2011 (Supplement 2, multivariate quantities)

Counterparts. FOCUS (documented): FOCUS prints no uncertainties. RefFIT (documented): RefFIT prints confidence limits only, with no Type B components.

Implementation.

- `evaluate_uncertainty(result: FitResult, budget: Budget | None = None, *, k: float | str = 2.0, level: float = 0.95, derived: dict[str, Callable[[list[DielectricModel], Any]] | None = None, monte_carlo: int = 0, seed: int | None = None) -> UncertaintyResult` — `src/difes/uncertainty.py:340`
- `UncertaintyResult(names: list[str], values: np.ndarray, contributions: list[Contribution], covariance: np.ndarray, chi2_reduced: float, n_points: int, n_free: int, k: float, k_source: str, dof_eff: np.ndarray, scaled_typeA: np.ndarray, derived: "dict[str, DerivedUncertainty]" = <factory>, monte_carlo: "MonteCarloResult | None" = None, notes: list[str] = <factory>) -> None` — `src/difes/uncertainty.py:199`
- `Contribution(name: str, type: str, u: np.ndarray, dof: float) -> None` — `src/difes/uncertainty.py:`

Library routines called. `numpy.linalg.pinv` (D04); `numpy.quantile` (D07)

Tests that name these symbols. `test_scale.py::test_scale_in_the_budget_and_in_a_session`, `test_uncertainty.py::test_type_a_equals_the_fitters_unscaled_errors`, `test_uncertainty.py::test_scale_coverage`, `test_uncertainty.py::test_monte_carlo_agrees_with_the_law_of_propagation`, `test_uncertainty.py::test_derived_chi2_reduced`, `test_uncertainty.py::test_t_factor_and_poor_fit_note`, `test_uncertainty.py::test_coverage_of_k2_interval`

Ledger records. T35 (Uncertainties following the GUM: the stated intervals cover the truth).

Check it yourself. The last printed line must be `scale True True`.

```
import numpy as np
from difes.models import DielectricModel, Lorentz
from difes.optics import simulate
from difes.fit import FitTerm, fit
from difes.uncertainty import evaluate_uncertainty, Budget, Component
rng = np.random.default_rng(1); w = np.linspace(100, 2000, 191)
true = DielectricModel(eps_inf=3.0, terms=[Lorentz(w0=450.0, wp=600.0, gamma=20.0)])
spec = simulate(true, w, "R")
spec = spec.copy(y=spec.y + rng.normal(0, 0.003, w.size), yerr=np.full(w.size, 0.003))
r = fit([FitTerm(spec, DielectricModel(eps_inf=2.8, terms=[Lorentz(w0=460.0, wp=580.0,
↳ gamma=25.0)))]))
u = evaluate_uncertainty(r, Budget([Component("scale", "scale", 0.01)]))      # a 1 %
↳ scale error
rel = {n.split(".")[1]: float(c / abs(v)) for n, c, v in zip(u.names,
↳ u.contributions[1].u, u.values)}
# a scale error moves the plasma frequency (and eps_inf) and leaves the resonance alone
print(u.contributions[1].name, rel["w0"] < 1e-3, rel["wp"] > 1e-3)
```

Printed `scale True True` at this build, in 0.0 s.

Changelog of the method. 0.20.0: the budget and the law of propagation; 0.21.0: Welch–Satterthwaite and the t-factor (M04).

M02 · Monte Carlo propagation of distributions (GUM Supplement 1)

Metrology · monte-carlo

Statement. n trials; in each, every spectrum is perturbed by one draw of every component (the point scatter as a normal deviate per point from the error bars, extra point components likewise, a scale as $y(1 + d)$, an offset as $y + d$, a shift or a custom effect as $y + d g_k$, with d from the component's declared distribution),

a perturbed R, E or T is clipped to $[0, 1]$, and the fit (N01) is rerun from the optimum with at most 200 evaluations; the parameters and the derived quantities are collected. The coverage interval is read from the quantiles at $(1 - \text{level})/2$ and $1 - (1 - \text{level})/2$, so it can be asymmetric; failed trials are counted and reported. The generator is N16's.

$$\mathbf{p}^{(t)} = \arg \min \chi^2(\mathbf{y} + \delta \mathbf{y}^{(t)}), \quad [Q_{\alpha/2}, Q_{1-\alpha/2}] \text{ of } \{\mathbf{p}^{(t)}\}$$

Why this choice. Refit from the optimum with a capped evaluation count rather than from scratch, because the perturbations are small and the optimum is the natural start; trials that leave the basin are the failures counted. Clipping bounded kinds to $[0, 1]$ keeps every draw physical. Supplement 1's adaptive stopping rule is not implemented: n is the caller's, and the card says so.

Error and its order. The Monte Carlo standard error of a quantile, of order one over the root of n; with 500 trials the 95 % limits are known to about 1 % of the interval's width.

Reference. JCGM 101:2008 §7

Counterparts. FOCUS (documented): None. RefFIT (documented): None.

Implementation.

- `run_monte_carlo(result: FitResult, budget: Budget, n: int, *, seed: int | None = None, derived: dict[str, Callable[[list[DielectricModel], Any]] | None = None, level: float = 0.95, max_nfev: int | None = 200) -> MonteCarloResult` — `src/difes/uncertainty.py:4`
- `MonteCarloResult(samples: np.ndarray, derived: dict[str, np.ndarray], failures: int, level: float) -> None` — `src/difes/uncertainty.py:308`

Library routines called. `Generator.normal / uniform (D02)`; `numpy.quantile (D07)`; `numpy.random.default_rng (D02)`

Ledger records. T35 (Uncertainties following the GUM: the stated intervals cover the truth).

Check it yourself. The last printed line must be `True True`.

```
import numpy as np
from difes.models import DielectricModel, Lorentz
from difes.optics import simulate
from difes.fit import FitTerm, fit
from difes.uncertainty import evaluate_uncertainty
rng = np.random.default_rng(1); w = np.linspace(100, 2000, 191)
true = DielectricModel(eps_inf=3.0, terms=[Lorentz(w0=450.0, wp=600.0, gamma=20.0)])
spec = simulate(true, w, "R")
spec = spec.copy(y=spec.y + rng.normal(0, 0.003, w.size), yerr=np.full(w.size, 0.003))
r = fit([FitTerm(spec, DielectricModel(eps_inf=2.8, terms=[Lorentz(w0=460.0, wp=580.0,
↳ gamma=25.0)))]))
u = evaluate_uncertainty(r, monte_carlo=60, seed=5)
v = evaluate_uncertainty(r, monte_carlo=60, seed=5)
ratio = u.monte_carlo.std() / u.u_combined          # the two routes agree on a linear
↳ problem
print(bool(np.array_equal(u.monte_carlo.samples, v.monte_carlo.samples)),
↳ bool(np.all((ratio > 0.7) & (ratio < 1.3))))
```

Printed True True at this build, in 0.24 s.

Changelog of the method. 0.20.0.

M03 · The input budget: Type A and Type B components, their distributions and effect vectors

Metrology · input-quantities

Statement. A Budget lists components. Kinds: scale (multiplicative on y, u relative), offset (additive, u absolute), xshift (a shift of the wavenumber scale, u in cm^{-1} , through dy/d), point (an extra independent per-point uncertainty, Type A by default) and custom (an arbitrary effect vector per unit of the component). Each carries a standard uncertainty, a distribution (normal, or rectangular from limits $\pm a$ with $u = a/\sqrt{3}$, GUM §4.3.7), degrees of freedom (infinite for a well-known Type B component) and the names of the spectra it acts on. Constructors exist for a rectangular component and for a digitised figure (the reading precision times the axis span as a per-point Type B component). A custom component without its vector, a negative u or an unknown kind are refused.

$$u = \frac{a}{\sqrt{3}} \text{ (rectangular, limits } \pm a\text{); } \quad \mathbf{g}_{\text{scale}} = \mathbf{y}, \mathbf{g}_{\text{offset}} = \mathbf{1}, \mathbf{g}_{\text{xshift}} = \frac{dy}{d\omega}$$

Why this choice. The four kinds are the effects an infrared reflectance or emissivity measurement actually has (the reference mirror or blackbody, stray light, the wavenumber calibration, the detector's scatter), with custom for the rest, rather than a general covariance matrix a user would never fill. The values are the laboratory's to state; the manual's examples use placeholders and say so.

Error and its order. None; the budget is input. What it cannot represent: a partially correlated effect between points (neither fully correlated nor independent), which would need a custom covariance.

Reference. JCGM 100:2008 §4.2 (Type A), §4.3 (Type B, the rectangular distribution), §5.2 (correlated inputs)

Counterparts. FOCUS (documented): None. RefFIT (documented): RefFIT's error column is the Type A part only.

Implementation.

- `Component(name: str, kind: str, u: float, type: str = B, distribution: str = normal, dof: float = inf, applies: Sequence[str] | None = None, effect: np.ndarray | None = None) -> None` — `src/difes/uncertainty.py:53`
- `rectangular(name: str, kind: str, half_width: float, **kw: Any) -> "Component"` — `src/difes/uncertainty.py:85`
- `digitised(precision: float, span: float, name: str = digitisation, **kw: Any) -> "Component"` — `src/difes/uncertainty.py:90`
- `Budget(components: list[Component] = <factory>) -> None` — `src/difes/uncertainty.py:142`

Library routines called. `Generator.normal` / `uniform` (D02); `numpy.gradient` (D07)

Tests that name these symbols. `test_scale.py::test_scale_in_the_budget_and_in_a_session`, `test_uncertainty.py::test_component_kinds_and_rectangular`, `test_uncertainty.py::test_scale_component_dof`, `test_uncertainty.py::test_monte_carlo_agrees_with_the_law_of_propagation`, `test_uncertainty.py::test_derivative`, `test_uncertainty.py::test_coverage_of_k2_intervals_on_synthetic_data`, `test_uncertainty.py::test_budget`, `test_uncertainty.py::test_kk_inversion_bands_cover_the_truth_and_grow_with_the_budget`, `test_uncertainty.py::test_cfrc_and_vdf_routes_and_transform_node`.

Ledger records. T35 (Uncertainties following the GUM: the stated intervals cover the truth); T37 (Effective-medium layers as native geometry: Irati's final model from native pieces).

Check it yourself. The last printed line must be `0.02 rectangular True`.

```
import math
from difes.uncertainty import Component
c = Component.rectangular("calibration", "scale", 0.02)           # limits +-2 %
try:
    Component("stray", "custom", 0.1); refused = False          # a custom component
    ↪ needs its vector
except ValueError:
    refused = True
print(round(c.u * math.sqrt(3), 6), c.distribution, refused)
```

Printed 0.02 rectangular True at this build, in 0.0 s.

Changelog of the method. 0.20.0; 0.23.0: the digitised constructor.

M04 · Coverage factor: $k = 2$ by convention, or from the t-distribution at Welch–Satterthwaite’s effective degrees of freedom

Metrology · coverage

Statement. The expanded uncertainty is $U = k u_c$. By default $k = 2$; with $k = “t”$ the factor is the two-sided quantile of Student’s t at the requested level and the effective degrees of freedom, computed per parameter by the Welch–Satterthwaite formula from the contributions ($n - m$ for the Type A part, the stated value for each Type B one, infinite contributions dropped) and averaged over the parameters for a single k . A normal quantile is used when the degrees of freedom are infinite or above 10^6 .

$$\nu_{\text{eff}} = \frac{u_c^4}{\sum_i u_i^4 / \nu_i}, \quad k = t_{(1+p)/2}(\nu_{\text{eff}})$$

Why this choice. One k for the whole report (the mean of the per-parameter factors) because a report with a different coverage factor on every line is not readable; the per-parameter degrees of freedom are printed so that a reader can recompute. The conventional $k = 2$ is the default because the group’s Type B components will be stated with infinite degrees of freedom.

Error and its order. Welch–Satterthwaite is itself an approximation to the distribution of a sum of chi-squares; GUM Annex G accepts it.

Reference. JCGM 100:2008 Annex G, G.4 and G.6; Welch, *Biometrika* 34 (1947) 28; Satterthwaite, *Biometrics Bull.* 2 (1946) 110

Counterparts. FOCUS (documented): None. RefFIT (documented): RefFIT’s confidence limits use a factor its manual does not derive.

Implementation.

- `_k_for(level: float, dof: float) -> float` — `src/difes/uncertainty.py:334`

Library routines called. `scipy.stats t.ppf / norm.ppf` (D05)

Ledger records. T35 (Uncertainties following the GUM: the stated intervals cover the truth).

Check it yourself. The last printed line must be 1.96 2.571.

```
from difes.uncertainty import _k_for
print(round(_k_for(0.95, float("inf")), 3), round(_k_for(0.95, 5.0), 3)) # the normal
↪ 1.96 and t with 5 degrees of freedom
```

Printed 1.96 2.571 at this build, in 0.0 s.

Changelog of the method. 0.21.0.

M05 · What the budget cannot say: the reduced χ^2 note, the inflated Type A, parameters at a bound

Metrology · model-adequacy

Statement. The GUM has no mechanism for an inadequate model. When the reduced χ^2 is above 1.5 the report says in words that either the input uncertainties are understated or the model does not describe the data, that the budget cannot tell which, and that no interval it computes covers a wrong model; it prints the Type A part both as it is and inflated by the square root of the reduced χ^2 (the usual practice, which assumes the first explanation). A parameter within 10^{-9} (relative) of a bound is named: its row is the sensitivity of the limit, not an uncertainty, and the Monte Carlo spread is clipped there.

$$u_A^{\text{scaled}} = u_A \sqrt{\chi_{\text{red}}^2} \text{ if } \chi_{\text{red}}^2 > 1$$

Why this choice. Say it rather than hide it: the alternative, silently scaling the errors by the reduced χ^2 as `curve_fit` does, turns a wrong model into a confident number. The chromium of the validation pages (reduced χ^2 above 200) is the example that made this a rule.

Error and its order. The inflated Type A is right only if the error bars are uniformly understated; the note is the honest statement, and the judgement is the user's.

Reference. JCGM 100:2008 §3.4.8 and Annex E; Press et al., Numerical Recipes §15.1 on the chi-square test

Counterparts. FOCUS (documented): None. RefFIT (documented): RefFIT's manual calls the absolute size of its confidence limits human-dependent.

Implementation.

- `evaluate_uncertainty(result: FitResult, budget: Budget | None = None, *, k: float | str = 2.0, level: float = 0.95, derived: dict[str, Callable[[list[DielectricModel]], Any]] | None = None, monte_carlo: int = 0, seed: int | None = None) -> UncertaintyResult` — `src/difes/uncertainty.py:340`
- `u_combined_scaled(self) -> np.ndarray` — `src/difes/uncertainty.py:220`

Library routines called. `numpy.linalg.pinv` (D04); `numpy.quantile` (D07)

Tests that name these symbols. `test_scale.py::test_scale_in_the_budget_and_in_a_session`, `test_uncertainty.py::test_type_a_equals_the_fitters_unscaled_errors`, `test_uncertainty.py::test_scale_coverage`, `test_uncertainty.py::test_monte_carlo_agrees_with_the_law_of_propagation`, `test_uncertainty.py::test_derived`, `test_uncertainty.py::test_t_factor_and_poor_fit_note`, `test_uncertainty.py::test_coverage_of_k2_interval`

Ledger records. T35 (Uncertainties following the GUM: the stated intervals cover the truth); T37 (Effective-medium layers as native geometry: Irati's final model from native pieces).

Check it yourself. The last printed line must be `True True True`.

```
import numpy as np
from difes.models import DielectricModel, Lorentz
from difes.optics import simulate
from difes.fit import FitTerm, fit
from difes.uncertainty import evaluate_uncertainty
rng = np.random.default_rng(1); w = np.linspace(100, 2000, 191)
true = DielectricModel(eps_inf=3.0, terms=[Lorentz(w0=450.0, wp=600.0, gamma=20.0)])
spec = simulate(true, w, "R")
spec = spec.copy(y=spec.y + rng.normal(0, 0.003, w.size), yerr=np.full(w.size, 0.001))
    ↪ # error bars three times too small
r = fit([FitTerm(spec, DielectricModel(eps_inf=2.8, terms=[Lorentz(w0=460.0, wp=580.0,
    ↪ gamma=25.0)))]))
u = evaluate_uncertainty(r)
print(u.chi2_reduced > 1.5, any("reduced chi-square" in n for n in u.notes),
      bool(np.allclose(u.scaled_typeA, u.contributions[0].u * np.sqrt(u.chi2_reduced))))
```

Printed True True True at this build, in 0.0 s.

Changelog of the method. 0.20.0.

M06 · Derived quantities and curve bands: the gradient of $q(p)$ through the parameter covariance

Metrology · derived-quantities

Statement. Any function of the fitted models (a static dielectric constant, a dc conductivity, a Lyddane–Sachs–Teller ratio, or a whole curve on a grid) is propagated by the law of propagation with its gradient by central differences (N03's rule) through the parameter covariance of M01, which for a curve gives an uncertainty band point by point; and, when trials are set, by the Monte Carlo of M02, which collects the

function on every trial and reads its quantiles. The fit node of a session exposes the fitted quantity, ε' and ε'' this way, each with its `ku_c` band and its Monte Carlo interval.

$$u^2(q) = \nabla q^T V_{\mathbf{p}} \nabla q, \quad u^2(q_i) = \sum_{jk} \frac{\partial q_i}{\partial p_j} (V_{\mathbf{p}})_{jk} \frac{\partial q_i}{\partial p_k}$$

Why this choice. The gradient is numerical so that any Python function of the models can be propagated without writing its derivatives; the cost is one model evaluation per parameter and direction. A curve's band is the diagonal of its covariance only; the correlation between points of the band is available from the Monte Carlo samples.

Error and its order. Linear in the parameters' uncertainties; M02's interval is the one to quote where the function is strongly non-linear (a ratio of small numbers).

Reference. JCGM 100:2008 §5.1.2; JCGM 102:2011 §6

Counterparts. FOCUS (documented): None. RefFIT (documented): None.

Implementation.

- `DerivedUncertainty(name: str, value: np.ndarray, u: np.ndarray, k: float, mc_low: np.ndarray | None = None, mc_high: np.ndarray | None = None) -> None` — `src/difes/uncertainty.py:28`

Ledger records. T35 (Uncertainties following the GUM: the stated intervals cover the truth).

Check it yourself. The last printed line must be `1 191 True True`.

```
import numpy as np
from difes.models import DielectricModel, Lorentz
from difes.optics import simulate
from difes.fit import FitTerm, fit
from difes.uncertainty import evaluate_uncertainty
rng = np.random.default_rng(1); w = np.linspace(100, 2000, 191)
true = DielectricModel(eps_inf=3.0, terms=[Lorentz(w0=450.0, wp=600.0, gamma=20.0)])
spec = simulate(true, w, "R")
spec = spec.copy(y=spec.y + rng.normal(0, 0.003, w.size), yerr=np.full(w.size, 0.003))
r = fit([FitTerm(spec, DielectricModel(eps_inf=2.8, terms=[Lorentz(w0=460.0, wp=580.0,
↪ gamma=25.0)))]))
u = evaluate_uncertainty(r, derived={"eps0": lambda ms:
↪ ms[0].epsilon(np.array([0.0])).real[0],
                                "R": lambda ms: simulate(ms[0], w, "R").y})
d = u.derived
print(d["eps0"].value.size, d["R"].value.size, bool(d["eps0"].u[0] > 0),
↪ bool(np.all(d["R"].u >= 0)))
```

Printed 1 191 True True at this build, in 0.0 s.

Changelog of the method. 0.20.0.

M07 · Uncertainty through an inversion: the budget propagated by Monte Carlo alone

Metrology · monte-carlo

Statement. A Kramers–Kronig inversion, a CFRC or a variational fit has no Jacobian worth writing, so the budget goes through it by Monte Carlo: the spectrum is perturbed by one draw of every component (the same step as M02), any function of a spectrum is evaluated on each draw, and the standard deviation and the coverage interval of the output come out on its grid. `kk_eps_uncertainty` does it for the finite-range phase transform, with or without the CFRC (two-pole phase model, anchors, positivity), and gives bands on θ , ε' and ε'' ; `vdf_uncertainty` repeats the two-step variational recipe on every draw and gives bands on the mesh.

$$u^2(f_i) = \frac{1}{n-1} \sum_t [f_i(\mathbf{y} + \delta \mathbf{y}^{(t)}) - \bar{f}_i]^2, \quad [Q_{\alpha/2}, Q_{1-\alpha/2}]$$

Why this choice. Monte Carlo alone, no linearisation, because the transform’s response to a scale error is not local (a scale error on R changes the phase everywhere) and the CFRC’s response to a perturbed phase is the point of the exercise. The bands say how much of the inversion the measurement itself determines; the truncation bias of the plain transform is not in them, which is the reason the corrections exist. At an anchor the band narrows to what the anchor allows (the suite checks it halves there).

Error and its order. Sampling error of order one over the root of n; 200 trials by default for a transform, 50 for the variational fit.

Reference. JCGM 101:2008

Counterparts. FOCUS (documented): None. RefFIT (documented): None.

Implementation.

- `perturb_spectrum(spec: Spectrum, budget: Budget, rng: np.random.Generator) -> Spectrum`
— `src/difes/uncertainty.py:499`
- `monte_carlo_transform(spec: Spectrum, budget: Budget, f: Callable[[Spectrum], Any],
n: int = 200, *, seed: int | None = None, level: float = 0.95) -> TransformUncertainty`
— `src/difes/uncertainty.py:527`
- `TransformUncertainty(value: np.ndarray, std: np.ndarray, low: np.ndarray, high:
np.ndarray, level: float, trials: int, failures: int) -> None` — `src/difes/uncertainty.py:485`
- `kk_eps_uncertainty(spec: Spectrum, budget: Budget, n: int = 200, *, seed: int | None
= None, level: float = 0.95, form: str = derivative, anchors: Sequence[tuple[float,
float]] = (), positive: bool = False, poles: Sequence[float] | None = None) ->
dict[str, TransformUncertainty]` — `src/difes/uncertainty.py:554`
- `vdf_uncertainty(spec: Spectrum, budget: Budget, base: DielectricModel | None = None,
n: int = 50, *, seed: int | None = None, level: float = 0.95, **vdf_kwargs: Any) ->
dict[str, TransformUncertainty]` — `src/difes/uncertainty.py:582`

Library routines called. `Generator.normal / uniform` (D02); `numpy.gradient` (D07); `numpy.quantile` (D07); `numpy.random.default_rng` (D02)

Tests that name these symbols. `test_uncertainty.py::test_kk_inversion_bands_cover_the_truth_and_grow_wi`
`test_uncertainty.py::test_cfrc_and_vdf_routes_and_transform_node`.

Ledger records. T36 (The budget through the Kramers–Kronig inversion, and in a session); T38 (The budget through the CFRC and the variational function).

Check it yourself. The last printed line must be `['eps1', 'eps2', 'theta'] True`.

```
import numpy as np
from difes.models import DielectricModel, Lorentz
from difes.optics import simulate
from difes.uncertainty import Budget, Component, kk_eps_uncertainty
m = DielectricModel(eps_inf=3.0, terms=[Lorentz(w0=450.0, wp=600.0, gamma=20.0)])
spec = simulate(m, np.linspace(100, 2000, 381), "R")
small = kk_eps_uncertainty(spec, Budget([Component("scale", "scale", 0.003)]), n=30,
    ↪ seed=1)
large = kk_eps_uncertainty(spec, Budget([Component("scale", "scale", 0.01)]), n=30,
    ↪ seed=1)
print(sorted(small), bool(np.mean(large["eps2"].std) > np.mean(small["eps2"].std))) #
    ↪ the band grows with the budget
```

Printed `['eps1', 'eps2', 'theta'] True` at this build, in 2.03 s.

Changelog of the method. 0.22.0.

M08 · Checking the statement: the stated intervals must cover the truth

Metrology · verification

Statement. A stated interval has to be checked, and the check is a test in the suite: many synthetic reflectivities are generated from a known model with a scatter and a scale error drawn anew each time, each is fitted, and its $k = 2$ interval is computed from a budget that states those two components; the intervals must contain the truth about 95 % of the time for every parameter. The suite also checks that the law of propagation reproduces the fitter's unscaled errors on an empty budget (N02), that the Monte Carlo spread agrees with the linear one (M02), and that a scale component moves ε_∞ and ω_p together while leaving ω_0 alone (M01).

$$\frac{1}{N} \sum_{\text{trials}} \mathbf{1}[|p_j^{\text{fit}} - p_j^{\text{true}}| \leq k u_c(p_j)] \approx 0.95$$

Why this choice. Coverage is the only definition of an uncertainty that can be tested, and the GUM's own; the test is statistical, so the suite's tolerance is loose (at least 88 % of 120 trials in T35) and the registry's check below is a smaller version of it.

Error and its order. The coverage estimate's own binomial scatter: with 30 trials a true 95 % reads between 83 % and 100 %.

Reference. JCGM 100:2008 §6.2 (coverage probability); JCGM 101:2008 §8 (validation of the GUM uncertainty framework)

Counterparts. FOCUS (documented): None. RefFIT (documented): None.

Implementation.

- `evaluate_uncertainty(result: FitResult, budget: Budget | None = None, *, k: float | str = 2.0, level: float = 0.95, derived: dict[str, Callable[[list[DielectricModel]], Any]] | None = None, monte_carlo: int = 0, seed: int | None = None) -> UncertaintyResult` — `src/difes/uncertainty.py:340`
- `expanded(self) -> np.ndarray` — `src/difes/uncertainty.py:226`

Library routines called. `numpy.linalg.pinv` (D04); `numpy.quantile` (D07)

Tests that name these symbols. `test_scale.py::test_scale_in_the_budget_and_in_a_session`, `test_uncertainty.py::test_type_a_equals_the_fitters_unscaled_errors`, `test_uncertainty.py::test_scale_coverage`, `test_uncertainty.py::test_monte_carlo_agrees_with_the_law_of_propagation`, `test_uncertainty.py::test_derived_errors`, `test_uncertainty.py::test_t_factor_and_poor_fit_note`, `test_uncertainty.py::test_coverage_of_k2_interval`

Ledger records. T35 (Uncertainties following the GUM: the stated intervals cover the truth).

Check it yourself. The last printed line must be True.

```
import numpy as np
from difes.models import DielectricModel, Lorentz
from difes.optics import simulate
from difes.fit import FitTerm, fit
from difes.uncertainty import evaluate_uncertainty
rng = np.random.default_rng(0); w = np.linspace(100, 2000, 191)
true = DielectricModel(eps_inf=3.0, terms=[Lorentz(w0=450.0, wp=600.0, gamma=20.0)])
R0 = simulate(true, w, "R"); tv = {"eps_inf": 3.0, "w0": 450.0, "wp": 600.0, "gamma":
↳ 20.0}
hits = None
for _ in range(30):
    spec = R0.copy(y=R0.y + rng.normal(0, 0.003, w.size), yerr=np.full(w.size, 0.003))
    r = fit([FitTerm(spec, DielectricModel(eps_inf=2.9, terms=[Lorentz(w0=455.0,
↳ wp=590.0, gamma=22.0)))]))
    u = evaluate_uncertainty(r)
    truth = np.array([tv[n.split(".")[1]] for n in u.names])
    hit = (np.abs(u.values - truth) <= u.expanded).astype(int)
```

```

    hits = hit if hits is None else hits + hit
print(bool(np.all(hits / 30 >= 0.8)))      # every parameter's k = 2 interval covers the
↪ truth in at least 80 % of 30 trials

```

Printed True at this build, in 0.09 s.

Changelog of the method. 0.20.0.

Dependencies

every borrowed routine, with its algorithm, its citation, its licence and the version the checks ran at.

D01 · `scipy.optimize.least_squares`

Dependencies · optimisation · BSD-3-Clause (SciPy)

Statement. The bounded non-linear least-squares solver behind N01 and N05: method `trf` is the trust-region reflective algorithm of Branch, Coleman and Li (1999), `lm` is MINPACK's `lmdr` (Moré 1978); `dogbox` is not used by `difes`. Default tolerances 10^{-8} on cost, step and gradient.

Reference. Branch, Coleman and Li, SIAM J. Sci. Comput. 21 (1999) 1; Moré, in Numerical Analysis, Lecture Notes in Mathematics 630 (1978) 105

What the version pins. The fits of T42 (RefFIT's tutorial) reproduce MODEL2 to 10^{-6} at the version recorded on this card; a change of version that moved them would be caught by the ledger's suite.

Used by. N01, N02, N05

Check it yourself. The last printed line must be 1 2.0.

```

import numpy as np
from scipy.optimize import least_squares
res = least_squares(lambda x: x - 3.0, [0.0], bounds=([-1.0], [2.0]))      # the
↪ unconstrained optimum 3 lies outside the bound
print(res.status, round(float(res.x[0]), 6))      # stops on the bound (status 1:
↪ gradient tolerance met on the active set)

```

Printed 1 2.0 at this build, in 0.0 s.

D02 · `numpy.random.default_rng (PCG64)`

Dependencies · random-numbers · BSD-3-Clause (NumPy)

Statement. The seeded generator of the Monte Carlo (N16, M02, M07): PCG64 (O'Neill 2014) with numpy's `ziggurat` for normal deviates. A seed gives the same stream on every machine at the same numpy version; the first uniform deviate of seed 0 is the number the check prints.

Reference. O'Neill, PCG (2014); `numpy.random` documentation

What the version pins. A seeded run of M02 is reproduced bit for bit; the seed is written into every uncertainty report.

Used by. N16, M02, M03, M07, L02

Check it yourself. The last printed line must be 0.6369616873214543.

```

import numpy as np
print(np.random.default_rng(0).random())

```

Printed 0.6369616873214543 at this build, in 0.0 s.

D03 · `scipy.interpolate.CubicSpline` (natural boundary)

Dependencies · *interpolation* · BSD-3-Clause (SciPy)

Statement. The cubic interpolant whose piecewise coefficients N04 integrates in closed form, and FOCUS's cubic-spline function in the language (N10). Natural boundary condition: second derivative zero at both ends.

Reference. de Boor, A Practical Guide to Splines (1978)

What the version pins. The coefficient layout (4, n-1), highest power first, in the local variable $t = s - s_i$, is what N04 relies on.

Used by. N04, N10

Check it yourself. The last printed line must be (4, 10) 0.0 0.0.

```
import numpy as np
from scipy.interpolate import CubicSpline
s = np.linspace(0, 10, 11); cs = CubicSpline(s, np.sin(s), bc_type="natural")
print(cs.c.shape, round(abs(float(cs(s[0], 2))), 12), round(abs(float(cs(s[-1], 2))),
↵ 12)) # (4, n-1) and zero curvature at the ends
```

Printed (4, 10) 0.0 0.0 at this build, in 0.0 s.

D04 · `numpy.linalg.pinv` (SVD pseudo-inverse)

Dependencies · *linear-algebra* · BSD-3-Clause (NumPy), LAPACK gesdd underneath

Statement. The inverse of the normal matrix in N02 and M01, by singular value decomposition with singular values below 10^{-15} of the largest treated as zero, so that a rank-deficient (degenerate) problem returns the minimum-norm inverse instead of raising.

Reference. Golub and Van Loan, Matrix Computations, 4th ed. (2013) §5.5

What the version pins. A degenerate two-Drude fit (the chromium of the validation pages) gets finite, large errors and a correlation near $\pm \$1$ rather than a failure.

Used by. N01, N02, M01, M05, M08

Check it yourself. The last printed line must be True 0.25.

```
import numpy as np
M = np.array([[1.0, 1.0], [1.0, 1.0]]) # rank one
P = np.linalg.pinv(M)
print(bool(np.all(np.isfinite(P))), round(float(P[0, 0]), 6))
```

Printed True 0.25 at this build, in 0.0 s.

D05 · `scipy.stats.t.ppf` and `norm.ppf`

Dependencies · *statistics* · BSD-3-Clause (SciPy)

Statement. The quantiles of Student's t and of the normal distribution for the coverage factor of M04.

Reference. JCGM 100:2008 Table G.2 for the values

What the version pins. t at 95 % and 5 degrees of freedom is 2.571 (GUM Table G.2).

Used by. M04

Check it yourself. The last printed line must be 1.96 2.571.

```
from scipy import stats
print(round(float(stats.norm.ppf(0.975)), 3), round(float(stats.t.ppf(0.975, 5)), 3))
```

Printed 1.96 2.571 at this build, in 0.0 s.

D06 · scipy.special wofz and dawson

Dependencies · special-functions · BSD-3-Clause (SciPy); the Faddeeva package is MIT

Statement. The Faddeeva function $w(z)$ (Algorithm 916, Zaghoul and Ali 2011, via Johnson’s implementation) for the Voigt and causal Voigt profiles, and the Dawson function for the causal Gaussian (N11). The value of w at iY is $\exp(Y^2) \operatorname{erfc}(Y)$, which the check uses.

Reference. Zaghoul and Ali, ACM Trans. Math. Softw. 38 (2011) 15; Johnson, Faddeeva package (2012)

What the version pins. Accuracy about 10^{-13} relative, far below FOCUS’s own Voigt ripple of 1.5×10^{-3} (T18).

Used by. N11, P05

Check it yourself. The last printed line must be 1.0 0.0 True.

```
import math
from scipy.special import wofz, dawson, erfc
print(round(abs(complex(wofz(0.0))), 12), round(float(dawson(0.0)), 12),
      ↪ bool(abs(wofz(1j).real - math.exp(1.0) * erfc(1.0)) < 1e-12))
```

Printed 1.0 0.0 True at this build, in 0.0 s.

D07 · numpy.interp, numpy.trapezoid, numpy.gradient, numpy.quantile

Dependencies · array-routines · BSD-3-Clause (NumPy)

Statement. The array routines behind N13 (linear interpolation, held at the ends), N04’s value at zero (the trapezoid rule), N15 (second-order gradient) and M02’s coverage intervals (the linear-interpolation quantile, numpy’s default). trapezoid replaced trapz in numpy 2.0.

Reference. NumPy documentation

What the version pins. numpy 2 is required for trapezoid; the version is recorded on this card at build time.

Used by. N04, N05, N06, N07, N10, N13, N15, N16, M01, M02, M03, M05, M07, M08, P16, L06

Check it yourself. The last printed line must be 0.5 1.0 2.0.

```
import numpy as np
x = np.linspace(0, 1, 11)
print(round(float(np.trapezoid(x, x)), 12), float(np.interp(2.0, x, x)),
      ↪ float(np.quantile(np.arange(5.0), 0.5)))
```

Printed 0.5 1.0 2.0 at this build, in 0.0 s.

Physics

the model algebra, the profiles, the optics of bulk, slab, film and stack, the mixing rules, the units.

P01 · The dielectric model: ε_∞ times the factorised modes plus the additive susceptibilities, with bounded, freezable parameters

Physics · model-algebra

Statement. A model is $\varepsilon(\omega) = \varepsilon_\infty \times \Pi(\text{multiplicative terms}) + \Sigma(\text{additive terms})$. Additive terms (Drude, Lorentz, the causal Gaussian, Fano, Tauc–Lorentz, an expression, the variational function) contribute a susceptibility χ ; multiplicative terms (the four-parameter modes) are factors on ε_∞ . Every parameter is a Parameter with a value, a lower and an upper bound, a frozen flag and a fitted error; a value outside its bounds is refused at construction. Parameters are addressed by path (`eps_inf`, `0.lorentz.w0`) and may be shared between models (a temperature series with a common ε_∞) by pointing several models at one object, which copies preserve. Conventions: ω in cm^{-1} , $e^{-i t}$, $\varepsilon'' \geq 0$ for $\omega > 0$.

$$\varepsilon(\omega) = \varepsilon_\infty \prod_j \frac{\omega_{LO,j}^2 - \omega^2 - i\gamma_{LO,j}\omega}{\omega_{TO,j}^2 - \omega^2 - i\gamma_{TO,j}\omega} + \sum_k \chi_k(\omega)$$

Why this choice. One algebra for both of FOCUS’s model families (the sum of profiles and the factorised product) so that a FOCUS model maps term by term and a RefFIT table row by row; bounds and frozen flags on the parameter itself, not on the fitter, so that a shell’s live edit and the optimiser see the same thing. Oscillators are stored as $(\omega_0, \omega_p, \gamma)$, RefFIT’s form, with FOCUS’s strength $S = \omega_p^2/\omega_0^2$ as a property and a constructor.

Error and its order. None; a definition. Models mixing a factorised mode with additive terms are difes’s own extension of FOCUS’s two families and round-trip through OSML only numerically (X03).

Reference. Kuzmenko, Rev. Sci. Instrum. 76 (2005) 083108 (RefFIT’s table); Gervais and Piriou, J. Phys. C 7 (1974) 2374 (the factorised form); FOCUS help, `osml-core.html`

Counterparts. FOCUS (documented): FOCUS’s sum and product containers (`help`, `data-analysis.html`); its sample projects import term by term (T15, T16). RefFIT (documented): RefFIT’s Einf and (Wo, Wp, G) table with codes (manual §4.6.2); mapped in X06.

Implementation.

- `DielectricModel(eps_inf: Any = 1.0, terms: list[Term] | None = None, name: str = ) -> None` — `src/difes/models.py:399`
- `Parameter(name: str, value: float, lower: float = -inf, upper: float = inf, frozen: bool = False, error: float = nan) -> None` — `src/difes/models.py:33`
- `Term(**params: Any) -> None` — `src/difes/models.py:83`
- `example_model() -> DielectricModel` — `src/difes/models.py:621`
- `link_parameters(models: list[DielectricModel], path: str) -> Parameter` — `src/difes/models.py:631`
- `clone_models(models: list[DielectricModel]) -> list[DielectricModel]` — `src/difes/models.py:652`

Tests that name these symbols. `test_bench.py::test_spectrum_model_sample_simulate_are_tagged_nodes`, `test_bench.py::test_fit_asks_then_adopts`, `test_bench.py::test_layers_and_thicknesses_in_the_bench`, `test_cli.py::test_fit_command_recovers_parameters`, `test_cli.py::test_kk_and_cfrc_commands`, `test_digitize.py::test_bitmap_precision_on_a_known_curve`, `test_expr.py::test_expression_term_equals_bui`, `test_expr.py::test_expression_term_rejects_bad_parameters`, `test_fit.py::test_r1_recovery_from_wrong_gue`, `test_fit.py::test_double_drude_with_one_damping_frozen` and 37 more.

Ledger records. T01 (The profiles against physics: limits, closed forms, passivity, causality); T06 (FOCUS’s OSML files: quartz E, LiNbO_3 E, the four libraries, the round trip); T15 (FOCUS’s sample projects: the spectra read from the binary .pjt files).

Check it yourself. The last printed line must be `True 5.0 True ['eps_inf', '0.lorentz.w0', '0.lorentz.wp', '0.lorentz.gamma']`.

```
import numpy as np
from difes.models import DielectricModel, Lorentz, Parameter, link_parameters,
    clone_models
a = DielectricModel(eps_inf=3.0, terms=[Lorentz(w0=450.0, wp=600.0, gamma=20.0)])
b = DielectricModel(eps_inf=1.0, terms=[Lorentz(w0=900.0, wp=300.0, gamma=60.0)])
```

```

link_parameters([a, b], "eps_inf") # one eps_inf object for both
a2, b2 = clone_models([a, b]); a2.eps_inf.value = 5.0 # the copy keeps the sharing
try:
    Parameter("wp", -1.0, 0.0, np.inf); refused = False
except ValueError:
    refused = True
print(b.eps_inf is a.eps_inf, b2.eps_inf.value, refused, [n for n, _ in a.parameters()])

```

Printed True 5.0 True ['eps_inf', '0.lorentz.w0', '0.lorentz.wp', '0.lorentz.gamma'] at this build, in 0.0 s.

Changelog of the method. 0.1.0; 0.6.0: shared parameters; 0.19.0: effective-medium and tabulated subclasses.

P02 · The Drude term: free carriers

Physics · profile

Statement. $\chi = -\omega_p^2/(\omega^2 + i\omega)$: a plasma frequency and a damping, both bounded below by zero, with analytic derivatives for the fitter (N03). At $\omega = \omega_p$ with no damping $\chi = -1$, so ε crosses zero there for $\varepsilon_\infty = 1$; the dc conductivity is $\omega_p^2/(60\gamma)$ in $\Omega^{-1}\text{cm}^{-1}$ (P17). The wavenumber is kept away from zero by a small floor so that the term evaluates on a grid starting at 0.

$$\chi_D(\omega) = -\frac{\omega_p^2}{\omega^2 + i\gamma\omega}$$

Why this choice. RefFIT's form (a Lorentz row with $\omega_0 = 0$) and FOCUS's `drude(x, wp, g)`, which the OSML reader maps directly. RefFIT's code `−9` (a Drude with signed spectral weight) and its extended Drude outputs are separate (X06, P17).

Error and its order. None; a closed form. Its causality is checked numerically in the suite (N09).

Reference. Drude, Ann. Phys. 306 (1900) 566; Dressel and Grüner, Electrodynamics of Solids (2002) §5.1

Counterparts. FOCUS (inferred from numbers): `drude(x, wp, g)` assumed as $-\text{wp}^2/(x^2 + i g x)$ and confirmed on FOCUS's exports (T06, T17). RefFIT (documented): A row with $W_0 = 0$ (manual §4.6.2); met on Irati's files (T34).

Implementation.

- `Drude(**params: Any) -> None` — `src/difes/models.py:148`

term	parameter	lower	upper
Drude	wp	0.0	inf
Drude	gamma	0.0	inf

Tests that name these symbols. `test_bench.py::test_layers_and_thicknesses_in_the_bench`, `test_fit.py::test_double_drude_with_one_damping_frozen`, `test_fit.py::test_analytic_jacobian_matches_fit`, `test_models.py::test_term_rejects_bad_parameter_names`, `test_models.py::test_drude_known_values`, `test_models.py::test_fpsq_equals_lorentz_when_dampings_equal`, `test_models.py::test_drude_causal`, `test_optics.py::test_metal_reflectivity_shape`, `test_optics.py::test_quantity_with_substrate`, `test_osml.py::test_write_and_read_mixed_model` and 7 more.

Ledger records. T01 (The profiles against physics: limits, closed forms, passivity, causality); T06 (FOCUS's OSML files: quartz E, LiNbO₃ E, the four libraries, the round trip); T34 (RefFIT's own output: the comparison protocol run by Irati, number against number).

Check it yourself. The last printed line must be `[-1.+0.j]`.

```
import numpy as np
from difes.models import Drude
print(Drude(wp=1000.0, gamma=0.0).epsilon(np.array([1000.0])))      # chi = -1 at the
    ↪ plasma frequency
```

Printed `[-1.+0.j]` at this build, in 0.0 s.

Changelog of the method. 0.1.0.

P03 · The Lorentz oscillator, in plasma-frequency and in strength form

Physics · profile

Statement. $\chi = \omega_p^2/(\omega_0^2 - \omega^2 - i\omega)$, stored as $(\omega_0, \omega_p, \gamma)$ with analytic derivatives; FOCUS’s phonon-3p form $\Delta\varepsilon \omega_0^2/(\omega_0^2 - \omega^2 - i\omega)$ is the same term with $\omega_p^2 = \Delta\varepsilon \omega_0^2$, offered as a constructor from the strength and as a property. The static limit is $\chi(0) = \Delta\varepsilon$, the strength, and the Lyddane–Sachs–Teller relation follows for a single mode: $\varepsilon(0)/\varepsilon_\infty = \omega_{LO}^2/\omega_{TO}^2$.

$$\chi_L(\omega) = \frac{\omega_p^2}{\omega_0^2 - \omega^2 - i\gamma\omega} = \frac{\Delta\varepsilon \omega_0^2}{\omega_0^2 - \omega^2 - i\gamma\omega}$$

Why this choice. The plasma-frequency form inside, because RefFIT’s table and the sum rules use it and because ω_p is the parameter whose uncertainty a scale error moves (M01); the strength form at the surface for FOCUS’s users and files.

Error and its order. None; a closed form. A negative non-integer ω_0 in a RefFIT file is the oscillator at $|\omega_0|$ (RefFIT squares it), handled in X06.

Reference. Lorentz, The Theory of Electrons (1909); Kuzmenko 2005 Eq. 9; FOCUS help, core-functions.htm (phonon-3p)

Counterparts. FOCUS (inferred from numbers): phonon-3p(x, dEps, w0, g) confirmed against FOCUS’s exports (T06, T15). RefFIT (documented): An ordinary table row (manual §4.6.2).

Implementation.

- Lorentz(**params: Any) -> None — src/difes/models.py:166

term	parameter	lower	upper
Lorentz	w0	0.0	inf
Lorentz	wp	0.0	inf
Lorentz	gamma	0.0	inf

Tests that name these symbols. test_bench.py::test_fit_asks_then_adopts, test_bench.py::test_layers_and_test_cli.py::test_fit_command_recovers_parameters, test_cli.py::test_kk_and_cfrc_commands, test_digitize.py::test_bitmap_precision_on_a_known_curve, test_expr.py::test_expression_term_equals_bui test_fit.py::test_r1_recovery_from_wrong_guess, test_fit.py::test_two_spectra_share_a_linked_parameter, test_fit.py::test_analytic_jacobian_matches_finite_differences, test_graph.py::test_evaluation_and_addr and 23 more.

Ledger records. T01 (The profiles against physics: limits, closed forms, passivity, causality); T06 (FOCUS’s OSMF files: quartz E, LiNbO₃ E, the four libraries, the round trip); T15 (FOCUS’s sample projects: the spectra read from the binary .pjt files); T42 (RefFIT’s own tutorial, Part 1: RefFIT’s data and saved models, and difes’s fitter against RefFIT’s).

Check it yourself. The last printed line must be `[4.+0.j] 4.0 800.0`.

```
import numpy as np
from difes.models import Lorentz
L = Lorentz.from_strength(400.0, 4.0, 10.0) # FOCUS's form: strength 4
print(L.epsilon(np.array([0.0])), L.strength, L["wp"].value) # chi(0) = strength; wp =
↳ sqrt(4) * 400
```

Printed [4.+0.j] 4.0 800.0 at this build, in 0.0 s.

Changelog of the method. 0.1.0.

P04 · The four-parameter factorised mode (Gervais–Piriou, FOCUS’s phonon-4p)

Physics · profile

Statement. One polar mode of the factorised semi-quantum model, a multiplicative factor on ε_∞ with TO and LO frequencies and dampings, all bounded below by zero. With $\gamma_{LO} = \gamma_{TO}$ the factor equals a Lorentz oscillator of $\omega_p^2 = \varepsilon_\infty(\omega_{LO}^2 - \omega_{TO}^2)$ added to ε_∞ , the limit the suite checks. Passivity at high frequency needs the sum of $(\gamma_{LO} - \gamma_{TO})$ over the modes to be non-negative (N12). The product of several modes is FOCUS’s product container and RefFIT’s TO–LO codes –1000 to –1002.

$$\varepsilon(\omega) = \varepsilon_\infty \prod_j \frac{\omega_{LO,j}^2 - \omega^2 - i\gamma_{LO,j}\omega}{\omega_{TO,j}^2 - \omega^2 - i\gamma_{TO,j}\omega}$$

Why this choice. Kept as a factor rather than expanded into partial fractions, so that FOCUS’s parameters are difes’s parameters and the LO frequencies are read off directly; the fitter handles the product’s derivatives numerically (N03), the one built-in term without analytic ones.

Error and its order. None; a closed form. Below the first TO frequency the product can be slightly non-passive (FOCUS’s quartz example): the principal root of ε keeps $n > 0$ and lets k go slightly negative, as FOCUS does (P10).

Reference. Gervais and Piriou, J. Phys. C 7 (1974) 2374; De Sousa Meneses et al., J. Non-Cryst. Solids 351 (2005) 124; FOCUS help, core-functions.htm (phonon-4p)

Counterparts. FOCUS (inferred from numbers): phonon-4p(x, wTO, gTO, wLO, gLO) confirmed to FOCUS’s seven digits on its quartz and LiNbO₃ examples (T15) and Irati’s model (T06). RefFIT (documented): RefFIT’s TO–LO formula, codes –1000/–1001/–1002 (manual Table 4-3), paired one to one in X06.

Implementation.

- FourParameterMode(**params: Any) -> None — src/difes/models.py:195

term	parameter	lower	upper
FourParameterMode	w_to	0.0	inf
FourParameterMode	g_to	0.0	inf
FourParameterMode	w_lo	0.0	inf
FourParameterMode	g_lo	0.0	inf

Tests that name these symbols. test_fit.py::test_fd_fallback_for_slab_and_fpsq, test_models.py::test_fps, test_models.py::test_fpsq_passive_when_lo_damping_not_smaller, test_models.py::test_fpsq_causal, test_osml.py::test_read_quartz_e_model, test_reffit.py::test_tolo_code_is_the_four_parameter_mode.

Ledger records. T01 (The profiles against physics: limits, closed forms, passivity, causality); T06 (FOCUS’s OSML files: quartz E, LiNbO₃ E, the four libraries, the round trip); T15 (FOCUS’s sample projects: the spectra read from the binary .pjt files).

Check it yourself. The last printed line must be True.

```

import numpy as np
from difes.models import DielectricModel, FourParameterMode, Lorentz
w = np.linspace(100, 2000, 5)
mode = DielectricModel(eps_inf=2.0, terms=[FourParameterMode(w_to=400.0, g_to=10.0,
    ↪ w_lo=600.0, g_lo=10.0)])
lor = DielectricModel(eps_inf=2.0, terms=[Lorentz(w0=400.0, wp=np.sqrt(2.0 * (600.0**2 -
    ↪ 400.0**2)), gamma=10.0)])
print(bool(np.allclose(mode.epsilon(w), lor.epsilon(w))))      # equal dampings: a Lorentz
    ↪ oscillator

```

Printed *True* at this build, in 0.0 s.

Changelog of the method. 0.1.0.

P05 · The causal Gaussian band for disordered solids (FOCUS's gaussian-kkg)

Physics · profile

Statement. χ'' is a Gaussian of peak A and full width at half maximum σ centred at ω_0 , minus its mirror image at $-\omega_0$ so that χ'' is odd; χ' is its Hilbert transform in closed form through the Dawson function (N11), so the pair is causal by construction. Analytic derivatives for the fitter. The published form without renormalisation, confirmed by a one-band FOCUS export of a Gaussian wider than its centre (T20).

$$\chi''(\omega) = A \left[e^{-a^2(\omega-\omega_0)^2} - e^{-a^2(\omega+\omega_0)^2} \right], \quad \chi'(\omega) = \frac{2A}{\sqrt{\pi}} [D(a(\omega + \omega_0)) - D(a(\omega - \omega_0))], \quad a = \frac{2\sqrt{\ln 2}}{\sigma}$$

Why this choice. De Sousa Meneses's form for glasses, where a Lorentzian's wings are wrong; the mirror term is what makes the band causal and matters only for bands wider than their centre. FOCUS's voigt-kkg, the Voigt generalisation with FOCUS's own normalisation, lives in the formula language (N11).

Error and its order. None; a closed form; causality checked to 0.1 % in the suite (N09).

Reference. De Sousa Meneses, Malki and Echegut, J. Non-Cryst. Solids 352 (2006) 769; Kim, Lee and Kim, Phys. Rev. B 55 (1997) 3485 (Gaussian with Dawson's function)

Counterparts. FOCUS (inferred from numbers): gaussian-kkg met on the phosphate glass's twelve bands (T17) and on one-band exports (T20). RefFIT (documented): RefFIT's Gaussian band is a different construction (T33).

Implementation.

- CausalGaussian(**params: Any) -> None — src/difes/models.py:220

Library routines called. `scipy.special.dawsn` (D06)

term	parameter	lower	upper
CausalGaussian	w0	0.0	inf
CausalGaussian	A	0.0	inf
CausalGaussian	sigma	1e-09	inf

Tests that name these symbols. `test_fit.py::test_analytic_jacobian_matches_finite_differences`, `test_models.py::test_gaussian_peak_and_symmetry`, `test_models.py::test_fpsq_equals_lorentz_when_dampings`, `test_models.py::test_gaussian_causal`, `test_osml.py::test_write_and_read_mixed_model`.

Ledger records. T01 (The profiles against physics: limits, closed forms, passivity, causality); T17 (The phosphate glass: twelve causal Gaussians against FOCUS's export); T20 (Four one-band exports: the Gaussian convention closed, FOCUS's Voigt characterised).

Check it yourself. The last printed line must be 1.5 0.5.

```

import numpy as np
from difes.models import CausalGaussian
g = CausalGaussian(w0=1000.0, A=1.5, sigma=120.0)
e = g.epsilon(np.array([1000.0, 1060.0])) # at the centre and half a
    ↪ width away
print(round(float(e.imag[0]), 6), round(float(e.imag[1] / e.imag[0]), 6)) # peak A;
    ↪ half height at sigma/2

```

Printed 1.5 0.5 at this build, in 0.0 s.

Changelog of the method. 0.1.0; 0.12.0: the normalisation question closed by T20.

P06 · The Fano-shaped Lorentz oscillator (RefFIT's codes —65 and —66)

Physics · profile

Statement. A Lorentz oscillator multiplied by $(1 + i \omega_q/\omega)^2$ plus the tail $(\omega_p \omega_q/(\omega_0 \omega))^2$, an asymmetric line that is Kramers–Kronig consistent and sum-rule friendly; $\omega_q = 0$ gives the symmetric Lorentzian and the conventional asymmetry parameter is $q = \omega_0/\omega_q$. χ'' can go negative, so the term needs a positive background to sit on. Analytic derivatives; the wavenumber floored away from zero.

$$\chi_F(\omega) = \frac{\omega_p^2}{\omega_0^2 - \omega^2 - i\gamma\omega} \left(1 + \frac{i\omega_q}{\omega}\right)^2 + \left(\frac{\omega_p \omega_q}{\omega_0 \omega}\right)^2$$

Why this choice. RefFIT's form (manual Table 4-3, after Damascelli), so that a RefFIT model with Fano rows imports as a term and exports to FOCUS as a formula symbol (X03).

Error and its order. None; a closed form; causality checked in the suite (T33).

Reference. Damascelli, PhD thesis, Groningen (1999); RefFIT manual Table 4-3; Fano, Phys. Rev. 124 (1961) 1866

Counterparts. FOCUS (documented): Not a FOCUS core profile; exported to FOCUS as a formula symbol. RefFIT (documented): Codes —65/—66 and their variants (manual Table 4-3), read in X06.

Implementation.

- `FanoLorentz(**params: Any) -> None` — `src/difes/models.py:303`

term	parameter	lower	upper
FanoLorentz	w0	1e-09	inf
FanoLorentz	wp	0.0	inf
FanoLorentz	gamma	0.0	inf
FanoLorentz	wq	0.0	inf

Tests that name these symbols. `test_reffit.py::test_fano_reduces_to_lorentz_and_is_causal`, `test_reffit.py::test_fano_exports_to_osml_and_reads_back`.

Ledger records. T33 (RefFIT's model table and its line shapes).

Check it yourself. The last printed line must be True.

```

import numpy as np
from difes.models import FanoLorentz, Lorentz
w = np.linspace(100, 2000, 5)
print(bool(np.allclose(FanoLorentz(w0=400.0, wp=800.0, gamma=10.0, wq=0.0).epsilon(w),
    ↪ Lorentz(w0=400.0, wp=800.0, gamma=10.0).epsilon(w))))

```

Printed True at this build, in 0.0 s.

Changelog of the method. 0.18.0.

P07 · The Tauc–Lorentz oscillator (Jellison and Modine, RefFIT’s codes —55 and —56)

Physics · profile

Statement. χ'' is zero below the gap ω_g and $A \omega_0 \gamma (\omega - \omega_g)^2 / [\omega((\omega^2 - \omega_0^2)^2 + \gamma^2 \omega^2)]$ above it; χ' is its Kramers–Kronig transform in the closed form of Jellison and Modine with the erratum, the two $\ln|\omega - \omega_g|$ terms combined so that the gap point is regular. The term is the susceptibility (RefFIT’s $\varepsilon = 1 + \dots$ puts the 1 in ε_∞). Requires $4 \omega_0^2 > \gamma^2$; the derivative with respect to A is analytic and the others are central differences of the closed form.

$$\chi''(\omega) = \frac{A \omega_0 \gamma (\omega - \omega_g)^2}{\omega [(\omega^2 - \omega_0^2)^2 + \gamma^2 \omega^2]} \quad (\omega > \omega_g), \quad 0 \text{ below}$$

Why this choice. The closed-form real part rather than a numerical transform, so that the term evaluates at the speed of the others and the fitter has it on every step; the suite checks χ' against a numerical transform of χ'' .

Error and its order. None in the closed form; the finite-difference derivatives carry N03’s order.

Reference. Jellison and Modine, Appl. Phys. Lett. 69 (1996) 371, erratum 2137; RefFIT manual Table 4-3

Counterparts. FOCUS (documented): Not a FOCUS profile and not exportable to OSML (no formula FOCUS can hold); X03 refuses it with a message. RefFIT (documented): Codes —55/—56 and their variants, read in X06.

Implementation.

- `TaucLorentz(**params: Any) -> None` — `src/difes/models.py:337`

term	parameter	lower	upper
TaucLorentz	w0	1e-09	inf
TaucLorentz	wg	0.0	inf
TaucLorentz	A	0.0	inf
TaucLorentz	gamma	1e-09	inf

Tests that name these symbols. `test_reffit.py::test_tauc_lorentz_shape_and_causality`, `test_reffit.py::test_fano_exports_to_osml_and_reads_back`.

Ledger records. T33 (RefFIT’s model table and its line shapes).

Check it yourself. The last printed line must be `0.0 True`.

```
import numpy as np
from difes.models import TaucLorentz
e = TaucLorentz(w0=3000.0, wg=1500.0, A=100.0, gamma=500.0).epsilon(np.array([1000.0,
↪ 2000.0]))
print(float(e.imag[0]), bool(e.imag[1] > 0))           # no absorption below the gap,
↪ some above
```

Printed 0.0 True at this build, in 0.0 s.

Changelog of the method. 0.18.0.

P08 · A user formula as a fittable term

Physics · profile

Statement. ExpressionTerm takes a formula in the math language, `name(x, a, b, ...) = expression`, FOCUS style: the first argument (or a named one) is the wavenumber and the others become Parameters with bounds

and frozen flags, so the term fits like a built-in one, through finite differences (N03). Causality is the user's responsibility and N09 measures it. The OSML reader produces such terms for FOCUS profiles given as formula symbols and for anything else the language can express; the writer exports them back as formula symbols (X02, X03).

$$\chi(\omega) = f(\omega; a, b, \dots) \quad \text{with } f \text{ parsed, not evaluated by Python}$$

Why this choice. One mechanism for everything FOCUS's formula editor can write, so that a FOCUS user's own profile enters difes unchanged; the price is a numerical Jacobian for the whole fit when one such term is present.

Error and its order. None of its own; the formula's. A term with an arity the language cannot see (a two-variable helper) is skipped by the reader with a note.

Reference. FOCUS help, formula-editor.html

Counterparts. FOCUS (documented): FOCUS's formula-encoded symbols (help, formula-editor.html); round-tripped on its examples (T15). RefFIT (documented): RefFIT's special codes that have a closed form are read as expression terms (X06).

Implementation.

- `ExpressionTerm(formula: str, variable: str | None = None, **params: Any) -> None` —
src/difes/models.py:260

Tests that name these symbols. `test_expr.py::test_expression_term_equals_builtin_lorentz_and_fits`,
`test_expr.py::test_expression_term_causality_check`, `test_expr.py::test_expression_term_rejects_bad_params`,
`test_osml.py::test_write_and_read_mixed_model`.

Ledger records. T05 (The software around the physics: language, spectra, sessions, server, Bench); T15 (FOCUS's sample projects: the spectra read from the binary .pjt files).

Check it yourself. The last printed line must be ('A', 'w0', 'g') True.

```
import numpy as np
from difes.models import ExpressionTerm, Lorentz
w = np.linspace(100, 2000, 5)
t = ExpressionTerm("myosc(x, A, w0, g) = A*w0^2/(w0^2 - x^2 - i*g*x)", A=4.0, w0=400.0,
    ↪ g=10.0)
print(t.param_names, bool(np.allclose(t.epsilon(w), Lorentz.from_strength(400.0, 4.0,
    ↪ 10.0).epsilon(w))))
```

Printed ('A', 'w0', 'g') True at this build, in 0.0 s.

Changelog of the method. 0.5.0.

P09 · Effective media: Bruggeman, Maxwell–Garnett, Looyenga and Lichtenecker, as a fittable model

Physics · effective-medium

Statement. An EffectiveMediumModel mixes an inclusion model into a host (a constant, air, or another model) by one of four rules with the inclusion's volume fraction as a parameter bounded to [0, 1]. Bruggeman solves the quadratic of the two-component self-consistent equation and takes the root with the larger imaginary part (the passive one), which is FOCUS's choice wherever a lossy inclusion is mixed and, unlike FOCUS's, gives the host at zero fill and the inclusion at full fill for a lossless pair. Maxwell–Garnett is the dilute-inclusion closed form. Looyenga is the cube-root rule on the principal branch, Lichtenecker the power law with an exponent (zero selecting the logarithmic rule), both for polycrystals and mixtures without a host.

$$f \frac{\varepsilon_i - \varepsilon}{\varepsilon_i + 2\varepsilon} + (1 - f) \frac{\varepsilon_h - \varepsilon}{\varepsilon_h + 2\varepsilon} = 0; \quad \frac{\varepsilon - \varepsilon_h}{\varepsilon + 2\varepsilon_h} = f \frac{\varepsilon_i - \varepsilon_h}{\varepsilon_i + 2\varepsilon_h}; \quad \varepsilon^{1/3} = f \varepsilon_i^{1/3} + (1 - f) \varepsilon_h^{1/3}$$

Why this choice. A native model rather than a formula so that the fill fits with bounds and errors and a layer of it can sit in a stack (P13); the formula route exists too (Irati’s Eeff-BG symbol imports as a formula, X01). The root rule is stated because it is where an implementation can silently differ from FOCUS’s.

Error and its order. None; closed forms. The root choice differs from FOCUS’s only for lossless pairs at the ends of the fill range, where FOCUS’s rule gives the wrong limit.

Reference. Bruggeman, Ann. Phys. 416 (1935) 636; Maxwell Garnett, Phil. Trans. R. Soc. A 203 (1904) 385; Looyenga, Physica 31 (1965) 401; Lichtenecker, Phys. Z. 27 (1926) 115

Counterparts. FOCUS (inferred from numbers): FOCUS’s Eeff-BG symbol in Irati’s projects (T06) and the Bruggeman, Looyenga and Lichtenecker nodes of Mireia’s project (T48). RefFIT (documented): Not in RefFIT.

Implementation.

- `EffectiveMediumModel(inclusion: DielectricModel, host: Any = 1.0, fill: Any = 0.5, rule: str = bruggeman, name: str = ) -> None` — `src/difes/models.py:500`
- `bruggeman(eps_host: np.ndarray | complex, eps_incl: np.ndarray | complex, fill: float) -> np.ndarray` — `src/difes/optics.py:246`
- `maxwell_garnett(eps_host: np.ndarray | complex, eps_incl: np.ndarray | complex, fill: float) -> np.ndarray` — `src/difes/optics.py:283`
- `looyenga(eps_host: np.ndarray | complex, eps_incl: np.ndarray | complex, fill: float) -> np.ndarray` — `src/difes/optics.py:265`
- `lichtenecker(eps_host: np.ndarray | complex, eps_incl: np.ndarray | complex, fill: float, alpha: float | None = None) -> np.ndarray` — `src/difes/optics.py:273`

Tests that name these symbols. `test_bench.py::test_layers_and_thicknesses_in_the_bench`, `test_stack.py::test_bruggeman_limits_and_passivity`, `test_stack.py::test_bruggeman_is_focus_eeff_bg`, `test_stack.py::test_one_layer_stack_is_the_film`, `test_stack.py::test_effective_medium_model_round_trip`, `test_stack.py::test_native_stack_meets_focus_export_to_the_files_digits`, `test_stack.py::test_thicknesse`, `test_tabulated.py::test_looyenga_and_lichtenecker_limits_and_passivity`, `test_tabulated.py::test_effecti`

Ledger records. T06 (FOCUS’s OSML files: quartz E, LiNbO₃ E, the four libraries, the round trip); T48 (Mireia’s alumina project: her Bergman model read from the OSML export, FOCUS’s own curve met to 4e-6, her two complaints explained).

Check it yourself. The last printed line must be `(1+0j) (4+1j) (1+0j) (3.375+0j)`.

```
import numpy as np
from difes.optics import bruggeman, maxwell_garnett, looyenga
print(bruggeman(1.0, 4.0 + 1j, 0.0), bruggeman(1.0, 4.0 + 1j, 1.0), maxwell_garnett(1.0,
↪ 4.0 + 1j, 0.0), np.round(looyenga(1.0, 8.0 + 0j, 0.5), 6))
```

Printed (1+0j) (4+1j) (1+0j) (3.375+0j) at this build, in 0.0 s.

Changelog of the method. 0.10.0: Bruggeman and Maxwell–Garnett; 0.30.0: Looyenga and Lichtenecker.

P10 · From ε to the bulk sample: the principal root, the Fresnel amplitude, R and Kirchhoff’s $E = 1 - R$

Physics · optics

Statement. $\tilde{n} = n + ik$ is the principal square root of ε (real part non-negative); for a passive medium that is the branch with $k \geq 0$, and where a model is slightly non-passive the principal root keeps $n > 0$ and lets k go slightly negative, as FOCUS does. The normal-incidence amplitude is $\tilde{r} = (\tilde{n} - 1)/(\tilde{n} + 1) = r e^{i\theta}$, zero phase for a transparent medium and θ the Kramers–Kronig phase of $\ln \tilde{r}$; $R = |\tilde{r}|^2$, and the emissivity of an opaque sample is $E = 1 - R$ (Kirchhoff). The conversions $\varepsilon \leftrightarrow (n, k) \leftrightarrow (r, \theta) \leftrightarrow \chi$ are the inverses of each other to machine precision.

$$\tilde{n} = \sqrt{\varepsilon}, \quad \tilde{r} = \frac{\tilde{n} - 1}{\tilde{n} + 1} = r e^{i\theta}, \quad R = |\tilde{r}|^2, \quad E = 1 - R$$

Why this choice. The principal root rather than forcing $k \geq 0$, because flipping the root where a model is slightly non-passive would make n negative and R larger than one; the phase convention (zero for a transparent medium) is the one the finite-range transform (N04) and the paper use.

Error and its order. None; closed forms. A non-passive model is the model's error, visible as $k < 0$.

Reference. Born and Wolf, Principles of Optics, §1.5; Dressel and Grüner, Electrodynamics of Solids (2002) §2.4

Counterparts. FOCUS (inferred from numbers): FOCUS's Reflectivity-E symbol, $\text{Norm}((\sqrt{E_i} - \sqrt{E_o})/(\sqrt{E_i} + \sqrt{E_o}))$, met to seven digits (T15). RefFIT (inferred from numbers): RefFIT's R and R_{ph} outputs met to its printed digits (T34).

Implementation.

- `refractive_index(eps: np.ndarray) -> np.ndarray` — `src/difes/optics.py:19`
- `eps_to_nk(eps: np.ndarray) -> tuple[np.ndarray, np.ndarray]` — `src/difes/optics.py:30`
- `nk_to_eps(n: np.ndarray, k: np.ndarray) -> np.ndarray` — `src/difes/optics.py:35`
- `eps_to_chi(eps: np.ndarray) -> np.ndarray` — `src/difes/optics.py:40`
- `chi_to_eps(chi: np.ndarray) -> np.ndarray` — `src/difes/optics.py:44`
- `amplitude_reflection(eps: np.ndarray) -> np.ndarray` — `src/difes/optics.py:48`
- `nk_to_r_theta(n: np.ndarray, k: np.ndarray) -> tuple[np.ndarray, np.ndarray]` — `src/difes/optics.py:54`
- `r_theta_to_nk(r: np.ndarray, theta: np.ndarray) -> tuple[np.ndarray, np.ndarray]` — `src/difes/optics.py:59`
- `reflectivity_normal(eps: np.ndarray) -> np.ndarray` — `src/difes/optics.py:67`
- `emissivity_opaque(eps: np.ndarray) -> np.ndarray` — `src/difes/optics.py:72`

Tests that name these symbols. `test_digitize.py::test_bitmap_precision_on_a_known_curve`, `test_optics.py::test_refractive_index_branch`, `test_optics.py::test_fresnel_textbook_values`, `test_optics.py::test_conversion_round_trips`, `test_optics.py::test_thick_absorbing_slab_tends_to_opaque`, `test_optics.py::test_quantity_dispatch_and_simulate`, `test_optics.py::test_film_on_substrate_limits`, `test_optics.py::test_slab_emissivity_equals_focus_emittance_formula`, `test_osml.py::test_focus_libraries`, `test_plate_refit.py::test_ellipsometry_of_one_ray_is_the_bulk_rho` and 13 more.

Ledger records. T02 (The optics against textbook values: Fresnel, slabs, films, energy balance); T15 (FOCUS's sample projects: the spectra read from the binary .pjt files); T34 (RefFIT's own output: the comparison protocol run by Irati, number against number).

Check it yourself. The last printed line must be `[0.111111] [0.] 2.0 0.0`.

```
import numpy as np
from difes.optics import reflectivity_normal, amplitude_reflection, r_theta_to_nk,
    nk_to_r_theta
eps = np.array([4.0 + 0j]) # a transparent medium of
    index 2
n, k = r_theta_to_nk(*nk_to_r_theta(np.array([2.0]), np.array([0.0]))) # there and back
print(np.round(reflectivity_normal(eps), 6), np.angle(amplitude_reflection(eps)),
    round(float(n[0]), 12), round(float(k[0]), 12))
```

Printed [0.111111] [0.] 2.0 0.0 at this build, in 0.0 s.

Changelog of the method. 0.1.0.

P11 · The free-standing slab: coherent (Airy) and incoherent sums, and $E = 1 - R - T$

Physics · optics

Statement. For a slab of thickness d in air at normal incidence the single-pass phase is $\delta = 2\tilde{n}d$ (complex, with the absorption in its imaginary part). Coherent: the Airy amplitudes $r = r_1(1 - e^{\tilde{\delta}})/(1 - r_1^2 e^{\tilde{\delta}})$ and $t = (1 - r_1^2 e^{\tilde{\delta}})/(1 - r_1^2 e^{\tilde{\delta}})$, with fringes. Incoherent: intensities summed over the multiple

reflections with the single-pass attenuation $a = \exp(-4 k d)$, the fringes averaged out, which is what a spectrometer sees when the slab is thick against its coherence length. The emissivity of a semitransparent slab is $1 - R - T$.

$$T_{\text{inc}} = \frac{(1 - R_1)^2 a}{1 - R_1^2 a^2}, \quad R_{\text{inc}} = R_1 + \frac{(1 - R_1)^2 R_1 a^2}{1 - R_1^2 a^2}, \quad a = e^{-4\pi k w d}, \quad E = 1 - R - T$$

Why this choice. Both sums, chosen per term by the user, because the group measures both thin films with fringes and millimetre plates without; RefFIT's partial coherence and oblique plate are N14's generalisation. A film on a substrate with $\varepsilon_{\text{sub}} = 1$ reproduces the slab exactly, the consistency the suite checks.

Error and its order. None; closed forms. Energy is conserved by construction: $R + T + E = 1$.

Reference. Born and Wolf, Principles of Optics, §7.6; FOCUS help, Reflectance-E and Transmittance-E (osml-elements.html)

Counterparts. FOCUS (inferred from numbers): FOCUS's incoherent Reflectance-E and Transmittance-E met on the Borofloat plate (T18) and the phosphate glass (T17). RefFIT (inferred from numbers): RefFIT's plate optics (code -13) with its degree of coherence, met on its tutorial (T46); N14.

Implementation.

- `transmission_slab(eps: np.ndarray, omega: np.ndarray, thickness: float, coherent: bool = True) -> np.ndarray` — `src/difes/optics.py:90`
- `reflectivity_slab(eps: np.ndarray, omega: np.ndarray, thickness: float, coherent: bool = True) -> np.ndarray` — `src/difes/optics.py:105`
- `emissivity_slab(eps: np.ndarray, omega: np.ndarray, thickness: float, coherent: bool = True) -> np.ndarray` — `src/difes/optics.py:115`

Tests that name these symbols. `test_optics.py::test_slab_lossless_limits`, `test_optics.py::test_slab_energy`, `test_optics.py::test_thick_absorbing_slab_tends_to_opaque`, `test_optics.py::test_film_on_substrate_limit`, `test_optics.py::test_slab_emissivity_equals_focus_emittance_formula`, `test_osml.py::test_focus_libraries`, `test_osml.py::test_silica_example_imports_as_fittable_terms_and_is_physical`, `test_plate_refit.py::test_plate_refit`, `test_plate_refit.py::test_no_coherence_is_the_intensity_sum_and_meets_the_incoherent_slab_where_absorbs`

Ledger records. T02 (The optics against textbook values: Fresnel, slabs, films, energy balance); T17 (The phosphate glass: twelve causal Gaussians against FOCUS's export); T18 (The Borofloat plate: 32 causal Voigt bands, R and T of a 1 mm slab).

Check it yourself. The last printed line must be `1.0 True`.

```
import numpy as np
from difes.optics import reflectivity_slab, transmission_slab, emissivity_slab,
    reflectivity_film
eps = np.array([2.0 + 0.3j]); w = np.array([1000.0])
total = reflectivity_slab(eps, w, 0.01) + transmission_slab(eps, w, 0.01) +
    emissivity_slab(eps, w, 0.01)
same = np.allclose(reflectivity_film(eps, np.array([1.0 + 0j]), w, 0.01, coherent=False),
    reflectivity_slab(eps, w, 0.01, coherent=False))
print(round(float(total[0]), 12), bool(same))           # energy conserved; a film on
    air is the slab
```

Printed 1.0 True at this build, in 0.0 s.

Changelog of the method. 0.1.0.

P12 · A film on an opaque substrate: the coherent amplitude and two incoherent forms, FOCUS's and the energy-conserving one

Physics · optics

Statement. Air | film (ε , d) | opaque substrate (ε_{sub}). Coherent: $r = (r_1 + r_2 e^{2i}) / (1 + r_1 r_2 e^{2i})$. Incoherent, two forms: the energy-conserving one, $R = R_1 + (1 - R_1)^2 R_2 A / (1 - R_1 R_2 A)$ with $A = |e^{2i}|^2$, consistent with the slab (the default); and FOCUS's Reflectance-Ei-E1-E2 symbol, eq. 27 of Irati's report, $R = R_1 + |1 - r_1|^2 R_2 A / (1 - R_1 R_2 A)$, the product of amplitude transmittances, exact only when the film's index is real. They differ where the film absorbs strongly, up to 0.05 in the reststrahlen band of Cr_2O_3 ; the FOCUS form reproduces a FOCUS result exactly. $E = 1 - R$, no transmission.

$$R_{\text{energy}} = R_1 + \frac{(1 - R_1)^2 R_2 A}{1 - R_1 R_2 A}, \quad R_{\text{focus}} = R_1 + \frac{|1 - r_1|^2 R_2 A}{1 - R_1 R_2 A}, \quad A = |e^{2i\beta}|^2$$

Why this choice. Both forms, named, with the energy-conserving one as the default and FOCUS's available for exact reproduction, because the difference is physical and quantified rather than a bug on either side; the ledger records where it shows (T06, T28).

Error and its order. FOCUS's form violates energy conservation by up to 0.05 in R where k is large; the energy form is exact for incoherent addition.

Reference. Irati Tolosa's TFG report, eq. 27; FOCUS help, [osml-elements.html](#) (Reflectance-Ei-E1-E2); Heavens, Optical Properties of Thin Solid Films (1955) §4

Counterparts. FOCUS (inferred from numbers): Reflectance-Ei-E1-E2 met to seven digits with form = focus (T06). RefFIT (documented): RefFIT's multilayer model is coherent (manual §4.7.8); not compared term by term.

Implementation.

- `reflectivity_film(eps: np.ndarray, eps_sub: np.ndarray, omega: np.ndarray, thickness: float, coherent: bool = True, form: str = energy) -> np.ndarray` — [src/difes/optics.py:210](#)
- `emissivity_film(eps: np.ndarray, eps_sub: np.ndarray, omega: np.ndarray, thickness: float, coherent: bool = True, form: str = energy) -> np.ndarray` — [src/difes/optics.py:239](#)

Tests that name these symbols. `test_optics.py::test_film_on_substrate_limits`, `test_stack.py::test_one_la`

Ledger records. T02 (The optics against textbook values: Fresnel, slabs, films, energy balance); T06 (FOCUS's OSML files: quartz E, LiNbO_3 E, the four libraries, the round trip); T28 (Iñigo's thesis: the Pyromark coating, two minima of one model).

Check it yourself. The last printed line must be `True True`.

```
import numpy as np
from difes.optics import reflectivity_film
w = np.array([1000.0]); sub = np.array([5.0 + 1j])
clear = np.array([2.0 + 0j]); lossy = np.array([2.0 + 0.3j])
agree = np.allclose(reflectivity_film(clear, sub, w, 0.001, False, "energy"),
    ↪ reflectivity_film(clear, sub, w, 0.001, False, "focus"))
differ = not np.allclose(reflectivity_film(lossy, sub, w, 0.001, False, "energy"),
    ↪ reflectivity_film(lossy, sub, w, 0.001, False, "focus"))
print(bool(agree), bool(differ))                                # the two forms agree for a
    ↪ real index and not otherwise
```

Printed `True True` at this build, in 0.0 s.

Changelog of the method. 0.1.0: energy form; 0.8.0: FOCUS's form named and selectable.

P13 · Layer stacks on an opaque substrate: FOCUS's nested incoherent recursion, or the coherent Airy recursion

Physics · optics

Statement. Layers (ε_k , d_k) listed from the surface down. Incoherent: intensities combined layer by layer from the substrate up; with form = focus each layer k between the medium above it and the intensity reflectance below gives $R_k = |r_k|^2 + |(1 - r_k^2)P_k|^2 R_{\text{below}} / (1 - |r_k P_k|^2 R_{\text{below}})$, FOCUS's

Reflectance-Ei-E1-R12 symbol nested as in Irati's Azkena project (an effective-medium layer over an oxide); with form = energy the conserving form of P12. Coherent: the amplitude recursion from the substrate up. One layer reproduces the film exactly; a layer may be an effective-medium model with a fittable fill.

$$R_k = |r_k|^2 + \frac{|(1 - r_k^2) P_k|^2 R_{\text{below}}}{1 - |r_k P_k|^2 R_{\text{below}}}, \quad r_k = \frac{\sqrt{\varepsilon_{\text{above}}} - \sqrt{\varepsilon_k}}{\sqrt{\varepsilon_{\text{above}}} + \sqrt{\varepsilon_k}}, \quad P_k = e^{2i\beta_k}$$

Why this choice. The recursion in the order FOCUS nests its symbols, so that Irati's two-layer project evaluates to its digits; the energy form kept alongside for the same reason as in P12.

Error and its order. As P12's, per layer.

Reference. FOCUS help, osml-elements.html (Reflectance-Ei-E1-R12); Irati Tolosa's Azkena project (T06)

Counterparts. FOCUS (inferred from numbers): Irati's Azkena project, an effective-medium layer over Cr_2O_3 on chromium, met to seven digits (T06). RefFIT (documented): RefFIT's multilayer model (manual §4.7.8) is coherent only.

Implementation.

- `reflectivity_stack(layers: list[tuple[np.ndarray, float]], eps_sub: np.ndarray, omega: np.ndarray, coherent: bool = False, form: str = 'focus', eps_in: np.ndarray | complex = 1.0) -> np.ndarray — src/difes/optics.py:292`

Tests that name these symbols. `test_stack.py::test_one_layer_stack_is_the_film`.

Ledger records. T06 (FOCUS's OSML files: quartz E, LiNbO_3 E, the four libraries, the round trip).

Check it yourself. The last printed line must be True.

```
import numpy as np
from difes.optics import reflectivity_stack, reflectivity_film
eps = np.array([2.0 + 0.3j]); sub = np.array([5.0 + 1j]); w = np.array([1000.0])
print(bool(np.allclose(reflectivity_stack([(eps, 0.001)], sub, w, False, "focus"),
    ↪ reflectivity_film(eps, sub, w, 0.001, False, "focus"))))
```

Printed True at this build, in 0.0 s.

Changelog of the method. 0.10.0.

P14 · Fresnel coefficients at oblique incidence, p and s

Physics · optics

Statement. From air at the angle θ : $r_p = (q - \varepsilon \cos \theta) / (q + \varepsilon \cos \theta)$ and $r_s = (\cos \theta - q) / (\cos \theta + q)$ with $q = \sqrt{\varepsilon - \sin^2 \theta}$ on the branch with non-negative imaginary part, RefFIT's manual's form; at $\theta = 0$ both give the normal-incidence R. Used by the plate model (N14) and by FOCUS's oblique libraries through the formula language.

$$r_p = \frac{q - \varepsilon \cos \theta}{q + \varepsilon \cos \theta}, \quad r_s = \frac{\cos \theta - q}{\cos \theta + q}, \quad q = \sqrt{\varepsilon - \sin^2 \theta}$$

Why this choice. RefFIT's sign convention for r_s (the negative of P10's $\tilde{r}$ at $\theta = 0$, the same $|r|^2$), because the plate model and its ellipsometric angles were reproduced from RefFIT's manual with it; the branch of q is forced so that a wave decays into the medium.

Error and its order. None; closed forms.

Reference. RefFIT manual §4.7.9; Born and Wolf, Principles of Optics, §1.5

Counterparts. FOCUS (documented): FOCUS's oblique-incidence libraries (Reflectance-WN.sml and others) carry the same formulas; imported as functions (X01). RefFIT (inferred from numbers): RefFIT's plate at an angle met on its tutorial (T46).

Implementation.

- `fresnel_oblique(eps: np.ndarray, theta_deg: float) -> tuple[np.ndarray, np.ndarray, np.ndarray]` — `src/difes/optics.py:121`

Tests that name these symbols. `test_plate_refit.py::test_no_coherence_is_the_intensity_sum_and_meets_t`
`test_plate_refit.py::test_truncated_rays_converge_to_the_closed_forms`, `test_plate_refit.py::test_elli`

Ledger records. T02 (The optics against textbook values: Fresnel, slabs, films, energy balance); T46 (RefFIT's own tutorial, Part 2: four datasets fitted together, RefFIT's weighted χ^2 to the printed digit).

Check it yourself. The last printed line must be `True True`.

```
import numpy as np
from difes.optics import fresnel_oblique, reflectivity_normal
eps = np.array([2.0 + 0.3j])
rp, rs, q = fresnel_oblique(eps, 0.0)
print(bool(np.allclose(np.abs(rp) ** 2, reflectivity_normal(eps))),
      ↪ bool(np.allclose(np.abs(rs) ** 2, reflectivity_normal(eps))))
```

Printed `True True` at this build, in 0.0 s.

Changelog of the method. 0.24.0.

P15 · What is computed from what: the quantity dispatch and the simulated spectrum

Physics · dispatch

Statement. `quantity(model,  $\omega$ , kind, thickness, coherent, substrate, form, layers)` chooses the geometry from what is given: no thickness, an opaque bulk sample; a thickness alone, a free-standing slab (R, T, E); a thickness and a substrate, a film (R, E, no T); layers above it, a stack (R, E). The derived kinds ε' , ε'' , χ' , χ'' , n , k and θ come from the model alone. `simulate()` wraps the result as a `Spectrum` with the model and the geometry recorded in its metadata, so a simulated curve carries what made it.

$$y(\omega) = Q_{\text{kind}}(\varepsilon(\omega; \mathbf{p}); d, \varepsilon_{\text{sub}}, \text{layers})$$

Why this choice. One function for every measurable so that a fit term, a session node and the command line share the same path; impossible combinations (transmission through an opaque substrate, a stack without a thickness) are refused with a message rather than silently computed.

Error and its order. None; a dispatch.

Reference. This registry: P10 to P14

Counterparts. FOCUS (documented): FOCUS's definitions name their geometry explicitly (Reflectance-E, Reflectance-Ei-E1-E2); the dispatch reproduces each (T06, T15, T17). RefFIT (documented): RefFIT's output abbreviations (P17).

Implementation.

- `quantity(model: DielectricModel, omega: np.ndarray, kind: Kind | str, thickness: float | None = None, coherent: bool = True, substrate: DielectricModel | None = None, form: str = energy, layers: list[tuple[DielectricModel, float]] | None = None) -> np.ndarray` — `src/difes/optics.py:336`
- `simulate(model: DielectricModel, omega: np.ndarray, kind: Kind | str, thickness: float | None = None, coherent: bool = True, name: str = , substrate: DielectricModel | None = None, form: str = energy) -> Spectrum` — `src/difes/optics.py:395`

Tests that name these symbols. `test_bench.py::test_spectrum_model_sample_simulate_are_tagged_nodes`,
`test_bench.py::test_bench_reads_a_graph_built_session`, `test_bench.py::test_invert_returns_epsilon`,
`test_bench.py::test_layers_and_thicknesses_in_the_bench`, `test_cli.py::test_simulate_then_info_then_conv`
`test_cli.py::test_simulate_transmission_needs_thickness`, `test_cli.py::test_fit_command_recovers_paramet`

test_cli.py::test_kk_and_cfrc_commands, test_expr.py::test_expression_term_equals_builtin_lorentz_and_f
test_fit.py::test_analytic_jacobian_matches_finite_differences and 26 more.

Ledger records. T02 (The optics against textbook values: Fresnel, slabs, films, energy balance); T05 (The software around the physics: language, spectra, sessions, server, Bench).

Check it yourself. The last printed line must be `E 0.01 True`.

```
import numpy as np
from difes.models import example_model
from difes.optics import quantity, simulate
m = example_model(); w = np.linspace(100, 2000, 4)
s = simulate(m, w, "E", thickness=0.01, coherent=False) # an incoherent slab's
↳ emissivity
try:
    quantity(m, w, "T", thickness=0.01, substrate=example_model()); refused = False
except ValueError:
    refused = True
print(s.kind.value, s.meta["thickness_cm"], refused)
```

Printed E 0.01 True at this build, in 0.0 s.

Changelog of the method. 0.1.0; 0.10.0: layers.

P16 · The spectrum type and its units: wavenumbers in cm^{-1} , strictly ascending, bounded quantities clamped within a tolerance

Physics · conventions

Statement. A Spectrum holds one x axis in cm^{-1} , strictly ascending with no duplicates (sorted on construction, duplicates refused), one quantity of a declared kind (R, E, T, ε' , ε'' , n, k, θ , χ' , χ'' or raw), optional error bars, a temperature, a name and free metadata. Readers accept eV, THz, nm and μm and convert on the way in (wavelength units reverse the axis); 1 eV = 8065.543937 cm^{-1} (CODATA 2018), 1 THz = 33.35640952 cm^{-1} . R, E and T must lie in [0, 1]: an excursion beyond 0.01 is refused, a smaller one is clamped with a warning.

$$\tilde{\nu} [\text{cm}^{-1}] = 8065.543937 E [\text{eV}] = 33.35640952 f [\text{THz}] = \frac{10^7}{\lambda [\text{nm}]}$$

Why this choice. One internal unit so that every formula and every file agree, with the original unit kept in the metadata; the clamp tolerance of 0.01 lets a measured reflectance a hair above 1 through while a wrong column (a percentage) is refused.

Error and its order. The conversion constants to ten significant digits; the clamp is a change of the data of at most 0.01, stated in a warning.

Reference. CODATA 2018 (the electron volt in wavenumbers)

Counterparts. FOCUS (documented): FOCUS reads data files as given, in the unit of the abscissa variable (help, data-files.html). RefFIT (documented): RefFIT's datasets are X Y Yerr in cm^{-1} (manual §4.4).

Implementation.

- Spectrum(x: np.ndarray, y: np.ndarray, kind: Kind = <Kind.RAW: raw>, yerr: np.ndarray | None = None, temperature: float | None = None, name: str = , meta: dict[str, Any] = <factory>) -> None — src/difes/spectra.py:88
- Kind(*values) — src/difes/spectra.py:25
- Unit(*values) — src/difes/spectra.py:43
- to_cm1(x: np.ndarray, unit: Unit) -> np.ndarray — src/difes/spectra.py:52
- from_cm1(x_cm1: np.ndarray, unit: Unit) -> np.ndarray — src/difes/spectra.py:68
- SpectrumError — src/difes/spectra.py:84

Library routines called. `numpy.interp` (D07)

Tests that name these symbols. `test_bench.py::test_layers_and_thicknesses_in_the_bench`, `test_expr.py::test_expression_term_equals_builtin_lorentz_and_fits`, `test_fit.py::test_analytic_jacobian`, `test_fit.py::test_weighted_joint_fit_of_R_and_E`, `test_graph.py::test_evaluation_and_addressable_outputs`, `test_graph.py::test_expr_uses_libraries`, `test_kk.py::test_cfrf_needs_anchors_or_constraint`, `test_kk.py::test_spectrum_wrappers`, `test_reffit.py::test_par_dat_and_protocol_files`, `test_reffit_output.py::test_difes_fitter_lands_where_reffits_did_on_reffits_own_data` and 21 more.

Ledger records. T05 (The software around the physics: language, spectra, sessions, server, Bench).

Check it yourself. The last printed line must be `[8065.543937] [10000. 5000.] True`.

```
import numpy as np
from difes.spectra import Spectrum, Kind, Unit, to_cm1, SpectrumError
try:
    Spectrum(np.array([1.0, 2.0]), np.array([0.5, 1.2]), kind=Kind.R); refused = False
    ↪ # 1.2 is not a reflectance
except SpectrumError:
    refused = True
print(to_cm1(np.array([1.0]), Unit.EV), to_cm1(np.array([1000.0, 2000.0]), Unit.NM),
    ↪ refused)
```

Printed `[8065.543937] [10000. 5000.] True` at this build, in 0.0 s.

Changelog of the method. 0.1.0.

P17 · RefFIT's output quantities and its extended Drude analysis

Physics · derived-quantities

Statement. From ε on a grid, RefFIT's abbreviations: R, Rph (the phase), E1, E2, $S1 = \omega\varepsilon''/60$ and $S2 = -\omega(\varepsilon' - 1)/60$ in $\Omega^{-1}\text{cm}^{-1}$ (the conductivity in cm^{-1} times $/15$), N1, N2, $PD = 1/(2k)$ the penetration depth in cm, $LF = \varepsilon''/|\varepsilon|^2$ the loss function, $AC = 4k$ the absorption coefficient in cm^{-1} . The extended Drude outputs (code -6): $1/\tau(\omega) = -(\omega_p^2/\omega) \text{Im}[1/(\varepsilon - \varepsilon_\infty)]$ and $m^*/m = -(\omega_p^2/\omega^2) \text{Re}[1/(\varepsilon - \varepsilon_\infty)]$; for a pure Drude term they are γ and 1 at every frequency, the identity the check uses. RefFIT's plate model is N14.

$$\sigma_1 = \frac{\omega\varepsilon''}{60} [\Omega^{-1}\text{cm}^{-1}], \quad \frac{1}{\tau(\omega)} = -\frac{\omega_p^2}{\omega} \text{Im} \frac{1}{\varepsilon - \varepsilon_\infty}, \quad \frac{m^*}{m}(\omega) = -\frac{\omega_p^2}{\omega^2} \text{Re} \frac{1}{\varepsilon - \varepsilon_\infty}$$

Why this choice. RefFIT's units and names so that its exported curves compare without conversion (T34, T43); the factor 60 is RefFIT's own (the manual's $/15$ on σ in cm^{-1}).

Error and its order. None; definitions.

Reference. RefFIT manual Table 4-2 and §4.7.6; Dressel and Grüner, *Electrodynamics of Solids* (2002) §12.1 (the extended Drude model)

Counterparts. FOCUS (documented): FOCUS has no built-in conductivity output; the formula language gives it. RefFIT (inferred from numbers): Met on RefFIT's exports (T34, T43).

Implementation.

- `quantity_reffit(abbrev: str, eps: np.ndarray, omega: np.ndarray) -> np.ndarray` — `src/difes/reffit.py:320`
- `extended_drude(eps: np.ndarray, omega: np.ndarray, wp: float, eps_inf: float) -> dict[str, np.ndarray]` — `src/difes/reffit.py:352`
- `plate(model: DielectricModel, omega: np.ndarray, thickness: float, theta_deg: float = 0.0, coherence: float = 0.0, rays: int | None = None, polarizer_deg: float = 0.0, scale: float = 1.0) -> dict[str, np.ndarray]` — `src/difes/reffit.py:430`

Tests that name these symbols. `test_plate_reffit.py::test_reffit_plate_wrapper_and_the_polarizer`, `test_reffit.py::test_quantities_follow_their_definitions`, `test_reffit.py::test_extended_drude_of_a_drude`, `test_reffit_output.py::test_reffits_weighted_chi_square_and_its_log_grid_reproduce_fig_3_25`.

Ledger records. T34 (RefFIT's own output: the comparison protocol run by Irati, number against number); T43 (Irati's own RefFIT fits of polished chromium (200 and 500 °C): her models, RefFIT's curves, her data).

Check it yourself. The last printed line must be `[150.0, 150.0, 150.0, 150.0, 150.0] [1.0, 1.0, 1.0, 1.0, 1.0] [10.]`.

```
import numpy as np
from difes.models import DielectricModel, Drude
from difes.reffit import extended_drude, quantity_reffit
w = np.linspace(100, 2000, 5)
ed = extended_drude(DielectricModel(eps_inf=3.0, terms=[Drude(wp=2000.0,
    ↪ gamma=150.0)]).epsilon(w), w, 2000.0, 3.0)
print(np.round(ed["tau_inv"], 6).tolist(), np.round(ed["mass_ratio"], 6).tolist(),
    ↪ quantity_reffit("S1", np.array([3.0 + 6.0j]), np.array([100.0])))
```

Printed `[150.0, 150.0, 150.0, 150.0, 150.0] [1.0, 1.0, 1.0, 1.0, 1.0] [10.]` at this build, in 0.0 s.

Changelog of the method. 0.24.0.

Language

the formula grammar, its safe evaluation, user functions, FOCUS's profiles in it, libraries, sessions and views.

L01 · The grammar of the math language: a recursive-descent parser over FOCUS's expression syntax

Language · parser

Statement. A tokenizer (numbers with exponents and a *j* suffix, identifiers, the operators `+` `-` `*` `/` `^` `**` and the comparisons) and a recursive-descent parser with the precedence comparison `< sum < product < unary < power < atom`; power is right-associative and binds tighter than a leading minus, so `232` is 512 and `-22` is -4, as in FOCUS and in Python. Calls are `identifier(args)`; the tree is five node kinds (number, name, unary, binary, call), immutable, and can be printed back as source fully parenthesised.

`expr := cmp ((== | < | > | <= | >=) cmp)*,   cmp := term ((+|-) term)*,   term := unary ((*|/) unary)*,   unary := (-|`

Why this choice. A hand-written parser rather than a library, so that the grammar is the dozen lines above and nothing else, and so that a formula can be inlined, substituted and unparsed for the OSML export.

Error and its order. None; a definition. Identifiers are case-sensitive, function names are looked up case-insensitively (FOCUS writes `Sqrt`, Python `sqrt`).

Reference. FOCUS help, formula-editor.html; Aho, Lam, Sethi and Ullman, *Compilers* (2006) §4.4

Counterparts. FOCUS (documented): FOCUS's formula editor (help, formula-editor.html) with its precedence; the same formulas round-trip (T05, T15). RefFIT (documented): RefFIT has no formula language; its special functions are codes (X06).

Implementation.

- `tokenize(src: str) -> list[tuple[str, str]]` — `src/difes/expr.py:254`
- `parse(src: str)` — `src/difes/expr.py:362`
- `Num(value: complex | float) -> None` — `src/difes/expr.py:267`
- `Name(name: str) -> None` — `src/difes/expr.py:271`

- Unary(op: str, arg: Any) -> None — src/difes/expr.py:275
- Binary(op: str, left: Any, right: Any) -> None — src/difes/expr.py:280
- Call(name: str, args: tuple) -> None — src/difes/expr.py:286
- ExprError — src/difes/expr.py:250

Tests that name these symbols. test_bergman.py::test_mireia_osml_read_as_document_with_its_tables, test_expr.py::test_focus_formula_editor_examples, test_expr.py::test_user_function_defaults_keywords_and, test_expr.py::test_errors, test_osml.py::test_focus_libraries_become_formulas_that_match_difes_optics, test_osml.py::test_library_export_import, test_osml.py::test_focus_projects_of_irati_read_and_evaluate, test_osml.py::test_focus_exported_curves_match_to_file_precision, test_recipe.py::test_parse_bare_and_m, test_recipe.py::test_comment_rules_keep_colours and 5 more.

Ledger records. T05 (The software around the physics: language, spectra, sessions, server, Bench).

Check it yourself. The last printed line must be 512.0 -4.0 1.0.

```
from difes.expr import evaluate_formula
print(evaluate_formula("2^3^2"), evaluate_formula("-2^2"), evaluate_formula("3 > 2")) #
↳ right-associative power, unary minus below it, comparisons as 1.0/0.0
```

Printed 512.0 -4.0 1.0 at this build, in 0.0 s.

Changelog of the method. 0.5.0.

L02 · Evaluation without eval: a tree walk over numpy with a closed table of functions

Language · evaluation

Statement. A tree is evaluated against an environment of names (arrays or scalars) and a table of callables with arities: FOCUS's names first (Real, Imag, Abs, Arg, Norm, Conjg, Sqr, Sqrt, Exp, Ln, Log as base 10, the trigonometric and hyperbolic families, Pow, Min, Max, If, Heaviside, Clip, Sum, Product, Dawson) and the Python aliases after; the constants PI, E, LN2, LN10, I; FOCUS's absorption profiles and peak functions (L04); the point-list functions (N10). There is no eval and no access to Python names, so a formula from a file or a library is safe to run. Unknown names and wrong arities raise with the name. Comparisons give 1.0 or 0.0 and If selects elementwise.

$$\text{eval}(\text{node}, \text{env}) \in \{\text{numbers, numpy arrays}\}, \quad \text{functions} = \text{a fixed table of } (f, \text{arity})$$

Why this choice. Safety and portability over speed: a shared library of formulas (L05) must not be able to run code, and a formula must evaluate the same on the command line, in a session and in the shell; numpy vectorisation keeps a 2000-point evaluation at a millisecond.

Error and its order. None of its own; the functions' (SciPy's special functions, N11).

Reference. FOCUS help, core-functions.htm (the function table)

Counterparts. FOCUS (documented): FOCUS's core function table (help, core-functions.htm); Log is base 10 as there. RefFIT (documented): Not applicable.

Implementation.

- evaluate(node, env: dict[str, Any], functions: "dict[str, UserFunction] | None" = None) — src/difes/expr.py:405
- evaluate_formula(src: str, **env: Any) — src/difes/expr.py:487

Library routines called. numpy.random.random (legacy global generator) (D02)

Tests that name these symbols. test_bergman.py::test_fritsch_carlson_interpolates_and_stays_monotone, test_bergman.py::test_mireia_osml_read_as_document_with_its_tables, test_expr.py::test_precedence_and_p, test_expr.py::test_complex_and_constants, test_expr.py::test_focus_formula_editor_examples,

test_expr.py::test_vectorised_and_comparisons, test_expr.py::test_errors, test_fit.py::test_analytic_jac
test_osml.py::test_focus_peak_profiles_and_anharmonic_phonons, test_osml.py::test_focus_projects_of_ira
and 1 more.

Ledger records. T05 (The software around the physics: language, spectra, sessions, server, Bench); T15 (FOCUS's sample projects: the spectra read from the binary .pjt files).

Check it yourself. The last printed line must be 3.0 20.0 True.

```
from difes.expr import evaluate_formula, ExprError
try:
    evaluate_formula("__import__('os')"); safe = False
except ExprError:
    safe = True
print(evaluate_formula("Log(1000)"), evaluate_formula("If(x > 1, 10, 20)", x=0.5), safe)
↪ # FOCUS's base-10 Log; If; no Python names
```

Printed 3.0 20.0 True at this build, in 0.0 s.

Changelog of the method. 0.5.0; 0.31.0: the point-list functions.

L03 · User functions: name(args) = body, with defaults, a free-name check, substitution and unparsing

Language · functions

Statement. A definition is parsed into a name, its argument list and a body tree; any name in the body that is neither an argument nor a constant is refused at parse time, so a library entry cannot depend on a hidden variable. Defaults may be attached to arguments (FOCUS's argument numbers). Calling binds the arguments positionally or by name and may pass a table of other user functions, so definitions call each other; substitute() inlines one body into another and unparse() writes a tree back as source, the two steps the OSML export uses to produce a single formula FOCUS can hold.

$$f(x, a, b) = \text{body}(x, a, b), \quad \text{free}(\text{body}) \setminus \{x, a, b\} \setminus \text{constants} = \emptyset$$

Why this choice. The free-name check is what makes a formula portable between a session, the app library and a FOCUS file; the inlining exists because FOCUS's formula symbols cannot call each other by name in every context, and a flat formula always can.

Error and its order. None; a definition.

Reference. FOCUS help, formula-editor.html and library-creation.html

Counterparts. FOCUS (documented): FOCUS's formula symbols with arguments and defaults (help, library-creation.html); read and written in X01 and X03. ReFIT (documented): Not applicable.

Implementation.

- UserFunction(name: str, args: tuple[str, ...], body: Any, source: str, defaults: dict[str, float] = <factory>) -> None — src/difes/expr.py:444
- names_in(node) -> set[str] — src/difes/expr.py:367
- substitute(node, mapping: dict[str, Any]) — src/difes/expr.py:375
- unparse(node) -> str — src/difes/expr.py:384

Tests that name these symbols. test_bergman.py::test_mireia_osml_read_as_document_with_its_tables,
test_expr.py::test_focus_formula_editor_examples, test_expr.py::test_user_function_defaults_keywords_an
test_expr.py::test_errors, test_osml.py::test_focus_libraries_become_formulas_that_match_difes_optics,
test_osml.py::test_library_export_import, test_osml.py::test_focus_projects_of_irati_read_and_evaluate,
test_osml.py::test_focus_exported_curves_match_to_file_precision, test_stack.py::test_bruggeman_is_focu

Ledger records. T05 (The software around the physics: language, spectra, sessions, server, Bench); T15 (FOCUS's sample projects: the spectra read from the binary .pjt files).

Check it yourself. The last printed line must be 18.0 ('x', 'a') (a * (x ^ 2)) True.

```

from difes.expr import UserFunction, ExprError, parse, unparse
f = UserFunction.parse("sq(x, a) = a*x^2")
try:
    UserFunction.parse("bad(x) = a*x"); refused = False           # a is neither an
    ↪ argument nor a constant
except ExprError:
    refused = True
print(f(3.0, 2.0), f.args, unparse(parse("a*x^2")), refused)

```

Printed 18.0 ('x', 'a') (a * (x ^ 2)) True at this build, in 0.0 s.

Changelog of the method. 0.5.0.

L04 · FOCUS's absorption profiles and peak functions as language functions, with the sign convention made explicit

Language · profiles

Statement. In the language, in FOCUS's argument order: `phonon_4p(x, wTO, gTO, wLO, gLO)`, `phonon_3p(x, dEps, w0, g)`, `drude(x, wp, g)`, the semi-quantum phonons with a self-energy expression `phonon_anharmonic(x, dEps, W, P) = dEps W2/(W2 - x2 + P)` and `phonon_anharmonic_i(x, dEps, W, G) = dEps W2/(W2 - x2 - iG)`, and the real peak functions `gaussian(x, A, Xc, w)` and `lorentzian(x, A, Xc, w)` with `w` the FWHM, `voigt(N11)`, and the causal bands `gaussian_kkg` and `voigt_kkg`. FOCUS writes $+iG$ with its e^{+it} convention; difes writes $-iG$ so that $\varepsilon'' \geq 0$ with e^{-it} , the same $|\tilde{r}|^2$. Profiles whose definition exists in FOCUS's help only as an image (`lorentzian-kkg`, `pearsonVII`, `plasmon`, `debye` and others) are not implemented and the OSML reader names them when it meets them (X01).

$$\text{phonon_anharmonic_i}(x) = \frac{\Delta\varepsilon W^2}{W^2 - x^2 - iG(x)} \text{ (difes, } e^{-i\omega t}) \quad \leftrightarrow \quad \frac{\Delta\varepsilon W^2}{W^2 - x^2 + iG(x)} \text{ (FOCUS, } e^{+i\omega t})$$

Why this choice. The sign is flipped at the boundary, once, rather than carrying FOCUS's convention inside difes, because every other part of the program assumes $\varepsilon'' \geq 0$; the reflectance is the same either way, which is what FOCUS's exports check.

Error and its order. None; closed forms. The undocumented profiles are a stated gap, not an approximation.

Reference. FOCUS help, `core-functions.htm`; De Sousa Meneses et al., J. Non-Cryst. Solids 351 (2005) 124 (the anharmonic phonon)

Counterparts. FOCUS (inferred from numbers): The documented profiles met on FOCUS's exports (T15, T17, T20); the undocumented ones listed as blocked (T29). RefFIT (documented): Not applicable.

Implementation.

- `evaluate_formula(src: str, **env: Any)` — `src/difes/expr.py:487`

Tests that name these symbols. `test_bergman.py::test_fritsch_carlson_interpolates_and_stays_monotone`, `test_expr.py::test_precedence_and_power`, `test_expr.py::test_complex_and_constants`, `test_expr.py::test_voigt`, `test_osml.py::test_focus_peak_profiles_and_anharmonic_phonons`.

Ledger records. T15 (FOCUS's sample projects: the spectra read from the binary .pjt files); T17 (The phosphate glass: twelve causal Gaussians against FOCUS's export); T29 (Papers that cannot be reproduced yet).

Check it yourself. The last printed line must be `(2.5+0j) True`.

```
import numpy as np
from difes.expr import evaluate_formula
static = evaluate_formula("phonon_3p(x, 2.5, 400, 10)", x=0.0) #
↳ the strength at x = 0
im = np.imag(evaluate_formula("phonon_anharmonic_i(x, 2.5, 400, 500)", x=np.array([300.0,
↳ 400.0])))
print(static, bool(np.all(im >= 0))) #
↳ eps'' >= 0 in difes's convention
```

Printed (2.5+0j) True at this build, in 0.0 s.

Changelog of the method. 0.5.0; 0.12.0: voigt_kkg; 0.31.0: the point-list functions.

L05 · Libraries at three scopes, with provenance and promotion

Language · libraries

Statement. Functions, terms and views live in libraries of three scopes: built-in (ships with difes: FOCUS's formula-editor examples, its Fresnel reflectivity, a Planck function, a Lorentz and a Drude term as worked examples), app (the group's, one JSON file, later a package in the Guix channel) and project (next to the session); a session also carries its own entries (a FOCUS project's functions). Lookup is project, app, built-in, innermost winning. Every entry records its provenance (author, date, source, note, validated); promotion copies an entry to a wider scope and records from where, when, by whom and why. A function entry is parsed on insertion (L03), so a library never holds a formula that does not parse.

$\text{lookup}(\text{name}) = \text{project} \succ \text{app} \succ \text{builtin}$

Why this choice. Provenance on every entry because the group's formulas come from FOCUS files, papers and colleagues and a formula without its source cannot be audited; promotion as a recorded act so that the app library grows by decisions, not by drift.

Error and its order. None; bookkeeping.

Reference. FOCUS help, library-creation.html (FOCUS's libraries, the model)

Counterparts. FOCUS (documented): FOCUS's user libraries (.sml) are read and written as library entries (X01, X03). RefFIT (documented): Not applicable.

Implementation.

- `Library(scope: str, entries: dict[str, dict[str, Any]] = <factory>, path: Path | None = None) -> None` — `src/difes/library.py:46`
- `Libraries(project_path: Path | None = None, app_path: Path | None = None) -> None` — `src/difes/library.py:81`
- `app_library_path() -> Path` — `src/difes/library.py:41`

Tests that name these symbols. `test_graph.py::test_library_scopes_and_promotion`, `test_osml.py::test_library`, `test_recipe_export.py::test_recipes_in_the_library`.

Ledger records. T05 (The software around the physics: language, spectra, sessions, server, Bench).

Check it yourself. The last printed line must be builtin function `['Gauss', 'Lor', 'Planck_WN']` `['lorentz_term', 'drude_term']`.

```
from difes.library import Libraries
L = Libraries()
scope, entry = L.get("Planck_WN")
print(scope, entry["kind"], sorted(L.functions())[:3], [n for s, n, k in
↳ L.names("term")])
```

Printed builtin function ['Gauss', 'Lor', 'Planck_WN'] ['lorentz_term', 'drude_term'] at this build, in 0.0 s.

Changelog of the method. 0.5.0.

L06 · The session graph and the views: named nodes, references, lazy cached evaluation, declarative plots

Language · sessions

Statement. A session is a set of named nodes of nine kinds (data, model, sample, quantity, expr, transform, fit, view, uncertainty) whose specs reference other nodes' outputs as @node or @node:output; the graph must be acyclic and a reference to an unknown node is refused. Evaluation is pure and lazy: an output depends only on the node's spec and its inputs' outputs, every node carries a version that grows on change, and a cached output is reused while the signature (own version plus the inputs' signatures) is unchanged. Every output is addressable, including one term of a model. A view is a declarative record that never holds data: it names node outputs as curves with a part (real, imaginary, both, modulus, real versus imaginary) and styles, rendered by the core for export or handed to a shell as arrays. Sessions and views are JSON.

$$\text{out}(n) = F_{\text{kind}}(\text{spec}(n), \{\text{out}(m) : m \in \text{deps}(n)\}), \quad \text{sig}(n) = (\text{version}(n), \{\text{sig}(m)\})$$

Why this choice. A graph rather than a script so that a shell can edit one parameter and recompute only what depends on it, and so that a session file is a complete record of how every curve was made; views separate from data so that the same plot specification applies to any session.

Error and its order. None; a definition.

Reference. Design pages of the project site: three levels of abstraction, two skins one graph

Counterparts. FOCUS (documented): FOCUS's project (math explorer, definitions, curves) imports as a session (X04). RefFIT (documented): RefFIT's scripts and macros play the role of a session (X07).

Implementation.

- `Session(base_dir: str | Path | None = None, libraries: Libraries | None = None) -> None` — `src/difes/graph.py:82`
- `Node(name: str, kind: str, spec: dict[str, Any] = <factory>, version: int = 1) -> None` — `src/difes/graph.py:67`
- `GraphError` — `src/difes/graph.py:44`
- `ViewSpec(title: str = , x: dict[str, Any] = <factory>, y: dict[str, Any] = <factory>, curves: list[Curve] = <factory>, size: tuple[float, float] = (8.0, 4.5), legend: bool = True) -> None` — `src/difes/views.py:45`
- `Curve(source: str, part: str = real, label: str | None = None, style: dict[str, Any] = <factory>, hidden: bool = False) -> None` — `src/difes/views.py:28`
- `curve_data(spec: ViewSpec, resolve) -> list[dict[str, Any]]` — `src/difes/views.py:72`
- `render(spec: ViewSpec, resolve, path: str | Path, dpi: int = 130) -> Path` — `src/difes/views.py:104`

Library routines called. `numpy.gradient` (D07); `numpy.interp` (D07)

Tests that name these symbols. `test_bench.py::test_bench_reads_a_graph_built_session`, `test_bench.py::test_layers_and_thicknesses_in_the_bench`, `test_bench.py::test_the_shared_chromium_session`, `test_bench.py::test_every_session_bundle_opens_with_relative_paths`, `test_focus_exact.py::test_single_ba`, `test_graph.py::test_view_data_and_render`, `test_graph.py::test_json_round_trip`, `test_graph.py::test_graph`, `test_osml.py::test_focus_projects_of_irati_read_and_evaluate`, `test_osml.py::test_focus_film_form_equals` and 14 more.

Ledger records. T05 (The software around the physics: language, spectra, sessions, server, Bench); T36 (The budget through the Kramers–Kronig inversion, and in a session).

Check it yourself. The last printed line must be `Spectrum 5 True`.

```
from difes.graph import Session, GraphError
from difes.models import DielectricModel, Lorentz
s = Session()
s.add("m", "model", model=DielectricModel(eps_inf=3.0, terms=[Lorentz(w0=450.0, wp=600.0,
↵ gamma=20.0)]).to_dict())
s.add("smp", "sample", model="@m")
```

```
s.add("R", "quantity", sample="@smp", kind="R", grid=[100, 2000, 5])
try:
    s.add("bad", "quantity", sample="@nope", kind="R", grid=[1, 2, 3]); refused = False
except GraphError:
    refused = True
print(type(s.get("R")).__name__, len(s.get("R")), refused)
```

Printed Spectrum 5 True at this build, in 0.0 s.

Changelog of the method. 0.5.0; 0.20.0: the uncertainty node.

Exchange

OSML in and out, FOCUS's binary project, RefFIT's table and files, text tables, receiving a folder, digitising figures.

X01 · Reading OSML: FOCUS's symbols and definitions as formulas

Exchange · osml-read

Statement. An OSML document (a FOCUS library or project) is an XML tree of symbols (functions with arguments and local sub-definitions, constants, or formula-encoded symbols) and a math block of definitions. Every symbol becomes a formula of the math language: FOCUS names with hyphens become identifiers with underscores, local sub-definitions are inlined on reference, the core functions are mapped by a table (binary and unary operators, sums and products, the profiles of L04, comparisons, if-else, the point-list functions of N10), a point list is flattened into the argument list, a file-2c table is read from the project's folder and inlined, and r-integrator over a point-list density becomes its closed-form integral. Every definition of the math block becomes a function of its variable, found by following its links (x by default, z for a spectral density). A core function the language does not have raises an error naming it; a definition of two variables is skipped with a note.

$$\text{OSML tree} \rightarrow \text{formula text} : \quad \langle \text{apply} \rangle(f, a_1 \dots a_n) \mapsto T[f](a_1, \dots, a_n)$$

Why this choice. Formulas rather than a parallel object model, so that a FOCUS project is readable, editable and callable in difes with nothing hidden; the unsupported list is explicit so that the reader stops and says what it met rather than guessing a profile whose definition exists only as an image in FOCUS's help.

Error and its order. None; a translation, checked by evaluation against FOCUS's exports on every project in hand (T06, T15, T48).

Reference. FOCUS help, osml.html, osml-core.html, osml-elements.html; the OSML schema (OSML.xsd)

Counterparts. FOCUS (documented): OSML is FOCUS's own documented format; its libraries and examples read and evaluate to FOCUS's digits (T15, T19). RefFIT (documented): Not applicable.

Implementation.

- `read_functions(path: str | Path) -> dict[str, dict[str, Any]]` — `src/difes/osml.py:235`
- `read_document(path: str | Path, variable: str = x) -> dict[str, Any]` — `src/difes/osml.py:275`
- `OsmlError` — `src/difes/osml.py:65`

Tests that name these symbols. `test_bergman.py::test_mireia_osml_read_as_document_with_its_tables`, `test_osml.py::test_focus_libraries_become_formulas_that_match_difes_optics`, `test_osml.py::test_unsupported`, `test_osml.py::test_library_export_import`, `test_osml.py::test_focus_projects_of_irati_read_and_evaluate`, `test_refit.py::test_fano_exports_to_osml_and_reads_back`, `test_stack.py::test_bruggeman_is_focus_eeff_b`

Ledger records. T06 (FOCUS's OSML files: quartz E, LiNbO₃ E, the four libraries, the round trip); T15 (FOCUS's sample projects: the spectra read from the binary .pjt files); T19 (SnO₂ Raman: four Voigt peaks)

on a cubic-spline base line); T48 (Mireia's alumina project: her Bergman model read from the OSML export, FOCUS's own curve met to 4e-6, her two complaints explained).

Check it yourself. The last printed line must be `['Eeff_BG', 'Reflectance_Ei_E1_E2'] 10 5`.

```
from pathlib import Path
import difes
from difes.osml import read_functions, read_document
ROOT = Path(difes.__file__).resolve().parents[2] # the difes repository
azk = ROOT / "examples/data/focus_projects/Azkena.sml" # Irati's two-layer
    ↪ chromium project
fs = read_functions(azk); doc = read_document(azk)
print(sorted(fs)[:2], len(doc["definitions"]), len(doc["models"]))
```

Printed `['Eeff_BG', 'Reflectance_Ei_E1_E2'] 10 5` at this build, in 0.02 s.

Changelog of the method. 0.6.0; 0.11.0: whole documents; 0.31.0: file-2c, point lists, r-integrator.

X02 · Reading a FOCUS dielectric function as a difes model, term by term

Exchange · osml-read

Statement. The definition holding ε (named E, Epsilon or eps, else the first product or sum) is taken apart: a product's numeric factor is ε_∞ and each phonon-4p becomes a four-parameter mode; a sum's numeric term adds to ε_∞ and each phonon-3p becomes a Lorentz term in strength form, each drude a Drude term, each formula-encoded document symbol called with numeric arguments an expression term with those arguments as parameters, and each core profile whose arguments are expressions an expression term built from them. Whatever remains in a product or a sum is inlined into one expression term (osml_factor or osml_rest), so nothing is dropped; a profile the language does not have raises an error naming it.

$$\varepsilon_{\text{OSML}} = \text{product}(c, \text{phonon-4p}, \dots) + \text{sum}(\dots) \mapsto \varepsilon_\infty \prod (\text{modes}) + \sum (\text{terms})$$

Why this choice. Native terms where FOCUS's profile has one (so that bounds, derivatives and the RefFIT mapping apply) and an expression term for everything else, rather than refusing anything FOCUS can write; the leftovers rule is what lets Mireia's Bergman model, built from point lists and helpers, evaluate to FOCUS's digits.

Error and its order. None; a translation. The strength form of phonon-3p and the drude sign were assumed from FOCUS's help and confirmed against its exports (T06, T15).

Reference. FOCUS help, osml-core.html; this registry, P01 to P05

Counterparts. FOCUS (inferred from numbers): FOCUS's quartz, LiNbO₃, phosphate and Raman samples import and round-trip exactly (T15, T16, T17). RefFIT (documented): RefFIT's table is read by X06.

Implementation.

- `read_model(path: str | Path, definition: str | None = None, variable: str = x) -> DielectricModel` — `src/difes/osml.py:449`

Tests that name these symbols. `test_focus_exact.py::test_quartz_a2_four_parameter_model_meets_focus`, `test_focus_exact.py::test_phosphate_glass_causal_gaussians_meet_focus`, `test_focus_exact.py::test_borofl`, `test_osml.py::test_read_quartz_e_model`, `test_osml.py::test_quartz_model_round_trip_through_osml`, `test_osml.py::test_unsupported_profile_is_reported_by_name`, `test_osml.py::test_write_and_read_mixed_mod`, `test_osml.py::test_silica_example_imports_as_fittable_terms_and_is_physical`, `test_reffit.py::test_fano`

Ledger records. T06 (FOCUS's OSML files: quartz E, LiNbO₃ E, the four libraries, the round trip); T15 (FOCUS's sample projects: the spectra read from the binary .pjt files); T16 (Quartz A2: five four-parameter modes against FOCUS's exported curve); T17 (The phosphate glass: twelve causal Gaussians against FOCUS's export); T48 (Mireia's alumina project: her Bergman model read from the OSML export, FOCUS's own curve met to 4e-6, her two complaints explained).

Check it yourself. The last printed line must be `8 {'fpsq'} 2.38584`.

```

from pathlib import Path
import difes
from difes.osml import read_model
ROOT = Path(difes.__file__).resolve().parents[2]
q = read_model(ROOT / "examples/output/R-Quartz-E_roundtrip.sml")      # FOCUS's quartz-E
    ↪ example, re-exported by difes
print(len(q.terms), {t.kind for t in q.terms}, q.eps_inf.value)

```

Printed 8 {'fpsq'} 2.38584 at this build, in 0.0 s.

Changelog of the method. 0.6.0; 0.11.0: formula symbols with numeric arguments; 0.31.0: leftovers inlined.

X03 · Writing OSML: a difes model as a FOCUS model document, with its reflectivity

Exchange · osml-write

Statement. Built-in terms are written as FOCUS's core functions (phonon-4p in a product on ε_∞ , drude and phonon-3p in strength form in a sum); the causal Gaussian, an expression term and the Fano term are written as formula-encoded symbols and called from the sum or the product; a Reflectivity-E symbol and a Reflectivity definition are added so that the file opens in FOCUS and draws. The Tauc–Lorentz real part and the variational function have no formula FOCUS can hold and are refused with a message (export ε as a curve instead). A library of difes functions is written as formula symbols with their provenance as annotations. FOCUS's quartz-E and LiNbO₃-E examples import and export unchanged.

$$\varepsilon_\infty \prod (\text{modes}) + \sum (\text{terms}) \mapsto \text{sum}(\text{product}(\varepsilon_\infty, \text{phonon-4p} \dots), \text{drude}, \text{phonon-3p}, \text{symbol}(\dots))$$

Why this choice. FOCUS's own export layout, so that a colleague opens the file without knowing difes exists; formula symbols for the rest because they are the one mechanism FOCUS has for a profile it does not ship.

Error and its order. Found while writing this card (2026-10-08): a difes model that mixes factorised modes with additive terms is written as `sum(product( $\varepsilon_\infty$ , modes), terms)` and read back with the product inlined as one expression term, numerically identical but structurally different; FOCUS-style models (all modes, or all additive terms) round-trip exactly. Recorded as a known limitation of the reader's leftovers rule.

Reference. FOCUS help, `osml.html` (File > Export Model)

Counterparts. FOCUS (inferred from numbers): difes's export of FOCUS's quartz example re-imports to FOCUS's digits (T15). RefFIT (documented): The RefFIT side is X06 and X07.

Implementation.

- `write_model(model: DielectricModel, path: str | Path | None = None, variable: str = x, definition: str = E, with_reflectivity: bool = True) -> str` — `src/difes/osml.py:543`
- `write_functions(entries: dict[str, dict[str, Any]], path: str | Path | None = None) -> str` — `src/difes/osml.py:600`

Tests that name these symbols. `test_osml.py::test_quartz_model_round_trip_through_osml`, `test_osml.py::test_write_and_read_mixed_model`, `test_osml.py::test_library_export_import`, `test_refit.py::test_fano_exports_to_osml_and_reads_back`.

Ledger records. T15 (FOCUS's sample projects: the spectra read from the binary .pjt files).

Check it yourself. The last printed line must be `True ['drude', 'lorentz', 'expr'] True`.

```

import numpy as np, tempfile, os
from difes.models import DielectricModel, FourParameterMode, Drude, Lorentz
from difes.osml import write_model, read_model
w = np.linspace(100, 2000, 5)

```

```

m = DielectricModel(eps_inf=2.0, terms=[FourParameterMode(w_to=400.0, g_to=10.0,
↳ w_lo=600.0, g_lo=12.0), Drude(wp=2000.0, gamma=150.0), Lorentz(w0=450.0, wp=600.0,
↳ gamma=20.0)])
p = os.path.join(tempfile.mkdtemp(), "mixed.sml"); doc = write_model(m, p)
back = read_model(p)
print("phonon-4p" in doc and "drude" in doc, [t.kind for t in back.terms],
↳ bool(np.allclose(back.epsilon(w), m.epsilon(w))))

```

Printed True ['drude', 'lorentz', 'expr'] True at this build, in 0.0 s.

Changelog of the method. 0.6.0; 0.18.0: the Fano term as a formula symbol.

X04 · A FOCUS project as a difes session

Exchange · osml-read

Statement. A FOCUS file is a model when it holds one dielectric function and at most one other definition (its reflectance, as FOCUS's examples do) and a project otherwise. A project is imported whole: every symbol and definition becomes a formula of the session's own library (saved inside the session file, so the language can call them by name), every dielectric-function definition becomes a model node, every other definition of x becomes an expression node evaluated on a grid, a definition of another variable (a spectral density $m(z)$) stays a formula only, and a view overlays the reflectances. Nothing of FOCUS's is changed; the session is the project as difes sees it.

$$\text{project} \mapsto \{\text{library entries}\} \cup \{\text{model nodes}\} \cup \{\text{expr nodes}\} \cup \{\text{view}\}$$

Why this choice. Whole-project import rather than model extraction, so that Irati's and Mireia's projects open as they are, with their helpers, and the receiving command (X09) can match every exported curve to the definition that produced it.

Error and its order. None; a translation. A definition of two variables is skipped and listed (T48).

Reference. FOCUS help, math-explorer.html and definition-wnd.html

Counterparts. FOCUS (inferred from numbers): Irati's Azkena and Bestebat projects and Mireia's Bergman project import and evaluate to FOCUS's digits (T06, T48). RefFIT (documented): Not applicable.

Implementation.

- `is_project(path: str | Path) -> bool` — `src/difes/osml.py:619`
- `import_project(session, path: str | Path, grid: list[float] | None = None, library_scope: str = session) -> dict[str, Any]` — `src/difes/osml.py:624`

Tests that name these symbols. `test_focus_exact.py::test_single_band_exports_settle_the_conventions`, `test_osml.py::test_focus_projects_of_irati_read_and_evaluate`.

Ledger records. T06 (FOCUS's OSML files: quartz E, LiNbO₃ E, the four libraries, the round trip); T48 (Mireia's alumina project: her Bergman model read from the OSML export, FOCUS's own curve met to 4e-6, her two complaints explained).

Check it yourself. The last printed line must be `True 5 5 True`.

```

from pathlib import Path
import difes
from difes.osml import is_project, import_project
from difes.graph import Session
ROOT = Path(difes.__file__).resolve().parents[2]
azk = ROOT / "examples/data/focus_projects/Azkena.sml"
s = Session(); made = import_project(s, azk)
print(is_project(azk), len(made["models"]), len(made["exprs"]), made["view"] is not None)

```

Printed True 5 5 True at this build, in 0.12 s.

Changelog of the method. 0.11.0.

X05 · FOCUS's binary project (.pjt): the numbers, found by their shape

Exchange · binary

Statement. A .pjt is FOCUS's own serialisation, a tagged tree whose model part (the formulas) is not decoded. The data part is plain: every spectrum is a run of little-endian doubles in triples (x, y, 0) with x increasing, every point list (the n, k tables of a file-2c, a Bergman density) a run of (x, y) pairs with x increasing, and every run follows the length-prefixed name of the node it belongs to. Runs of plausible doubles are found at the eight byte alignments, triples and pairs recognised by their shape, and each attributed to the nearest preceding node name. A curve FOCUS computed and cached (a grid starting at 0) is listed too, and the caller tells it apart by its first abscissa. Found against FOCUS's four sample projects and Mireia's alumina project.

$\text{spectrum} = \{(x_i, y_i, 0)\}_{x_i \uparrow}, \quad \text{table} = \{(x_i, y_i)\}_{x_i \uparrow}, \quad \text{name} = \text{last length-prefixed string before the run}$

Why this choice. Reading the data and not the model, because the data are what a colleague has and cannot otherwise export without opening FOCUS, while the model is asked for as an OSML export with declarations; the heuristic nature of the reading is stated and every run carries its offset so that a wrong attribution can be seen.

Error and its order. A heuristic reader: a run shorter than twenty points, or a table with fewer than thirty, is not found; a cached curve is listed next to the measured ones.

Reference. difes.pjt module (October 2026), from FOCUS's sample projects and Mireia's project (T48)

Counterparts. FOCUS (inferred from numbers): Mireia's seven spectra and four tables read from her .pjt and matched to her exports (T48). RefFIT (documented): RefFIT's binary model file is X07.

Implementation.

- `double_runs(b: bytes, min_len: int = 60) -> list[tuple[int, np.ndarray]]` — `src/difes/pjt.py:22`
- `spectra_in(path: str | Path) -> list[tuple[int, np.ndarray, np.ndarray]]` — `src/difes/pjt.py:43`
- `names_in(b: bytes) -> list[tuple[int, str]]` — `src/difes/pjt.py:66`
- `spectra(path: str | Path) -> list[tuple[str, np.ndarray, np.ndarray]]` — `src/difes/pjt.py:76`
- `tables(path: str | Path, min_points: int = 30) -> list[tuple[str, np.ndarray]]` — `src/difes/pjt.py:84`

Tests that name these symbols. `test_bench.py::test_spectrum_model_sample_simulate_are_tagged_nodes`, `test_bench.py::test_bench_reads_a_graph_built_session`, `test_bench.py::test_the_shared_chromium_session`, `test_bench.py::test_every_session_bundle_opens_with_relative_paths`, `test_bergman.py::test_mireia_osml_r`, `test_bergman.py::test_receive_mireia_export_folder`, `test_bergman.py::test_pjt_spectra_and_tables`, `test_cli.py::test_fit_command_recovers_parameters`, `test_cli.py::test_kk_and_cfr_commands`, `test_expr.py::test_user_function_defaults_keywords_and_nesting` and 11 more.

Ledger records. T48 (Mireia's alumina project: her Bergman model read from the OSML export, FOCUS's own curve met to 4e-6, her two complaints explained).

Check it yourself. The last printed line must be 14 R-LLL 4.

```
from pathlib import Path
import difes
from difes import pjt
ROOT = Path(difes.__file__).resolve().parents[2]
f = ROOT / "examples/data/mireia_al2o3/Bergman-alpha-Al3O2-200nm-Em-polycryst-160126.pjt"
sp, tb = pjt.spectra(f), pjt.tables(f)
print(len(sp), sp[0][0], len(tb))                                # fourteen runs (seven spectra
↳ and FOCUS's cached curves), four tables
```

Printed 14 R-LLL 4 at this build, in 0.27 s.

Changelog of the method. 0.30.0; 0.31.0: tables and cached curves told apart.

X06 · RefFIT’s model table and its codes, mapped to difes terms and back

Exchange · reffit-model

Statement. RefFIT’s dielectric-function model is EINF and rows (Wo, Wp, G): an ordinary row is a Lorentz oscillator (a Drude term when Wo = 0; a negative non-integer Wo is the oscillator at |Wo|, since RefFIT squares it), and a row whose Wo is a negative integer is a code naming a special function whose parameters are in the Wp and G columns of that row and, for some codes, of the following one. Handled: −9 (Drude with signed weight), −10 (a constant), −25/−26 and variants (Lorentz with signed weight), −27/−28 (a difference of two Lorentzians), −55/−56 (Tauc–Lorentz, P07), −65/−66 (Fano, P06), −1000/−1001/−1002 (the TO–LO factorised formula, paired one to one into four-parameter modes). The cuprate formulas, the gapped Drude, the power law, the electron–boson and Tanguy codes raise an error with their code and name. A negative EINF (which RefFIT allows in a fit) is kept. The reverse writes the rows difes can express.

$$(W_o, W_p, G) \mapsto \frac{W_p^2}{W_o^2 - \omega^2 - iG\omega}; \quad W_o = 0 \mapsto \text{Drude}; \quad W_o = -k \in \mathbb{Z}^- \mapsto \text{code } k$$

Why this choice. Term by term rather than through a formula, so that RefFIT’s models fit in difes with the same parameters and bounds (T42) and the ledger can compare χ^2 and uncertainties between the two fitters; the unsupported codes stop with a name, as the OSML reader does.

Error and its order. None; a translation, checked against RefFIT’s listings and the manual’s printed tables (T34, T42, T46).

Reference. RefFIT manual §4.6.2 and Table 4-3

Counterparts. FOCUS (documented): Not applicable. RefFIT (documented): The manual’s table of codes, confirmed on Irati’s and the tutorial’s files (T34, T42, T46, T47).

Implementation.

- `model_from_table(einf: float, rows: Sequence[Row], name: str = reffit, drude_below: float | None = 1.0) -> DielectricModel` — `src/difes/refit.py:78`
- `table_from_model(model: DielectricModel) -> tuple[float, list[Row]]` — `src/difes/refit.py:151`
- `RefFitError` — `src/difes/refit.py:43`

Tests that name these symbols. `test_refit.py::test_ordinary_rows_are_drude_and_lorentz`, `test_refit.py::test_signed_weight_codes_and_constant`, `test_refit.py::test_tolo_code_is_the_four_param`, `test_refit.py::test_unsupported_code_is_named`, `test_refit.py::test_fano_reduces_to_lorentz_and_is_cau`, `test_refit.py::test_tauc_lorentz_shape_and_causality`, `test_refit.py::test_quantities_follow_their_def`, `test_refit_output.py::test_rfm_active_flags_follow_the_macro_and_the_screenshots`, `test_refit_output.py::test_refits_weighted_chi_square_and_its_log_grid_reproduce_fig_3_25`.

Ledger records. T34 (RefFIT’s own output: the comparison protocol run by Irati, number against number); T42 (RefFIT’s own tutorial, Part 1: RefFIT’s data and saved models, and difes’s fitter against RefFIT’s); T46 (RefFIT’s own tutorial, Part 2: four datasets fitted together, RefFIT’s weighted χ^2 to the printed digit); T47 (RefFIT’s own tutorial, Part 4: the CuO phonon against temperature, the macro’s loop in difes against Fig. 3-30).

Check it yourself. The last printed line must be `['drude', 'lorentz', 'fano'] True`.

```
from difes.refit import model_from_table, table_from_model
rows = [(0.0, 2000.0, 150.0), (450.0, 600.0, 20.0), (-65.0, 300.0, 50.0), (-66.0, 800.0,
↪ 30.0)] # a Drude, a Lorentz, a Fano pair
m = model_from_table(3.0, rows)
print([t.kind for t in m.terms], table_from_model(m) == (3.0, rows))
```

Printed `['drude', 'lorentz', 'fano'] True` at this build, in 0.0 s.

Changelog of the method. 0.24.0; 0.27.0: the negative Wo and EINF conventions from Irati’s files.

X07 · RefFIT’s files: PAR.DAT in two layouts, the ASC listing, the binary RFM in two layouts, datasets and exports

Exchange · reffit-files

Statement. PAR.DAT: one line per saved model, name, χ^2 , Einf and the rows, either as the manual’s plain triples or, as RefFIT 1.3.05 writes them, each value interleaved with its error bar and one unnamed trailing number; both layouts are read and a code difes does not hold leaves the line read with its error noted. ASC: the text listing next to every saved model, ChiSq, Einf and per row Wo, Wp, G as value $\pm$ error, the errors zero unless RefFIT computed them. RFM: the binary model file, records of 232 bytes (RefFIT 1.3.05, the name at byte 40) or 218 bytes (the 2004 RefFIT of the tutorial, the name at byte 36), one record for Einf, an int32 with the row count, three records per row, the value a double at offset 0 of the record, the bounds two doubles before the name, the active flag an int32 before them (2018 layout only); the layout is read off the file from the spacing of the names and every record’s name is checked. Datasets: X Y Yerr in cm^{-1} with no header, the error 0.01 when the file has none, RefFIT’s own convention; a macro sets parameters and exports curves on a grid (XGrid = 1 linear, 2 logarithmic). Exports: two columns, further columns ignored. A spectral_emissivity file holds $R = 1 - E$. An ordinary row is a Lorentz oscillator, a Drude term when $Wo = 0$; since RefFIT squares Wo , a fit there can leave Wo tiny and of either sign (Irati’s T500.rfm: -0.049147), and such a row ($|Wo| < 1 \text{ cm}^{-1}$, DRUDE_BELOW) is read as the Drude term it is in effect, so that a refit in difes has no free resonance to walk up into the spectrum; the literal Lorentz at $|Wo|$ is kept available (drude_below=None) and agrees with it to the file’s digits on the measured range.

RFM record : value@0, lower@(n − 16), upper@(n − 8), active@(n − 24), name@n; $n \in \{40, 36\}$

Why this choice. The binary read because a colleague’s folder holds RFM files and not always the listings; two layouts because the tutorial’s 2004 files differ from Irati’s 2026 ones; the layout detected from the file rather than assumed, and every name verified, so that a third layout fails loudly. The tiny- Wo rule because the faithful Lorentz misrepresented RefFIT’s behaviour in a refit: her near-zero resonance, free with bounds $0.. \infty$ in difes, moved to 1029 cm^{-1} in the bundled comparison session, which RefFIT’s squared Wo never does.

Error and its order. None where the names verify; the trailing number of PAR.DAT is read and not interpreted.

Reference. RefFIT manual §4.4 (datasets), §4.5.1 (PAR.DAT), §4.10.2 (errors); the files of Irati and of the TUTORIAL folder (T34, T42)

Counterparts. FOCUS (documented): Not applicable. RefFIT (inferred from numbers): RFM contents checked against the ASC listings, PAR.DAT and the manual’s printed tables on every file in hand (T34, T42, T46, T47).

Implementation.

- `read_par_dat(path: str | Path) -> list[dict[str, Any]] — src/difes/reffit.py:183`
- `read_asc(path: str | Path) -> dict[str, Any] — src/difes/reffit.py:223`
- `read_rfm(path: str | Path) -> dict[str, Any] — src/difes/reffit.py:257`
- `write_dataset(spec: Spectrum, path: str | Path, yerr: float | None = None) -> Path — src/difes/reffit.py:367`
- `write_macro(path: str | Path, dataset: str, quantity: str, model: DielectricModel | None = None, model_file: str | None = None, export: Sequence[str] = (R, E1, E2), xmin: float | None = None, xmax: float | None = None, xpts: int = 1000, prefix: str = difes_) -> Path — src/difes/reffit.py:377`
- `read_export(path: str | Path) -> tuple[np.ndarray, np.ndarray] — src/difes/reffit.py:414`

Tests that name these symbols. `test_reffit.py::test_par_dat_and_protocol_files`, `test_reffit_output.py::test_reffit_output.py::test_rfm_active_flags_follow_the_macro_and_the_screenshots`, `test_reffit_output.py::test_iratis_models_regenerate_her_graph_exports`, `test_reffit_output.py::test_a_t`, `test_reffit_output.py::test_the_2004_rfm_layout_holds_the_manuals_printed_tables`, `test_reffit_output.py`

Ledger records. T34 (RefFIT’s own output: the comparison protocol run by Irati, number against number); T42 (RefFIT’s own tutorial, Part 1: RefFIT’s data and saved models, and difes’s fitter against RefFIT’s);

T46 (RefFIT's own tutorial, Part 2: four datasets fitted together, RefFIT's weighted χ^2 to the printed digit); T47 (RefFIT's own tutorial, Part 4: the CuO phonon against temperature, the macro's loop in difes against Fig. 3-30).

Check it yourself. The last printed line must be 2018 2004 2 True.

```
import numpy as np
from pathlib import Path
import difes
from difes.reffit import read_asc, read_rfm
ROOT = Path(difes.__file__).resolve().parents[2]
d = ROOT / "examples/data/reffit_irati/Adibideai-T500" # Irati's chromium at
    ↪ 500 C, RefFIT 1.3.05
a, r = read_asc(d / "T500.asc"), read_rfm(d / "T500.rfm")
old = read_rfm(ROOT / "examples/data/reffit_tutorial/Part1/Model1.rfm") # the 2004
    ↪ RefFIT of the tutorial
print(r["layout"], old["layout"], r["n_rows"], bool(np.allclose(r["rows"], a["rows"])))
    ↪ # the binary agrees with the listing
```

Printed 2018 2004 2 True at this build, in 0.0 s.

Changelog of the method. 0.24.0: datasets, macros, PAR.DAT; 0.27.0: ASC and RFM; 0.28.0: the 2004 layout; 0.34.0: a tiny |Wo| read as a Drude term.

X08 · Numeric text tables in and out: delimiters, headers, column selection, exponents, error bars

Exchange · text-files

Statement. A text table is split on whitespace, commas, semicolons and tabs; lines that do not start with a number are skipped and counted as header lines; Fortran exponents (1.0D-3) are accepted; decimal commas are not. Without a column selection the first two or three numeric fields are x, y and the error bar; with 1-based column indices any line whose selected fields are numeric is kept, so a wide CSV with a label column is read. The unit of x is converted on the way in (P16) and the source, unit, header count and columns used go into the metadata. The writer emits x in the requested unit, y and the error bar with ten significant digits and a one-line header.

$$\text{row} = (x, y[\cdot], \sigma), \quad x \mapsto \tilde{\nu} [\text{cm}^{-1}]$$

Why this choice. Tolerant on the way in (the group's files come from OPUS exports, FOCUS exports, RefFIT exports and spreadsheets, each with its own header) and strict on the way out, so that a difes file is unambiguous.

Error and its order. Ten significant digits on output, below any measurement; a label column that looks numeric would be read as data, which the column selection avoids.

Reference. numpy.savetxt; FOCUS help, data-files.html

Counterparts. FOCUS (documented): FOCUS's data files are columns with an optional header (help, data-files.html). RefFIT (documented): RefFIT's X Y Yerr datasets (X07).

Implementation.

- `read_text(path: str | Path, kind: Kind | str = <Kind.RAW: raw>, unit: Unit | str = <Unit.CM1: cm-1>, temperature: float | None = None, name: str | None = None, columns: tuple[int, ...] | None = None) -> Spectrum` — src/difes/spectra.py:260
- `write_text(spec: Spectrum, path: str | Path, unit: Unit | str = <Unit.CM1: cm-1>, header: bool = True) -> None` — src/difes/spectra.py:282

Tests that name these symbols. `test_bench.py::test_invert_returns_epsilon`, `test_bench.py::test_layers_an`, `test_bergman.py::test_receive_mireia_export_folder`, `test_cli.py::test_simulate_then_info_then_convert`,

test_cli.py::test_simulate_transmission_needs_thickness, test_cli.py::test_fit_command_recovers_paramet
test_cli.py::test_model_describe, test_cli.py::test_kk_and_cfr_commands, test_digitize.py::test_write_c
test_focus_samples.py::test_written_files_load_as_spectra and 24 more.

Ledger records. T05 (The software around the physics: language, spectra, sessions, server, Bench).

Check it yourself. The last printed line must be 2 True 2 R.

```
import os, tempfile
from difes.spectra import read_text
p = os.path.join(tempfile.mkdtemp(), "r.csv")
open(p, "w").write("wavenumber,R,err\n# a comment\n100,0.5,0.01\n200,0.6,0.01\n")
s = read_text(p, kind="R")
print(len(s), s.yerr is not None, s.meta["header_lines"], s.kind.value)
```

Printed 2 True 2 R at this build, in 0.0 s.

Changelog of the method. 0.1.0; 0.4.0: column selection.

X09 · Receiving a colleague's folder: inventory, classification, matching by name or by numbers, refits, a report

Exchange · receiving

Statement. A folder as a FOCUS or RefFIT user exports it, in any names: the OSML projects are imported (X04), the binary projects' spectra read (X05), the RefFIT files read (X07), and every text table classified as an exported curve or a measured spectrum by its name and its third column (an uncertainty when positive and small against the values, an imaginary part when not). Every exported curve is matched to the definition that produced it, by name when a definition's name is in the file's and otherwise by evaluating every real-valued definition on the export's grid and taking the closest, the runner-up reported so that an ambiguous match is visible; the difference is measured against the file's own rounding (seven digits, propagated through the curve's slope). Each measured spectrum is then fitted with FOCUS's model from FOCUS's parameters and from a generic start (frequencies 4 % off, dampings 1.6 times, strengths 0.9 times, bounds opened). Out come a session, a Markdown and a JSON report and a figure. Nothing of the colleague's numbers is changed.

$$\text{match}(c) = \arg \min_d \|c - d(x_c)\|_{\text{rms}}, \quad \text{bound}(x, y) = |y'| \frac{1}{2} 10^{-6} |x| + \frac{1}{2} 10^{-6} |y|$$

Why this choice. One command for the day a colleague sends a folder, so that the demonstration in front of Iñigo is a single step; matching by the numbers because FOCUS users name their exports freely; the rounding bound so that agreement is judged against what the file can express.

Error and its order. A match is a choice; the runner-up and the ratio to the rounding bound are printed so that a wrong one is visible. The refit from a generic start can stop in another minimum, which the report shows.

Reference. The manual page Receiving a colleague's folder; T48 for the run on Mireia's folder

Counterparts. FOCUS (inferred from numbers): Run on FOCUS's quartz and phosphate folders, Irati's Azkena export and Mireia's folder (T15, T16, T48). RefFIT (inferred from numbers): Run on Irati's RefFIT protocol folder (T34).

Implementation.

- `receive(folder: str | Path, out: str | Path | None = None, sigma: float = 0.005, refit: bool = True, grid_points: int = 1200) -> Received` — `src/difes/receive.py:195`
- `Received(folder: Path, out: Path, projects: list[Path] = <factory>, rfms: list[Path] = <factory>, pjts: list[Path] = <factory>, inputs: list[Path] = <factory>, tables: dict[Path, np.ndarray] = <factory>, images: list[Path] = <factory>, other: list[Path] = <factory>, comparisons: list[dict[str, Any]] = <factory>, data: list[dict[str, Any]] = <factory>, notes: list[str] = <factory>, reffit_models: list[dict[str, Any]] = <factory>, session_path: Path | None = None) -> None` — `src/difes/receive.py:85`

- `inventory(folder: Path, rec: Received) -> None` — `src/difes/receive.py:109`
- `render_report(rec: Received) -> str` — `src/difes/receive.py:449`
- `draw(rec: Received, path: Path) -> None` — `src/difes/receive.py:491`

Tests that name these symbols. `test_bergman.py::test_receive_mireia_export_folder`, `test_bergman.py::test_receive_mireia_full_folder`, `test_receive.py::test_focus_sample_project_folder`, `test_receive.py::test_refits_stay_on_focus_values_for_the_phosphate_glass`, `test_receive.py::test_an_exp`, `test_receive.py::test_a_refit_folder`.

Ledger records. T15 (FOCUS’s sample projects: the spectra read from the binary .pjt files); T16 (Quartz A2: five four-parameter modes against FOCUS’s exported curve); T34 (RefFIT’s own output: the comparison protocol run by Irati, number against number); T48 (Mireia’s alumina project: her Bergman model read from the OSM export, FOCUS’s own curve met to 4e-6, her two complaints explained).

Check it yourself. The last printed line must be `[2.51e-07 2.51e-07]`.

```
import numpy as np
from difes.receive import _rounding_bound
print(_rounding_bound(np.array([1000.0, 2000.0]), np.array([0.5, 0.5]))) # half a unit
↳ in the seventh digit of x and y, plus 1e-9
```

Printed `[2.51e-07 2.51e-07]` at this build, in 0.0 s.

Changelog of the method. 0.29.0; 0.31.0: binary projects, cached curves and tables told apart.

X10 · Curves out of published figures: colour masks on a calibrated bitmap, or the vector paths of a PDF page

Exchange · digitising

Statement. A bitmap (PNG, JPEG): the plot frame is found, the pixels of a named colour (eight masks on the RGB values) are taken, in line mode one y per x column, and the pixel coordinates are mapped to the axes by the user’s two-point calibration of each axis. A PDF page: the drawn polylines are read with PyMuPDF, split into curves, and calibrated on the tick labels the user names, so that a vector figure gives the author’s numbers to the width of the line. Both return plain arrays; the digitised precision (about 5×10^{-3} of the axis for a bitmap) enters the uncertainty budget as a per-point Type B component (M03).

$$x = x_0 + (p_x - p_{x0}) \frac{x_1 - x_0}{p_{x1} - p_{x0}}, \quad \text{likewise } y; \quad \text{PDF: axis map from tick-label centres}$$

Why this choice. Vector paths first wherever a PDF exists (class II of the ledger: exact to the line), bitmaps as the fallback (class III); the colour masks are fixed thresholds, stated here, rather than a classifier, so that a result can be reproduced from the mask’s definition.

Error and its order. Bitmaps: about 5×10^{-3} of the axis span; vector figures: the line’s own width and the author’s rounding.

Reference. The ledger’s classes II and III; the manual page Digitising figures

Counterparts. FOCUS (documented): Not applicable. RefFIT (documented): Not applicable.

Implementation.

- `frame_of(arr: np.ndarray) -> tuple[int, int, int, int]` — `src/difes/digitize.py:48`
- `colour_census(arr: np.ndarray, L: int, R: int, T: int, B: int) -> list[dict[str, Any]]` — `src/difes/digitize.py:58`
- `extract_bitmap(path: str | Path, x: tuple[float, float], y: tuple[float, float], color: str = red, mode: str = line, xunit: str = cm^-1, kind: str = E, exclude: list[list[float]] | None = None, inset: int = 3, frame: tuple[int, int, int, int] | None = None, overlay: bool = True) -> dict[str, Any]` — `src/difes/digitize.py:69`
- `pdf_page_info(path: str | Path, page: int, dpi: int = 70) -> dict[str, Any]` — `src/difes/digitize.py`

- `extract_pdf(path: str | Path, page: int, xlabel: list[str], ylabel: list[str], color: tuple[float, float, float], kind: str = R, decimal: str = ., longest: bool = True, minseg: int = 50, split: float | None = None, names: list[str] | None = None, xunit: str = cm-1) -> list[dict[str, Any]]` — `src/difes/digitize.py:158`
- `inspect(path: str | Path, page: int = 1) -> dict[str, Any]` — `src/difes/digitize.py:207`
- `write_curve(path: str | Path, x: list[float], y: list[float], kind: str, source: str) -> Path` — `src/difes/digitize.py:220`
- `DigitizeError` — `src/difes/digitize.py:34`

Tests that name these symbols. `test_digitize.py::test_bitmap_matches_the_script_extraction`, `test_digitize.py::test_pdf_page_info_lists_labels_and_colours`, `test_digitize.py::test_pdf_curve_matches`, `test_digitize.py::test_pdf_split_names_six_curves`, `test_digitize.py::test_errors_are_spoken`, `test_digitize.py::test_write_curve`, `test_digitize.py::test_bitmap_precision_on_a_known_curve`, `test_registry.py::test_numerics_modules_are_covered`.

Ledger records. T12 (The fluorides of Passerat de Silans et al. (2009): every table computed forward); T13 (Sapphire, the same paper: four phonons with Gaussian self-energies).

Check it yourself. The last printed line must be `True False ['black', 'blue', 'cyan', 'green', 'grey', 'magenta', 'orange', 'red']`.

```
from difes.digitize import COLORS
print(bool(COLORS["red"])(200, 10, 10), bool(COLORS["red"])(10, 200, 10), sorted(COLORS))
```

Printed `True False ['black', 'blue', 'cyan', 'green', 'grey', 'magenta', 'orange', 'red']` at this build, in 0.0 s.

Changelog of the method. 0.13.0: bitmaps; 0.23.0: PDF vector paths.

What the registry says about FOCUS

FOCUS’s help describes its automatic fit as “a non-linear least square optimization method” and names no algorithm anywhere; none of the eighteen papers in the literature audit names one either. The program’s author has said in conversation that he coded from *Numerical Recipes*, whose licence restricts the code and not the algorithms, which are public. So the minimiser of FOCUS is recorded here as *unknown*, and next to it the ledger’s finding that difes’s fitter lands on the same minima as FOCUS on every project where FOCUS’s numbers exist. Where a FOCUS method could be identified from its numbers (the slopes of its monotone cubic, the normalisation of its causal Voigt band, the approximation in its Voigt function) the card says so and cites the record. difes’s own borrowed routines are listed with their algorithm, citation, licence and version. That is what independence from FOCUS means in practice: a program that says what it does, and a document that says where it does not know what the other one did.

Everything in this document is on the project site under Registry, with the cards, the JSON and the checks.